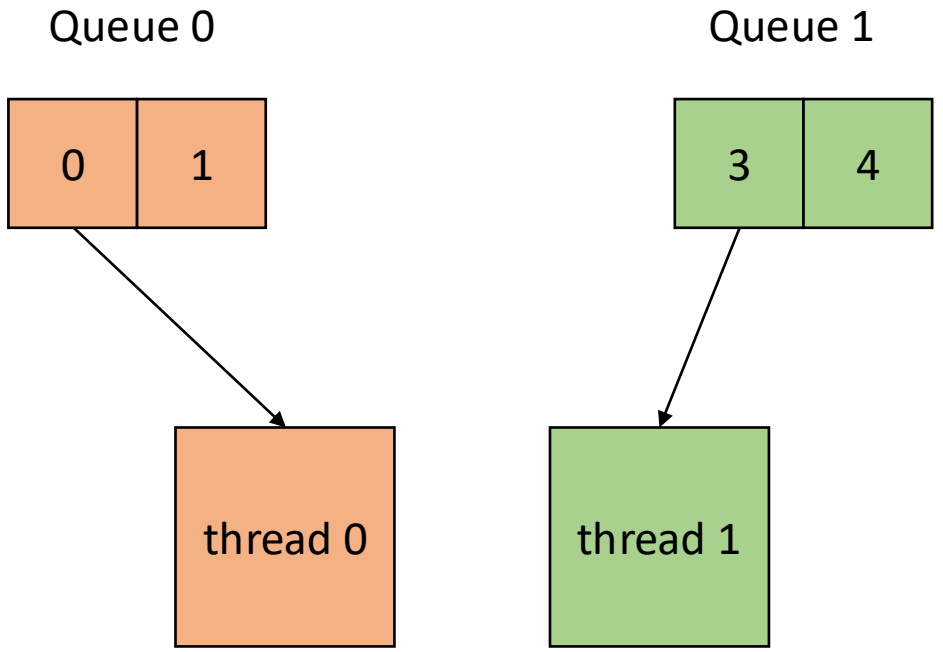
# CSE113: Parallel Programming

- **Topics**:
  - Concurrent Queues
  - DOALL loops
  - Static scheduling
  - Workstealing

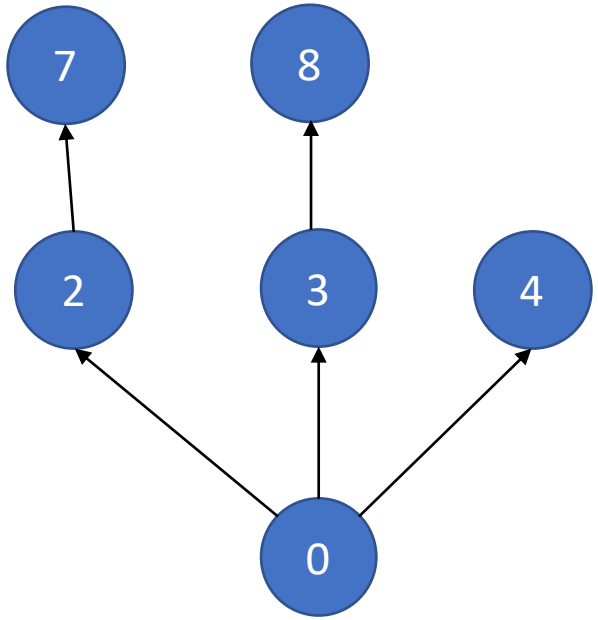


# Announcements

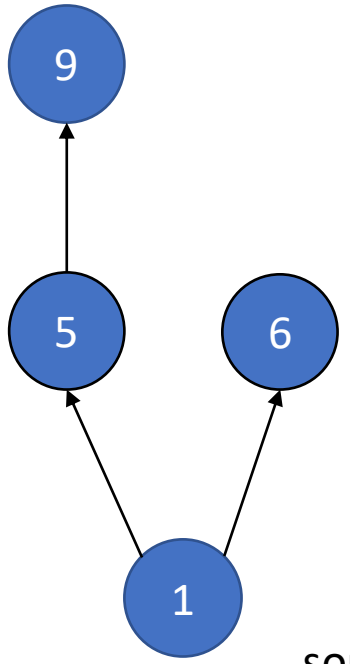
- Midterm is being graded
- HW 2 graded this week.
- HW 3 due this week
  - You should be able to do part 2 after today's lecture.
  - If we cannot finish by then, we might extend the deadline.

# Input/Output Queues

- Example: Information flow in graph applications:




source 0




source 1

*and so on...*

output

queue 1

input

| 7 | 9 | 8 | | |
|---|---|---|---|---|

queue 0

# Implementation

# Implementation

Allocate a contiguous array



Pros:
?

Cons:
?

# Implementation

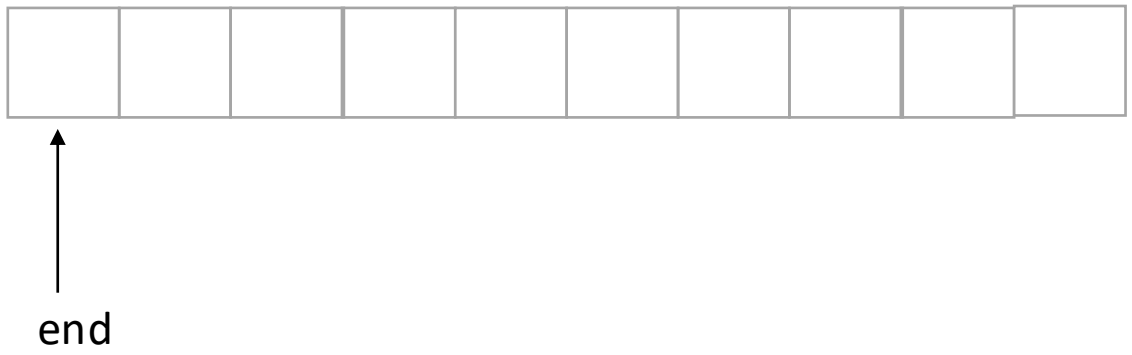
Allocate a contiguous array



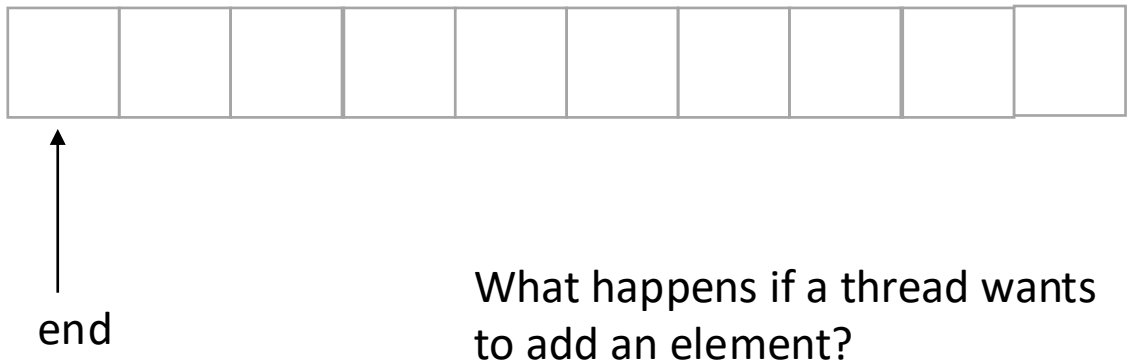
Pros:
+ fast!
+ we can use indexes instead of addresses

Cons:
- Fixed size. We need to reason about overflow!

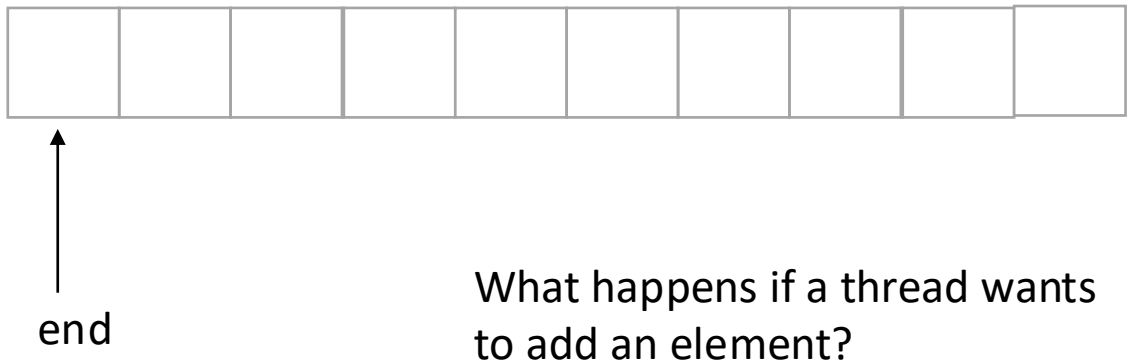
# Implementation



# Implementation



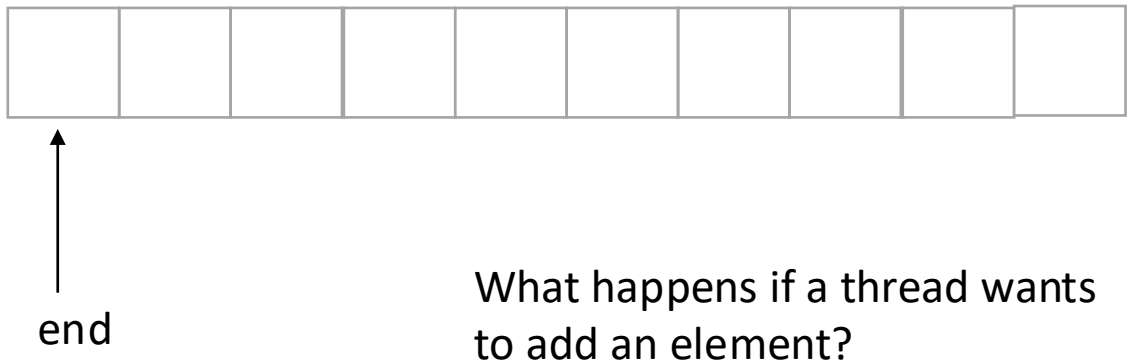
# Implementation




What happens if a thread wants to add an element?

Think sequentially:

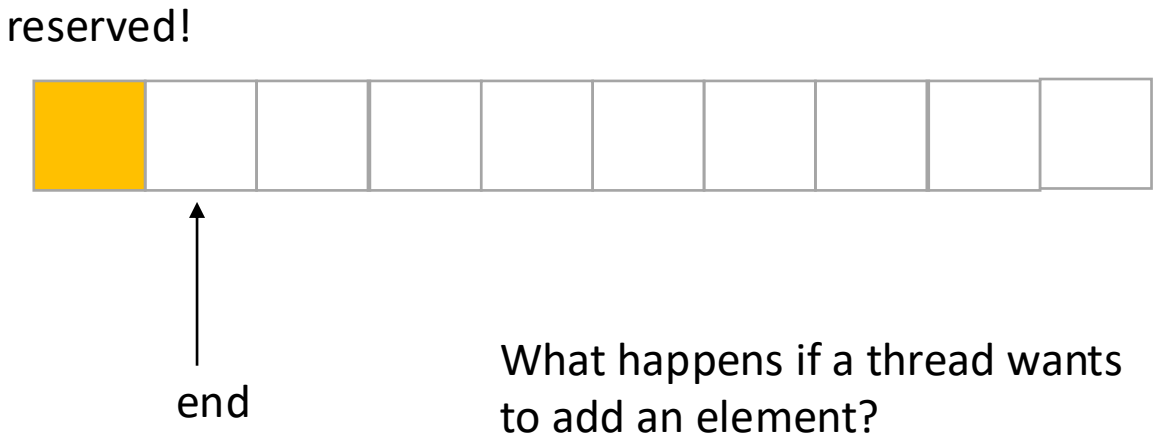
# Implementation




What happens if a thread wants to add an element?

Think sequentially:
*reserve a space - increment end

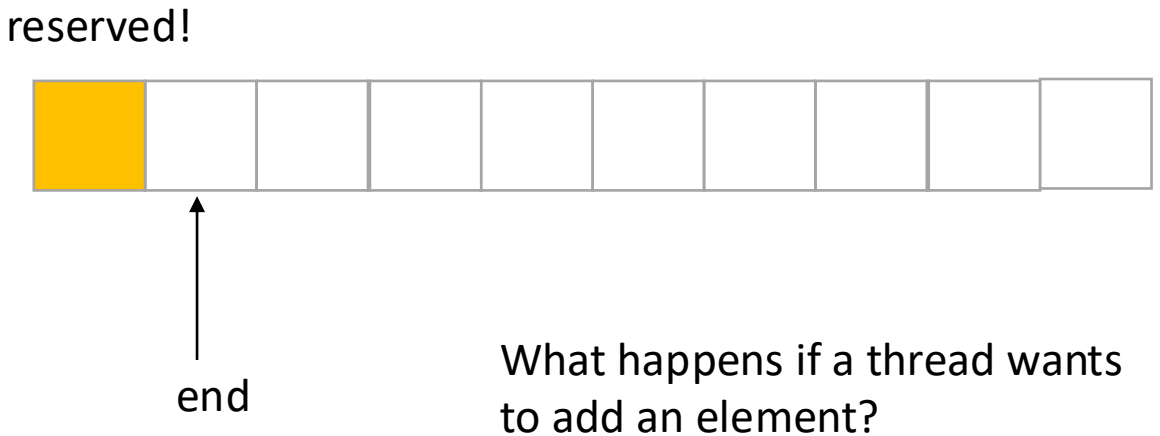
# Implementation




What happens if a thread wants to add an element?

Think sequentially:
*reserve a space - increment end

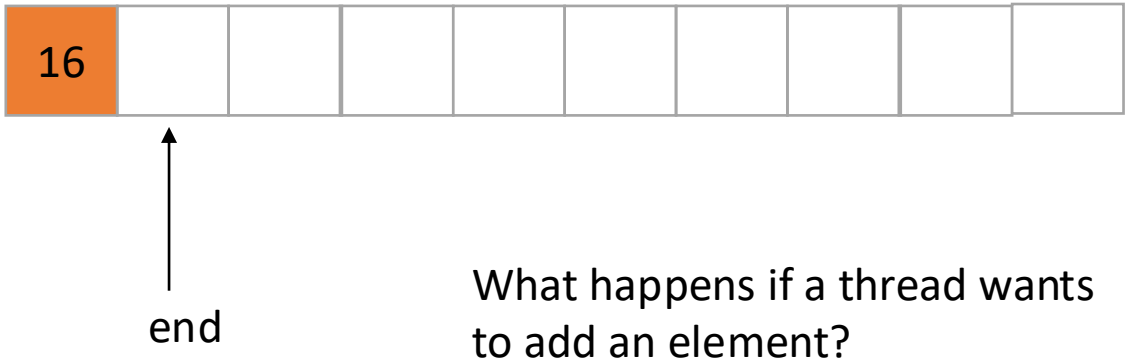
# Implementation




What happens if a thread wants to add an element?

Think sequentially:
* reserve a space - increment end
* add the element

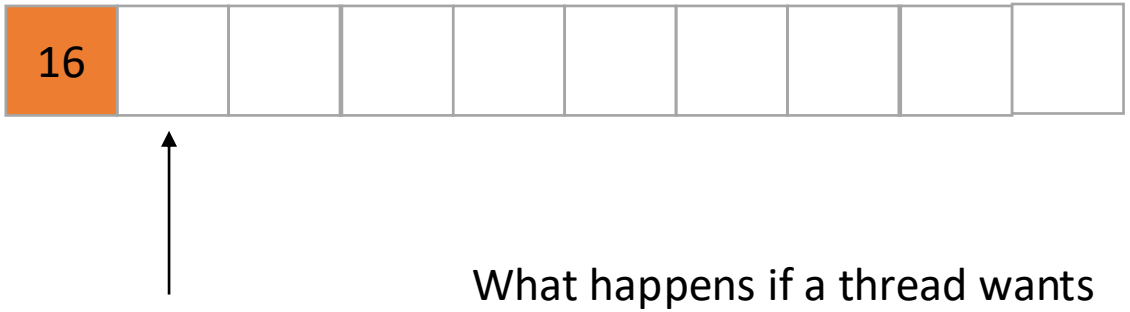
# Implementation




What happens if a thread wants to add an element?

Think sequentially:
* reserve a space - increment end
* add the element

# Implementation

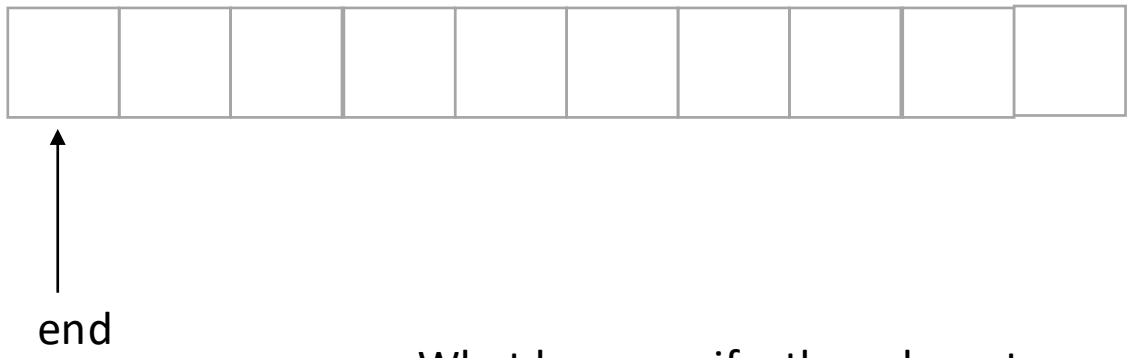



What happens if a thread wants to add an element?

Think sequentially:
* reserve a space - increment end
* add the element

done!

# Implementation

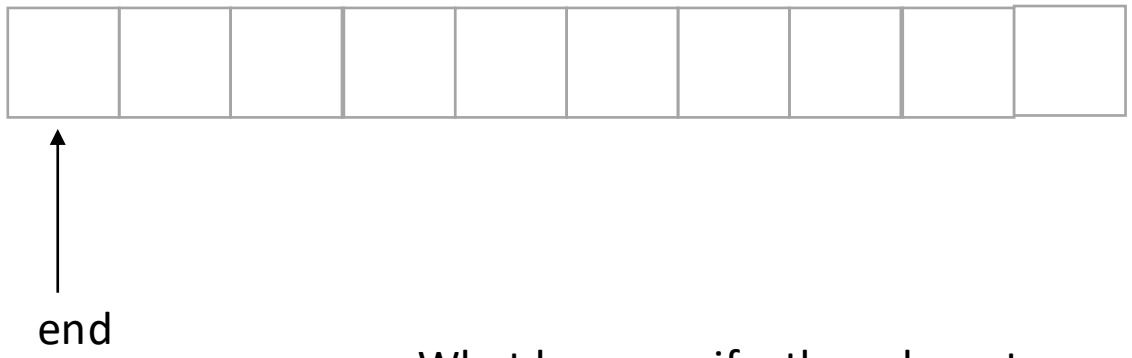



What happens if a thread wants
to add an element?

Think concurrently:

*Two threads cannot reserve the same space!*
*We've seen this before*

# Implementation

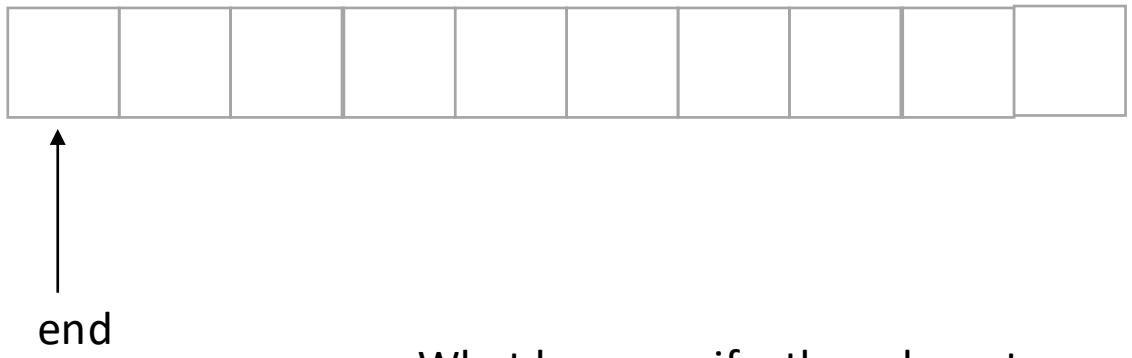



What happens if a thread wants to add an element?

Think concurrently:

```
reserved_index = atomic_fetch_add(&end, 1);
```

# Implementation




Thread 0:
*enq(6);*

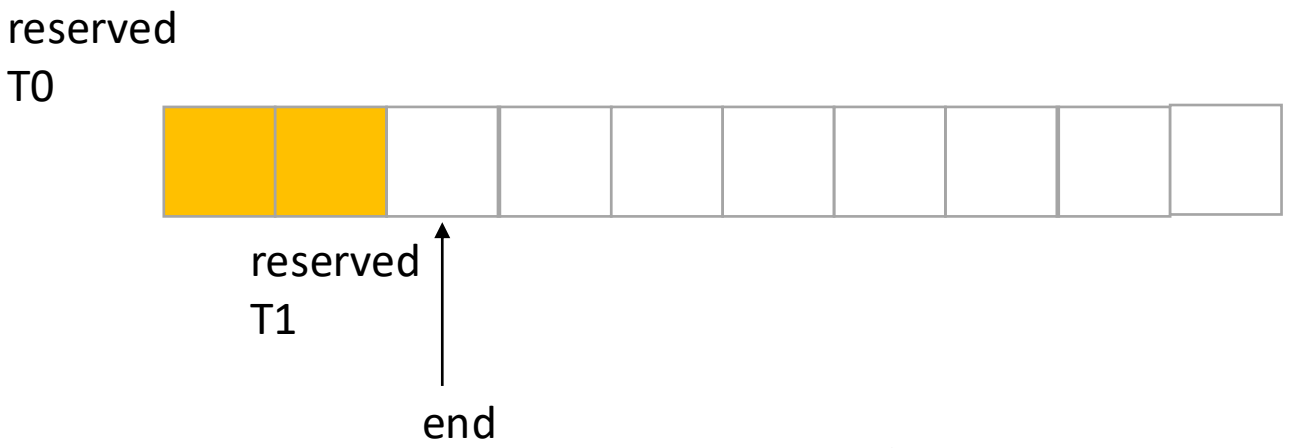
Thread 1:
*enq(7);*

What happens if a thread wants to add an element?

Think concurrently:

```
reserved_index = atomic_fetch_add(&end, 1);
```

# Implementation




Thread 0:
*enq(6);*

Thread 1:
*enq(7);*

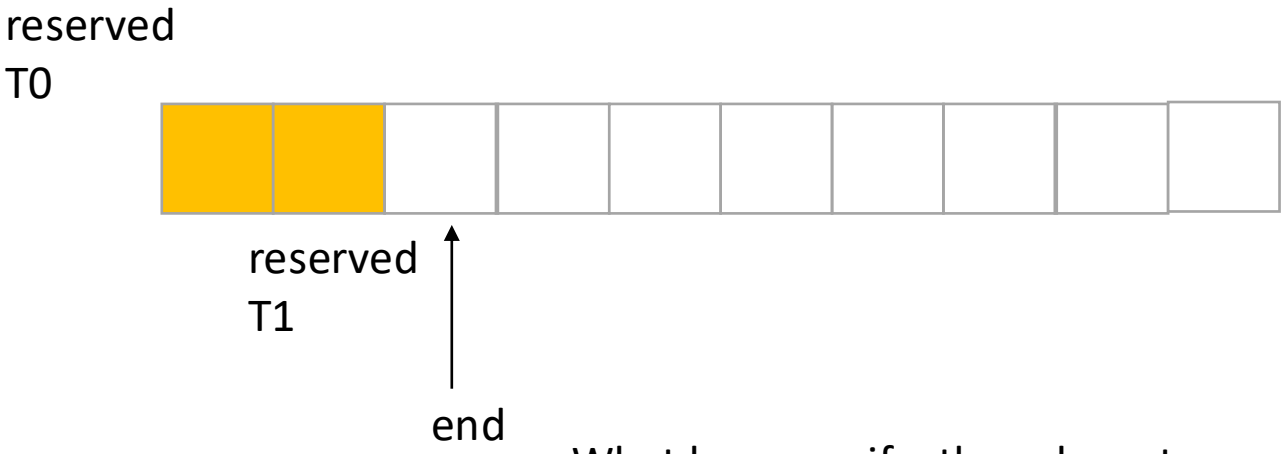
What happens if a thread wants to add an element?

Think concurrently:

```
reserved_index = atomic_fetch_add(&end, 1);
```

# Implementation

*does it matter which order threads add their data?*




Thread 0:
*enq(6);*

Thread 1:
*enq(7);*

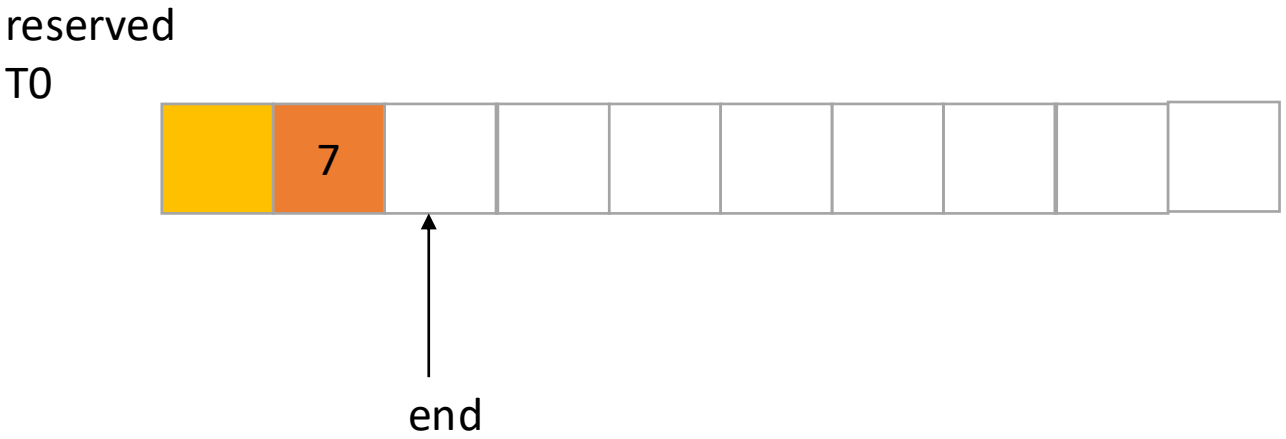
What happens if a thread wants to add an element?

Think concurrently:

```
reserved_index = atomic_fetch_add(&end, 1);
```

# Implementation

*does it matter which order threads add their data?*




Thread 0:
*enq(6);*

Thread 1:
*enq(7);*

What happens if a thread wants to add an element?

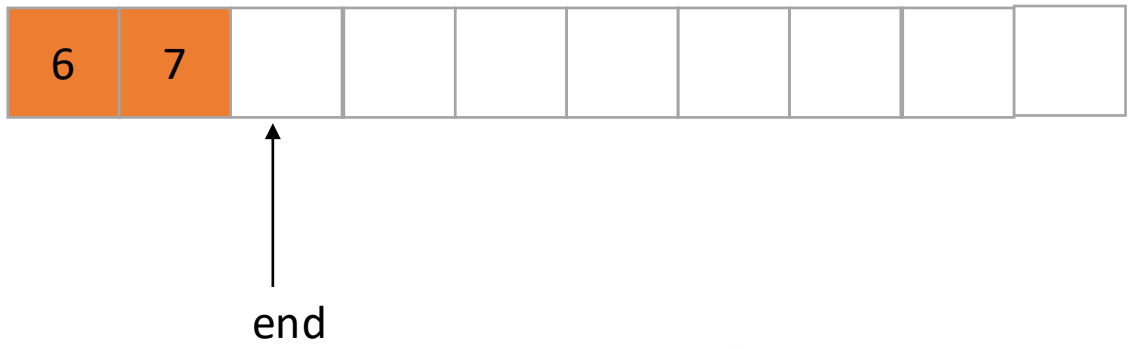
Think concurrently:

```
reserved_index = atomic_fetch_add(&end, 1);
```

# Implementation

*does it matter which order threads add their data? No! The are concurrent. Both linearization orders acceptable.*

reserved
T0




Thread 0:
*enq(6);*

Thread 1:
*enq(7);*

What happens if a thread wants to add an element?

Think concurrently:

```
reserved_index = atomic_fetch_add(&end, 1);
```

```
class InputOutputQueue {
  private:
    atomic_int end;
    int list[SIZE];

  public:
     InputOutputQueue() {
        end = 0;
     }

     void enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        list[reserved_index] = x;
     }

     int size() {
        return end.load();
     }
 }
```

```
class InputOutputQueue {
  private:
    atomic_int end;
    int list[SIZE];

  public:
     InputOutputQueue() {
        end = 0;
     }

     void enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        list[reserved_index] = x;
     }

     int size() {
        return end.load();
     }
 }
```

How to protect against overflows?

```
class InputOutputQueue {
  private:
    atomic_int end;
    int list[SIZE];

  public:
    InputOutputQueue() {
       end = 0;
    }

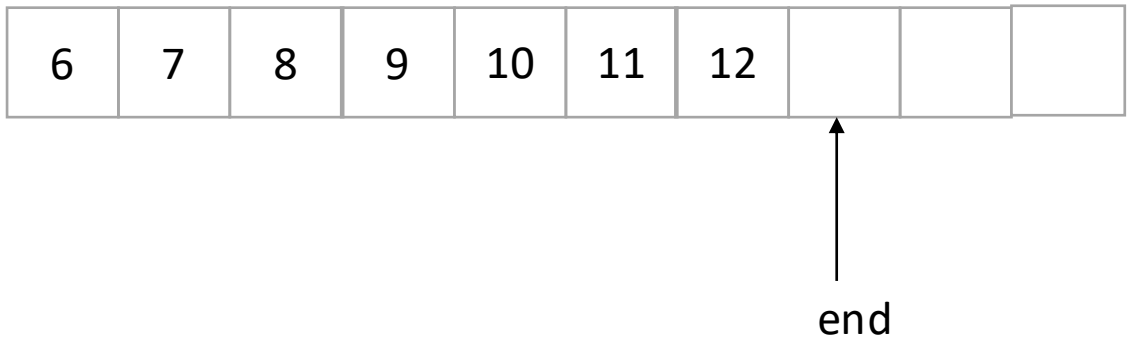
     boolean enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        if (reserved_index >= SIZE) {
           atomic_fetch_sub(&end, 1); // give the reservation back
           return false;
        }
        list[reserved_index] = x;
        return true;
     }

     int size() { return min(end.load(), SIZE); }
 }
```

Because end can be momentarily larger than SIZE.

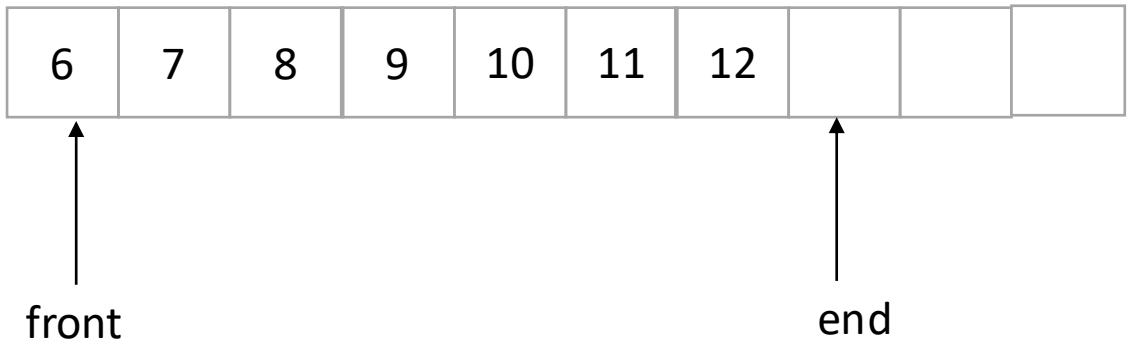
# What about Input?

- Now we only do deqs



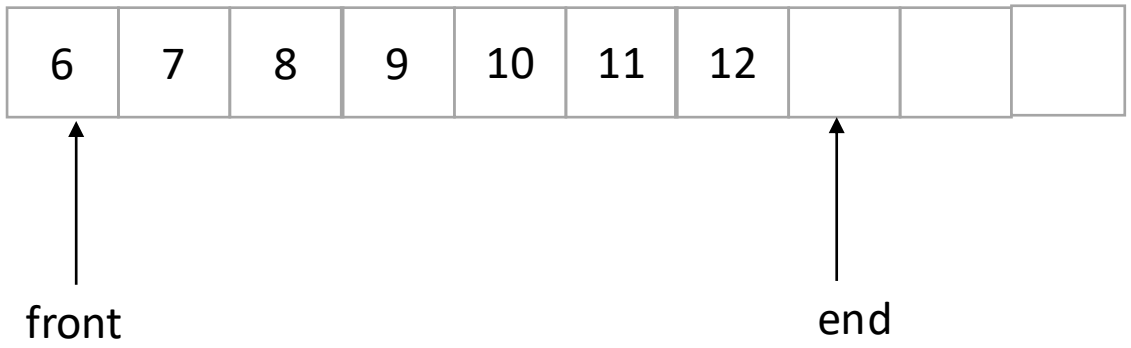
# What about Input?

- Now we only do deqs



# What about Input?

- Now we only do deqs



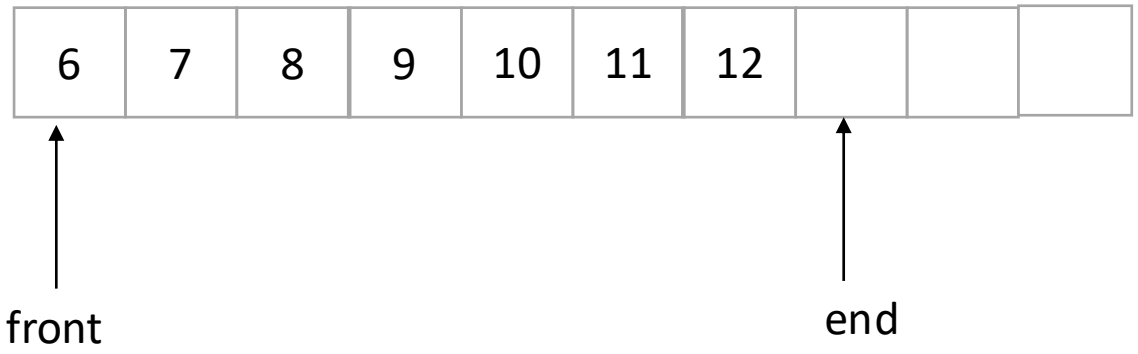

What happens if a thread wants to dequeue an element?

Think concurrently:

```
data_index = atomic_fetch_add(&front, 1);
```

# What about Input?

- Now we only do deqs




Thread 0:
*deq();*

Thread 1:
*deq();*

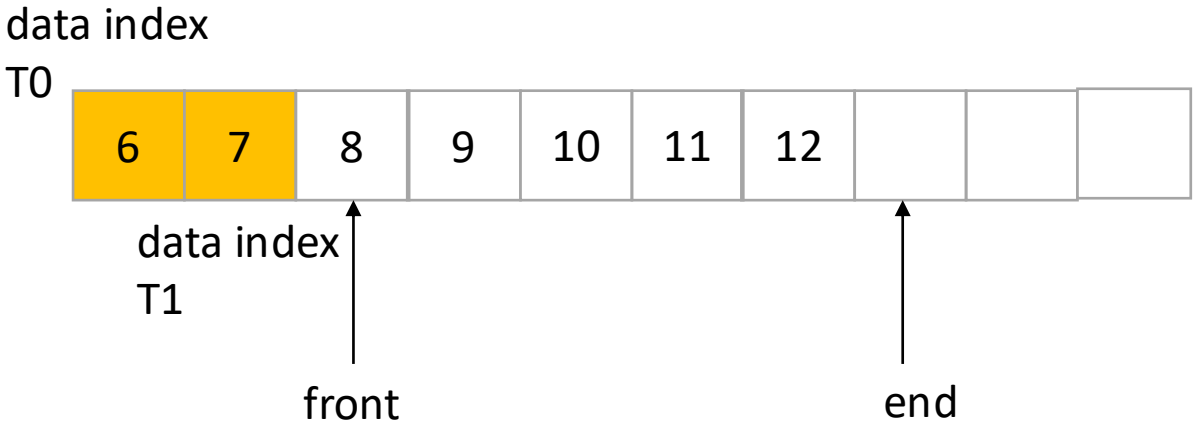
What happens if a thread wants to dequeue an element?

Think concurrently:

```
data_index = atomic_fetch_add(&front, 1);
```

# What about Input?

- Now we only do deqs




Thread 0:
*deq();*

Thread 1:
*deq();*

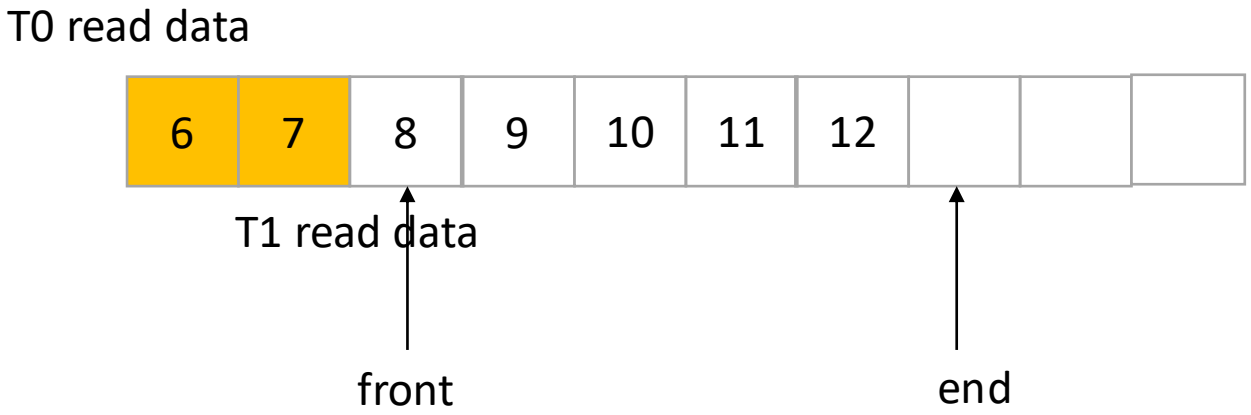
What happens if a thread wants to dequeue an element?

Think concurrently:

```
data_index = atomic_fetch_add(&front, 1);
```

# What about Input?

- Now we only do deqs




Thread 0:
*deq(); // reads 6*

Thread 1:
*deq(); // reads 7*

What happens if a thread wants to dequeue an element?

Think concurrently:

```
data_index = atomic_fetch_add(&front, 1);
```

```
class InputOutputQueue {
  private:
    atomic_int front;
    atomic_int end;
    int list[SIZE];

  public:
    InputOutputQueue() {
        front = end = 0;
     }

     void enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        list[reserved_index] = x;
     }

     void deq() {
       int reserved_index = atomic_fetch_add(&front, 1);
       return list[reserved_index];
     }

     int size() {
        return ??;
     }
 }
```

```
class InputOutputQueue {
  private:
    atomic_int front;
    atomic_int end;
    int list[SIZE];

  public:
    InputOutputQueue() {
        front = end = 0;
     }

     void enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        list[reserved_index] = x;
     }

     void deq() {
       int reserved_index = atomic_fetch_add(&front, 1);
       return list[reserved_index];
     }

     int size() {
        return ??;
     }
 }
```

How about size?

```
class InputOutputQueue {
  private:
    atomic_int front;
    atomic_int end;
    int list[SIZE];

  public:
    InputOutputQueue() {
        front = end = 0;
     }

     void enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        list[reserved_index] = x;
     }

     void deq() {
       int reserved_index = atomic_fetch_add(&front, 1);
       return list[reserved_index];
     }

     int size() {
        return end.load() - front.load();
     }
 }
```

how about size?

how do we reset?

```
class InputOutputQueue {
  private:
    atomic_int front;
    atomic_int end;
    int list[SIZE];

  public:
    InputOutputQueue() {
        front = end = 0;
     }

     void enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        list[reserved_index] = x;
     }

     void deq() {
       int reserved_index = atomic_fetch_add(&front, 1);
       return list[reserved_index];
     }

     int size() {
        return end.load() - front.load();
     }
 }
```

how about size?

how do we reset?
Reset front and end

```
class InputOutputQueue {
  private:
    atomic_int front;
    atomic_int end;
    int list[SIZE];

  public:
    InputOutputQueue() {
        front = end = 0;
     }

     void enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        list[reserved_index] = x;
     }

     void deq() {
       int reserved_index = atomic_fetch_add(&front, 1);
       return list[reserved_index];
     }

     int size() {
        return end.load() - front.load();
     }
 }
```

how about size?

how do we reset?
Reset front and end

does the list need
to be atomic?

```
class InputOutputQueue {
  private:
    atomic_int front;
    atomic_int end;
    int list[SIZE];

  public:
    InputOutputQueue() {
        front = end = 0;
     }

     void enq(int x) {
        int reserved_index = atomic_fetch_add(&end, 1);
        list[reserved_index] = x;
     }

     void deq() {
       int reserved_index = atomic_fetch_add(&front, 1);
       return list[reserved_index];
     }

     int size() {
        return end.load() - front.load();
     }
 }
```

how about size?

how do we reset?
Reset front and end

does the list need
to be atomic?
No, its elements are
not concurrently accessed

# Quiz 9.

Input/output queues use atomic increments and decrements to protect against threads that are trying to concurrently enqueue and dequeue

*Select one*

A. True

B. False

Scan to answer

**forms.gle/B93D4MK5Vdf3z4426**

chart: docs.google.com/forms/d/e/1FAIpQLSdlLCu00P27FbaKSWUB0RW0SeHHMdedLIPQ_PquwfUE671gxA/viewanalytics

# Producer Consumer Queues

- 1 thread enqueues, 1 thread dequeues
  - enq’er cannot deq
  - deq’er cannot enq

- Example: printf:
  - your program enqueues values to print
  - the terminal process dequeues values and prints them

# Quiz 9.

The performance of an application using a producer-consumer queue depends most on:

*Select one*

A. If the queue is implemented using mutex or not

B. The rate at which the consumer enqueues elements

C. The rate at which the producer enqueues elements



Scan to answer

**forms.gle/5wAVCbhdhbkkpdpd6**

chart: docs.google.com/forms/d/e/1FAIpQLSfnVotYqFnn7UF8_NBYAe3LW6AaSSSLwrwTCH8npRpk_oyIeA/viewanalytics

# Synchronous Producer Consumer Queues

- First implementation:
  - Synchronous
  - Slow
  - Good for debugging

# Synchronous Producer Consumer Queues

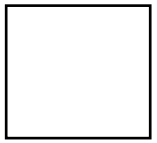
- First implementation:
  - Synchronous
  - Slow
  - Good for debugging

- enq does not return until value is deq’ed

# Synchronous Producer Consumer Queues

Producer Thread

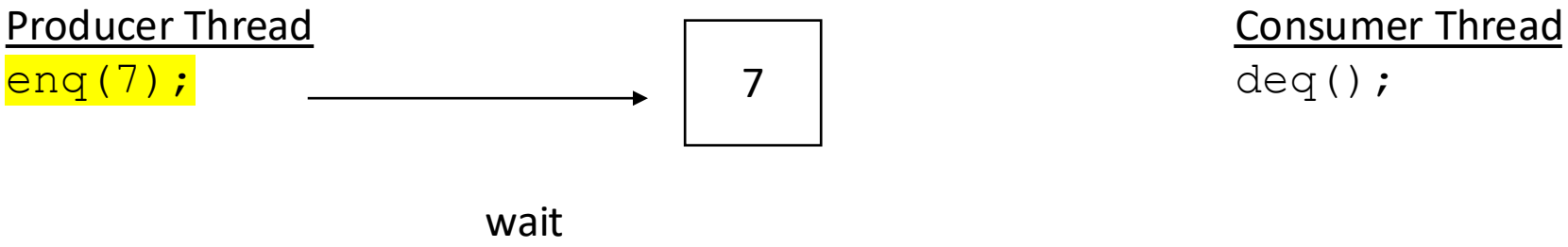
```
enq(7);
```



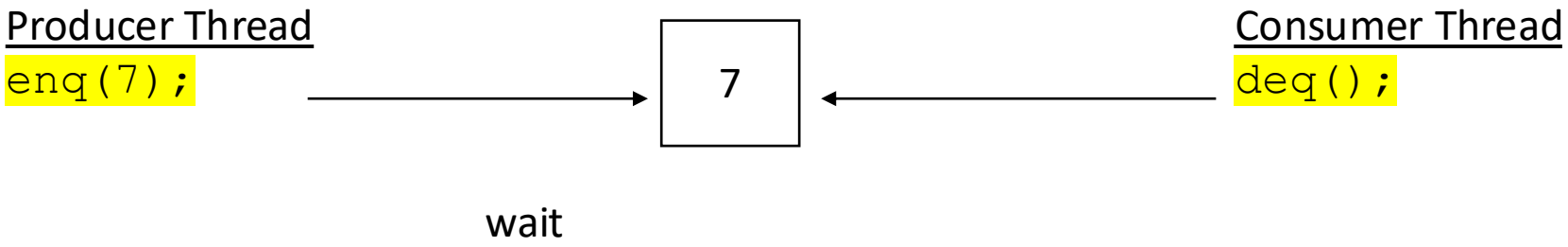
Consumer Thread

```
deq();
```

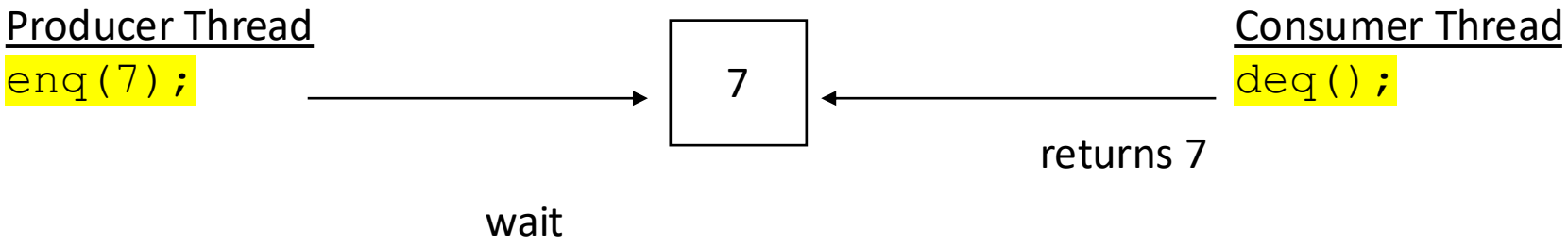
# Synchronous Producer Consumer Queues



# Synchronous Producer Consumer Queues



# Synchronous Producer Consumer Queues



# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
```



Consumer Thread

```
deq();
```

both can continue

# Synchronous Producer Consumer Queues

Producer Thread

```
sleep();
enq(7);
```



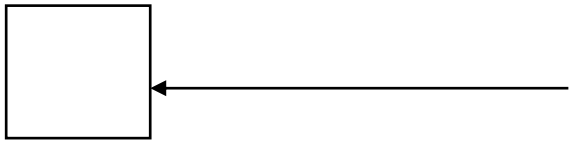
Consumer Thread

```
deq();
```

# Synchronous Producer Consumer Queues

Producer Thread

```
sleep();
enq(7);
```

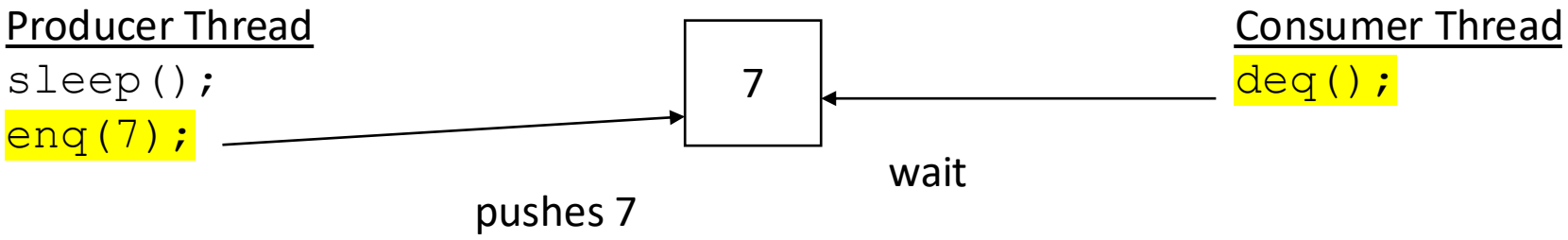


wait

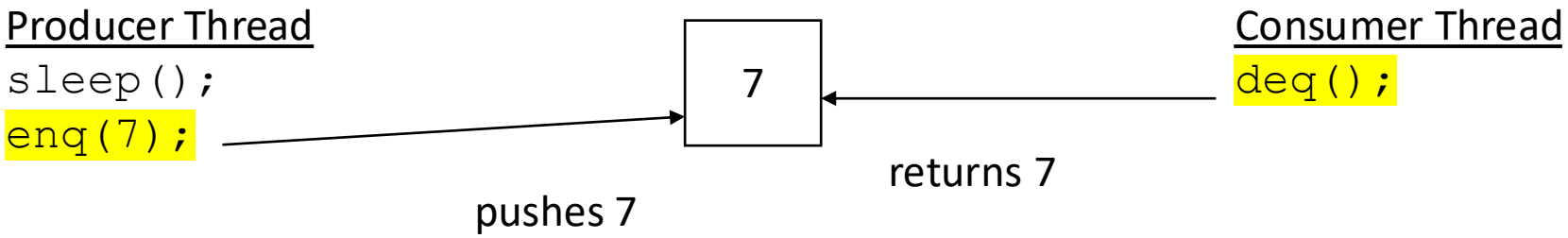
Consumer Thread

```
deq();
```

# Synchronous Producer Consumer Queues



# Synchronous Producer Consumer Queues

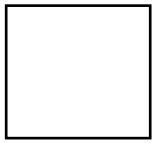



Then both can continue

# Synchronous Producer Consumer Queues Implementation

Producer Thread

```
enq(7);
```



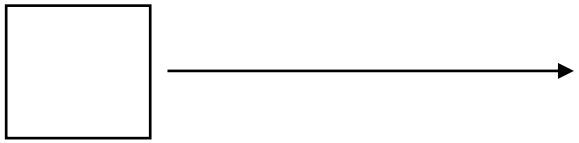
Consumer Thread

```
deq();
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
```



Consumer Thread

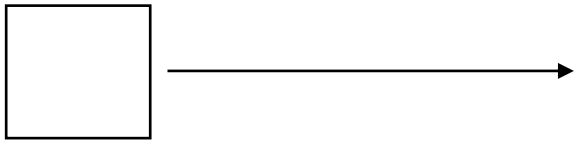
```
deq();
```

*Can the consumer just read?*

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
```



Consumer Thread

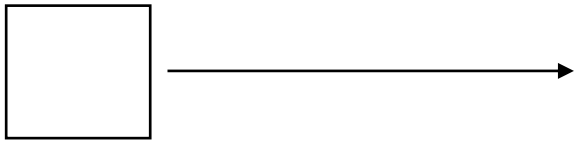
```
deq();
```

*Can the consumer just read?*
*Needs to wait for a value to appear*

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
```



Consumer Thread

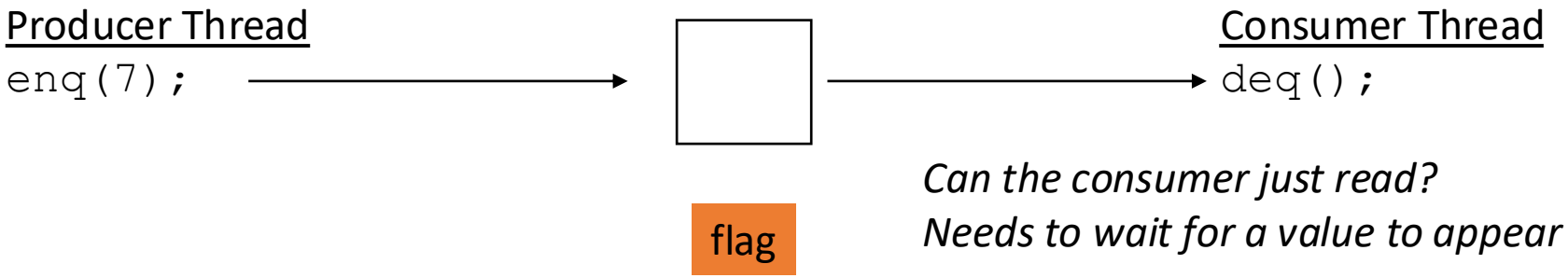
```
deq();
```

flag

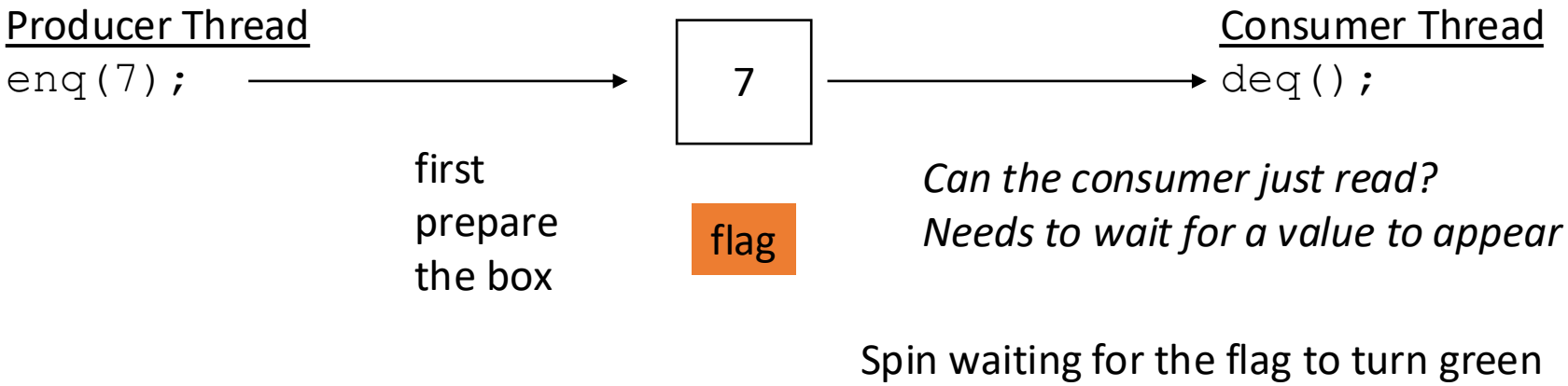
*Can the consumer just read?*
*Needs to wait for a value to appear*

Spin waiting for the flag to turn green

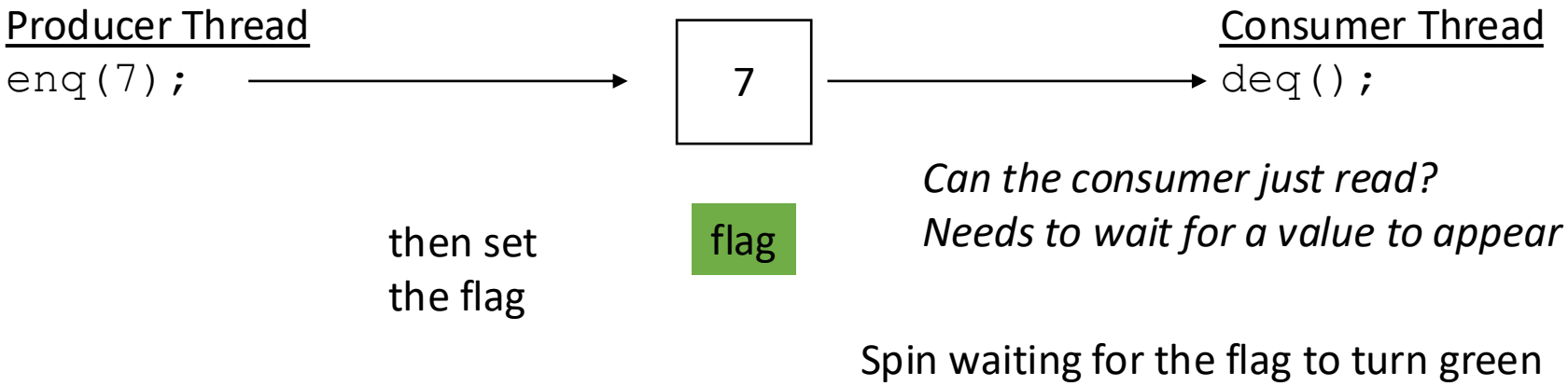
# Synchronous Producer Consumer Queues



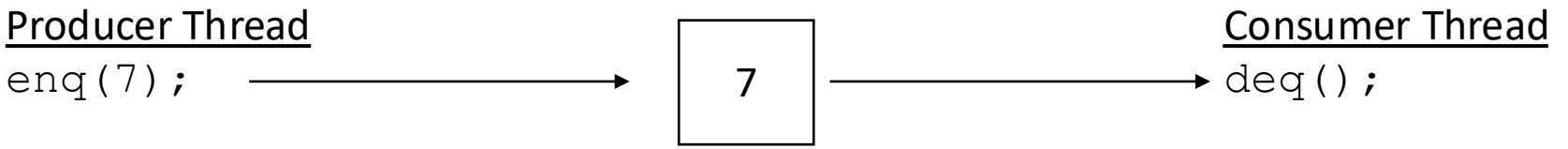
# Synchronous Producer Consumer Queues



# Synchronous Producer Consumer Queues



# Synchronous Producer Consumer Queues




*Can the consumer just read?*
*Needs to wait for a value to appear*

then set
the flag

flag

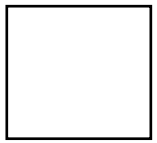
Spin waiting for the flag to turn green

Now the consumer can read from the box!

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
```



flag

Consumer Thread

```
deq();
```

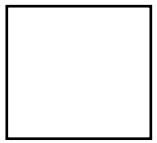
```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
    }
  }
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
```



flag

Consumer Thread

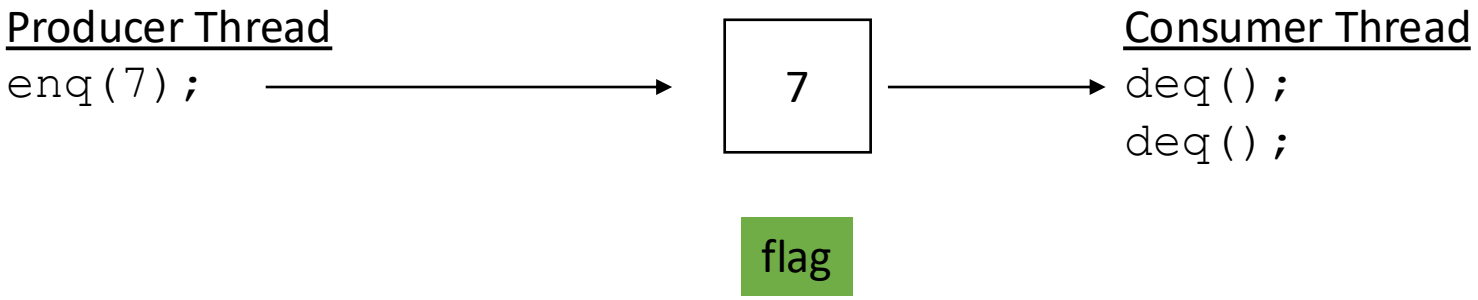
```
deq();
deq();
```

what happens when there are two deqs?

```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
    }
  }
```

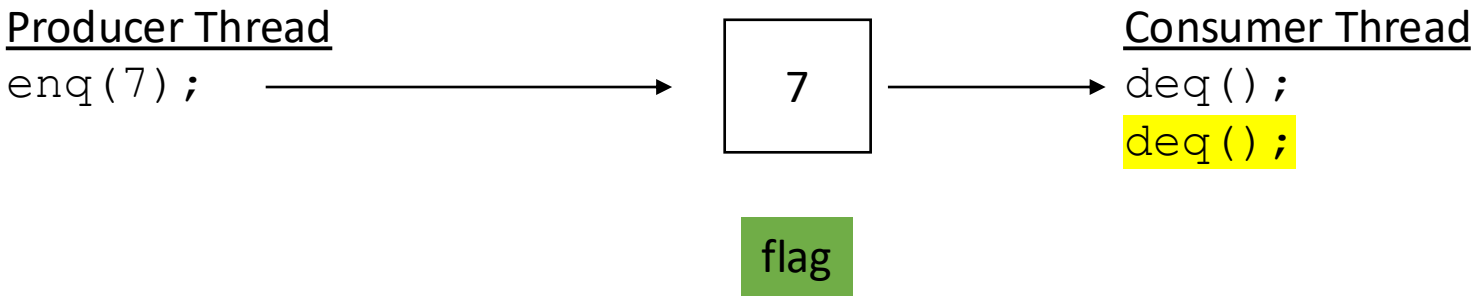
# Synchronous Producer Consumer Queues




```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
    }
  }
```

# Synchronous Producer Consumer Queues



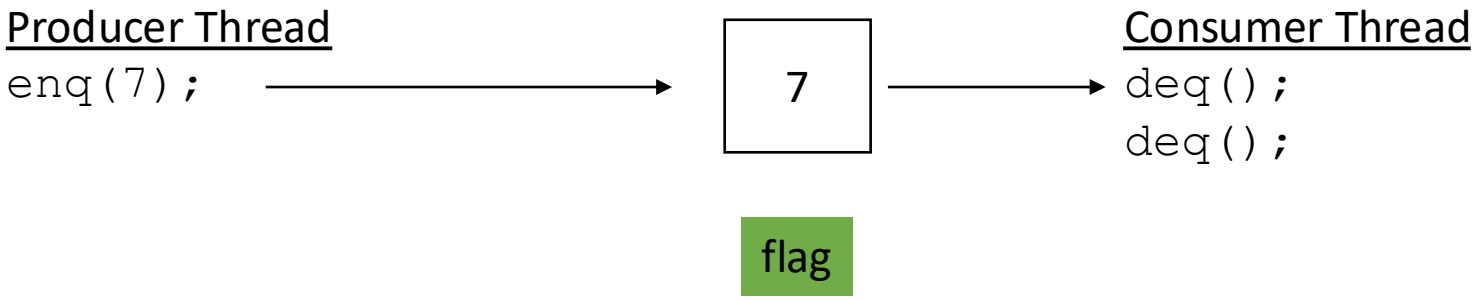

```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
    }
  }
```

what happens in the next deq?

How to fix?

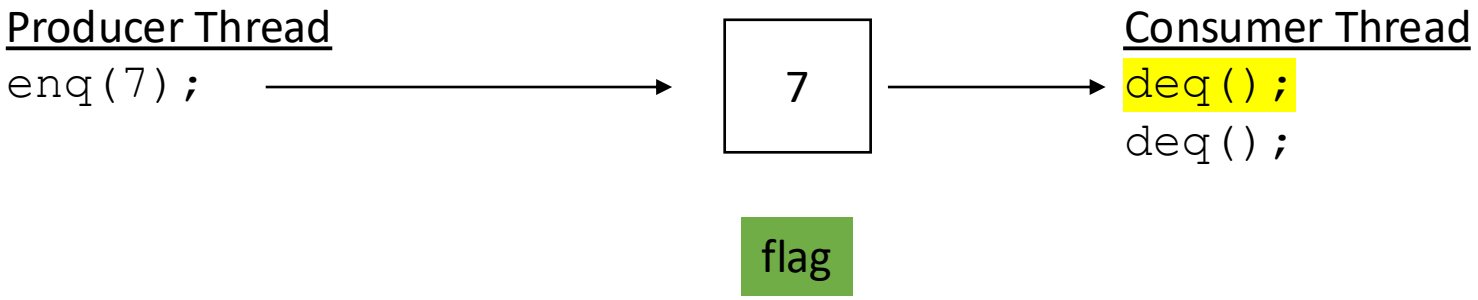
# Synchronous Producer Consumer Queues




```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

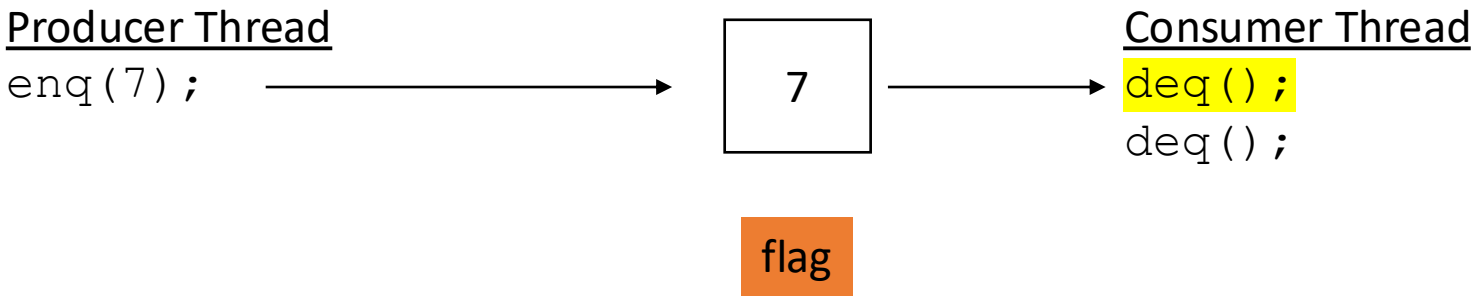
# Synchronous Producer Consumer Queues




```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

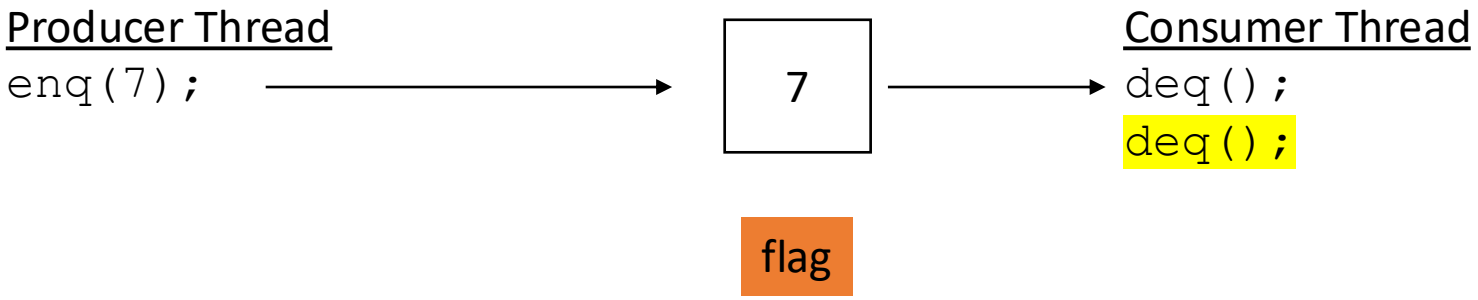
# Synchronous Producer Consumer Queues




```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues




```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

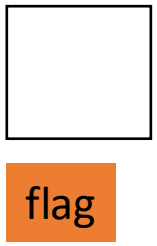
waiting like we are supposed to

# Synchronous Producer Consumer Queues

reset (now with extra enq)

Producer Thread

```
enq(7);
enq(8);
```




Consumer Thread

```
deq();
deq();
```

extra enq

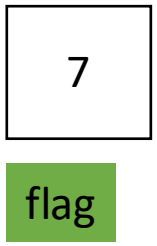
```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
enq(8);
```




Consumer Thread

```
deq();
deq();
```

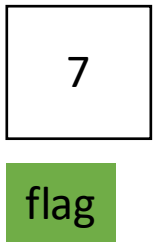
```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
enq(8);
```




Consumer Thread

```
deq();
deq();
```

```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
enq(8);
```




Consumer Thread

```
deq();
deq();
```

*7 was dropped!*

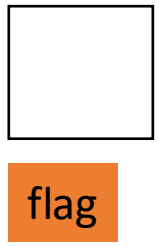
*how to fix?*

```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues

reset




Producer Thread

```
enq(7);
enq(8);
```

Consumer Thread

```
deq();
deq();
```

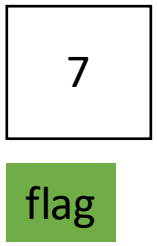
```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
      // wait for flag to be reset
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
enq(8);
```




Consumer Thread

```
deq();
deq();
```

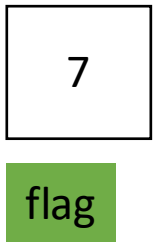
```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
      // wait for flag to be reset
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
enq(8);
```




Consumer Thread

```
deq();
deq();
```

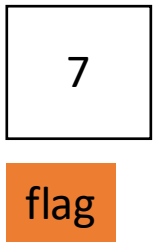
```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
      // wait for flag to be reset
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
enq(8);
```




Consumer Thread

```
deq();
deq();
```

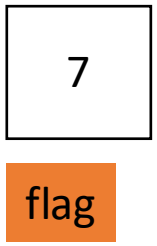
```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
      // wait for flag to be reset
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

# Synchronous Producer Consumer Queues

Producer Thread

```
enq(7);
enq(8);
```




Consumer Thread

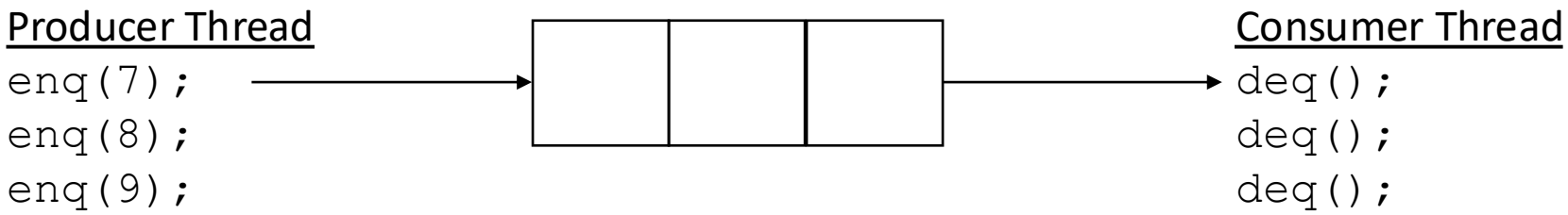
```
deq();
deq();
```

```
class SyncQueue {
  private:
    atomic_int box;
    atomic_bool flag;

  public:
    void enq(int x) {
      // put value in box
      // set flag
      // wait for flag to be reset
    }
    void deq() {
      // wait for flag to be set
      // read from the box
      // reset flag
    }
  }
```

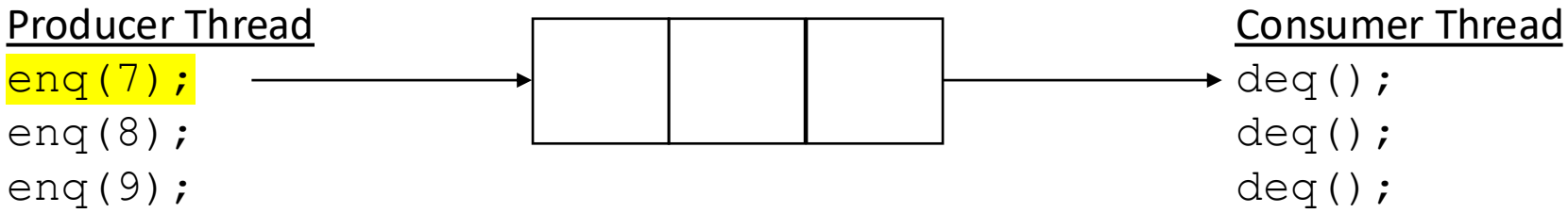
# Producer Consumer Queues

- Asynchronous:



# Producer Consumer Queues

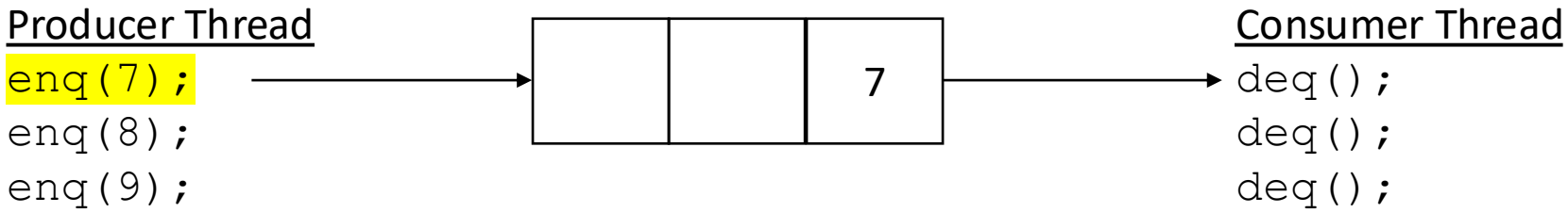
- Asynchronous:




no waiting for producer (while there is room)

# Producer Consumer Queues

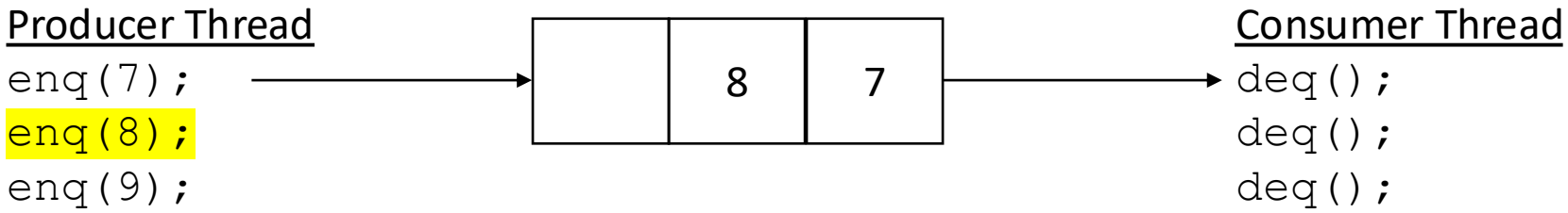
- Asynchronous:




no waiting for producer (while there is room)

# Producer Consumer Queues

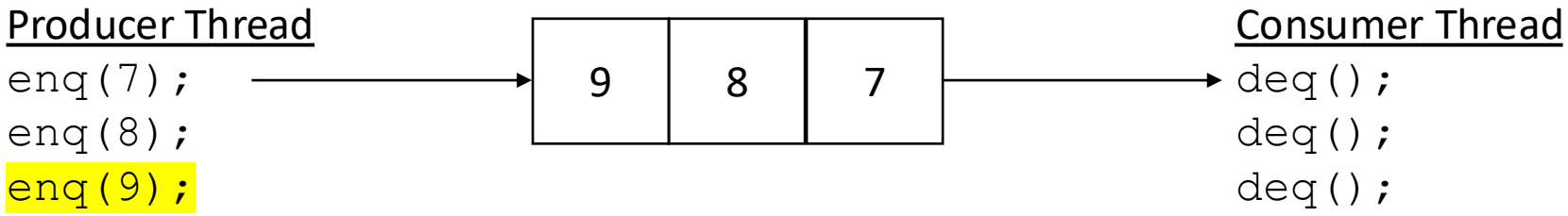
- Asynchronous:




no waiting for producer (while there is room)

# Producer Consumer Queues

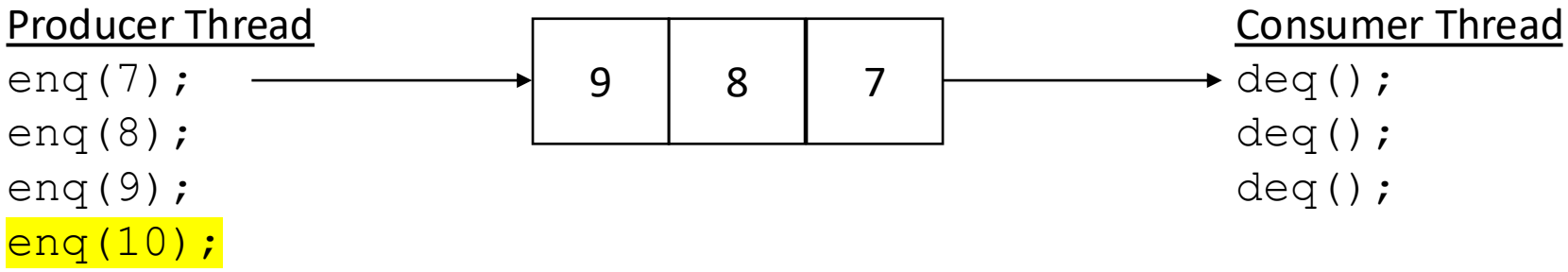
- Asynchronous:




no waiting for producer (while there is room)

# Producer Consumer Queues

- Asynchronous:

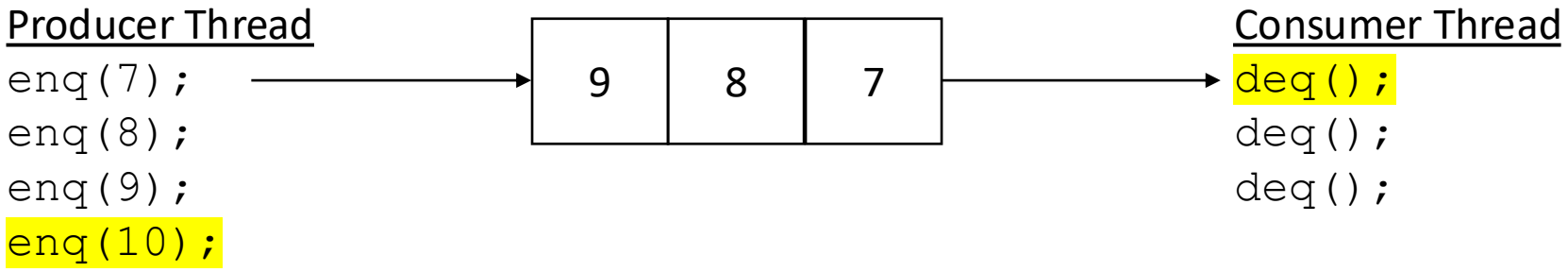



no waiting for producer (while there is room)

when there is no room, the queue will wait

# Producer Consumer Queues

- Asynchronous:



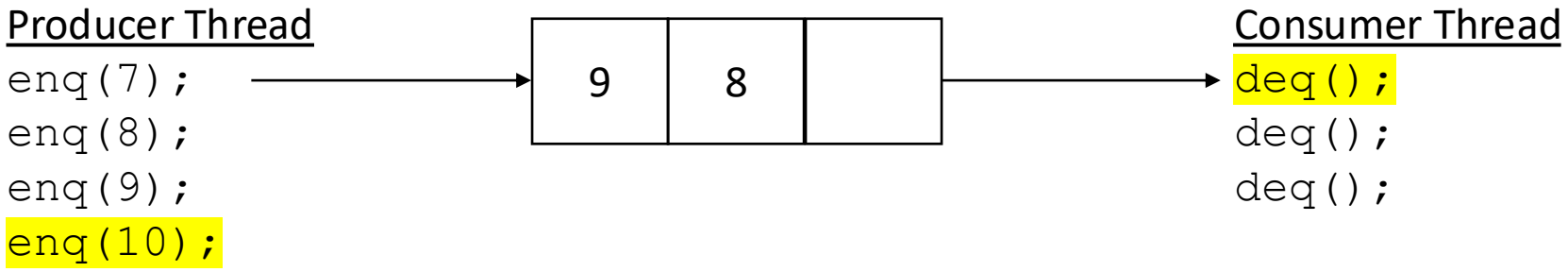

no waiting for producer (while there is room)

when there is no room, the queue will wait

returns 7

# Producer Consumer Queues

- Asynchronous:



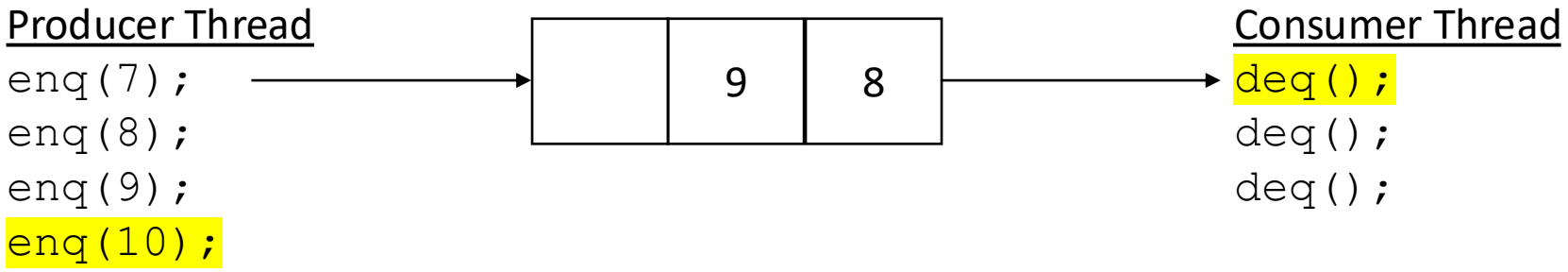

no waiting for producer (while there is room)

when there is no room, the queue will wait

returns 7

# Producer Consumer Queues

- Asynchronous:



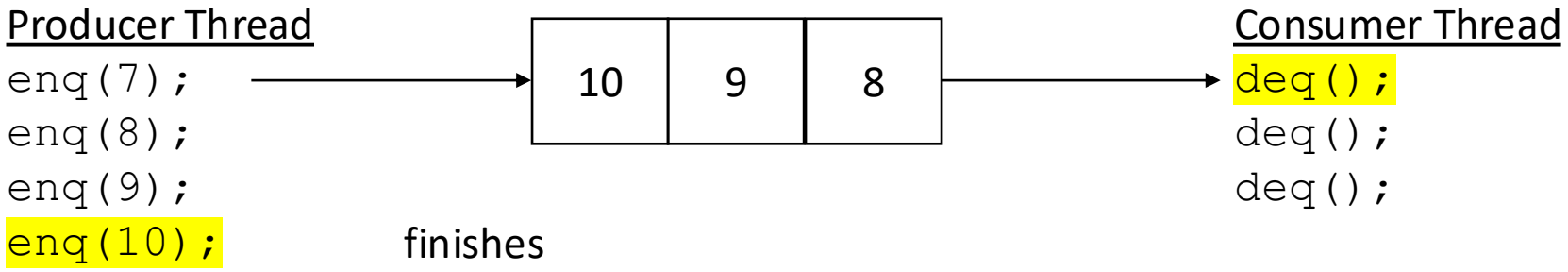

no waiting for producer (while there is room)

when there is no room, the queue will wait

returns 7

# Producer Consumer Queues

- Asynchronous:



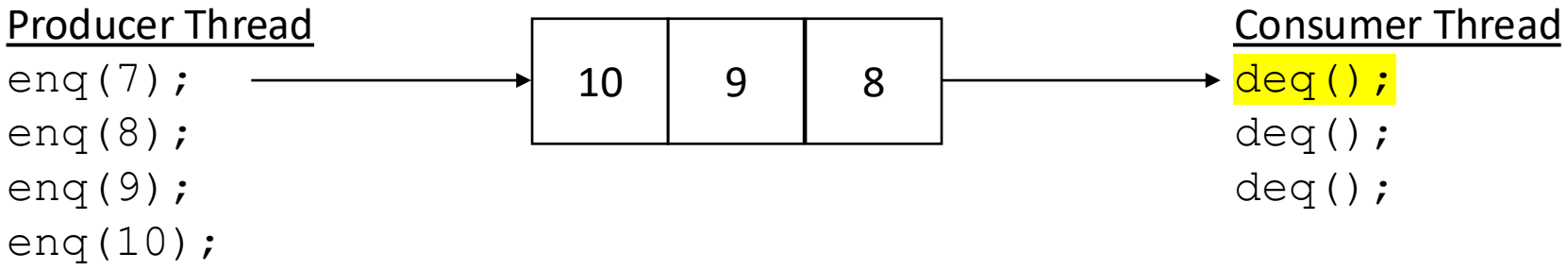

no waiting for producer (while there is room)

when there is no room, the queue will wait

returns 7

# Producer Consumer Queues

- Asynchronous:



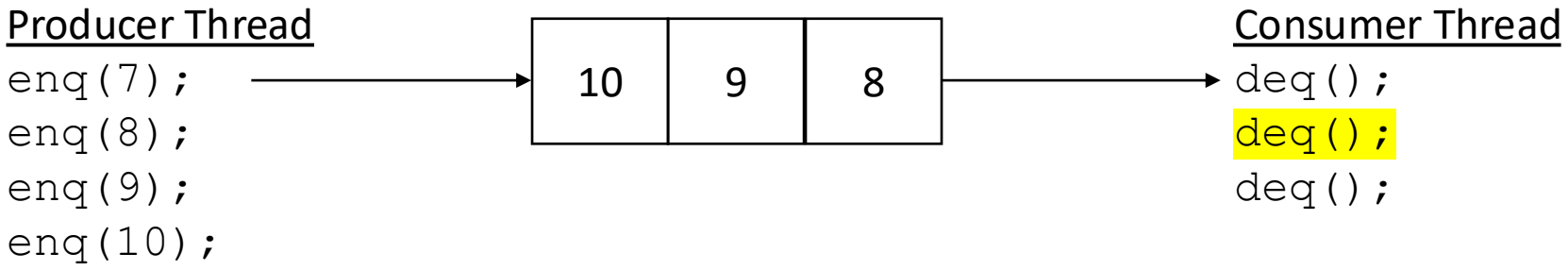

no waiting for producer (while there is room)

when there is no room, the queue will wait

returns 7

# Producer Consumer Queues

- Asynchronous:



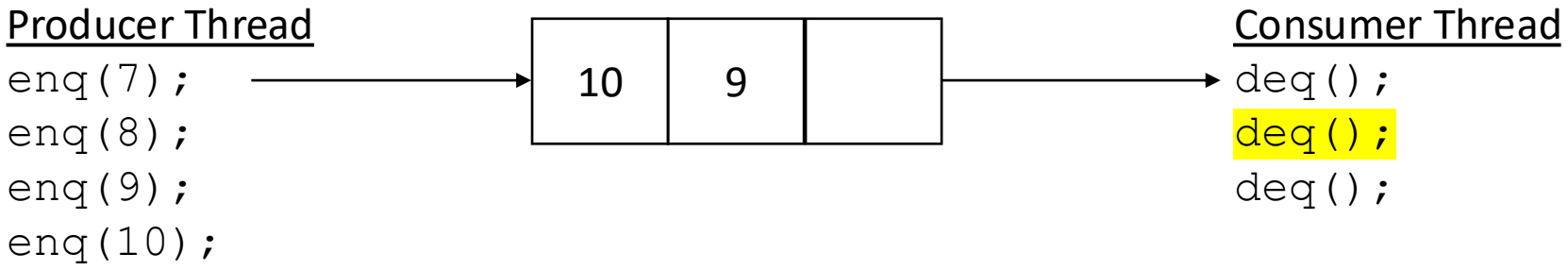

no waiting for producer (while there is room)

when there is no room, the queue will wait

returns 8

# Producer Consumer Queues

- Asynchronous:



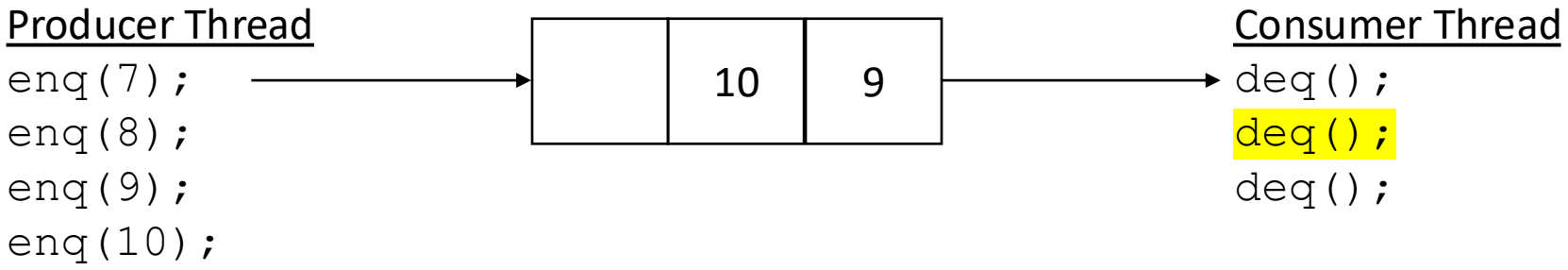

no waiting for producer (while there is room)

when there is no room, the queue will wait

returns 8

# Producer Consumer Queues

- Asynchronous:



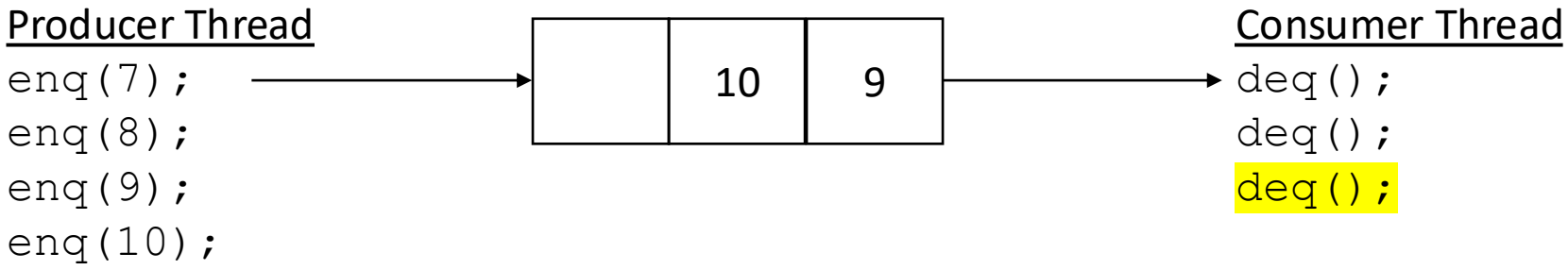

no waiting for producer (while there is room)

when there is no room, the queue will wait

returns 8

# Producer Consumer Queues

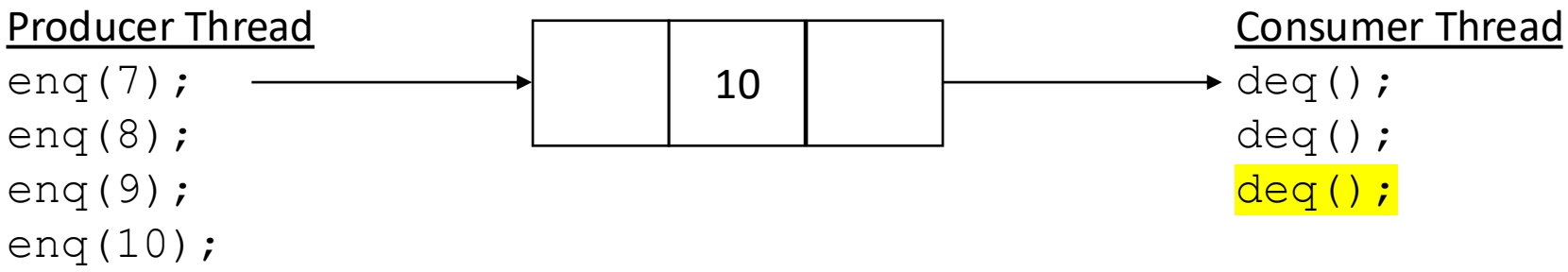
- Asynchronous:




returns 9

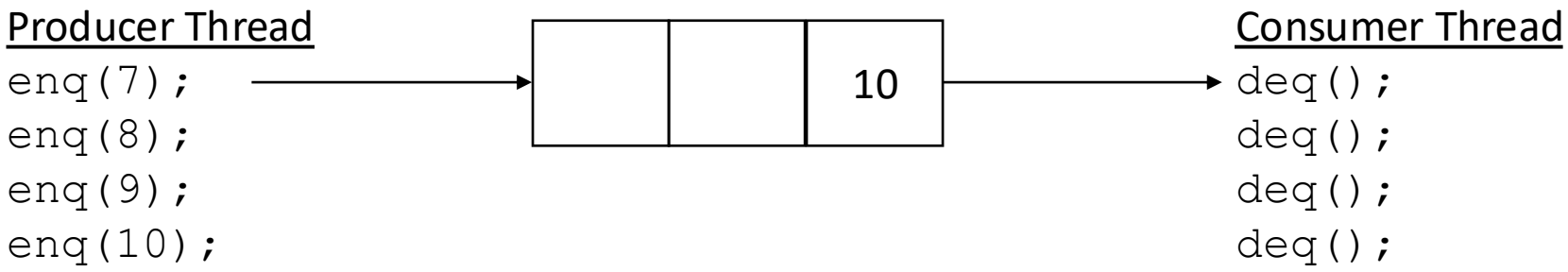
# Producer Consumer Queues

- Asynchronous:



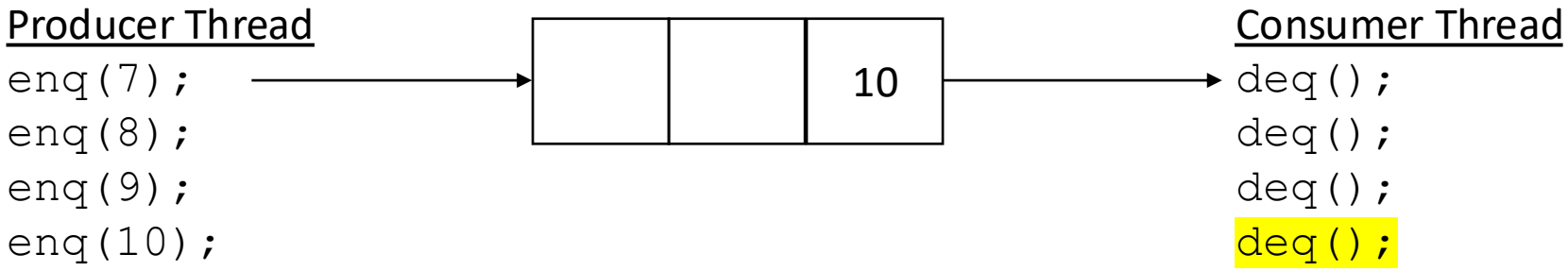
# Producer Consumer Queues

- Asynchronous:



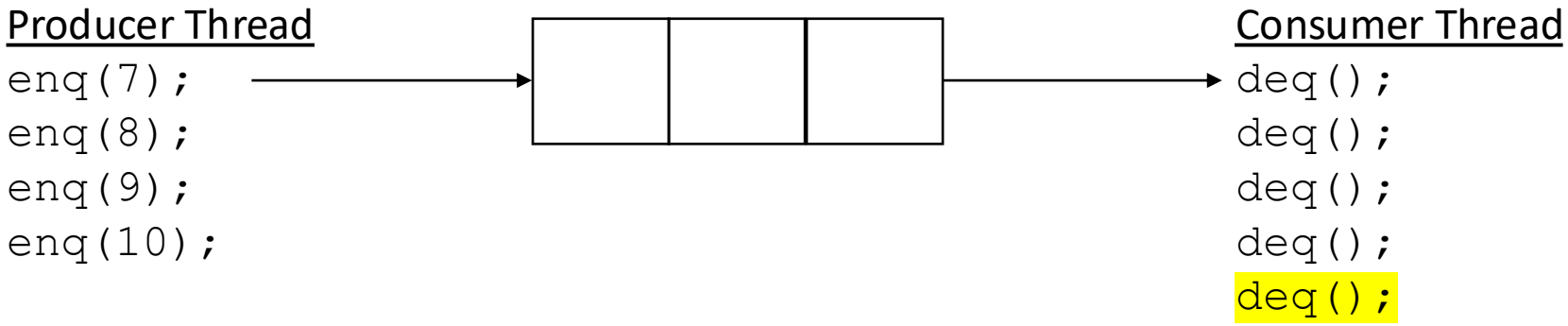
# Producer Consumer Queues

- Asynchronous:



# Producer Consumer Queues

- Asynchronous:




blocks when there is nothing in the queue

# Producer Consumer Queues

- How do we implement it?

# Producer Consumer Queues

- Start with a fixed size array



# Producer Consumer Queues

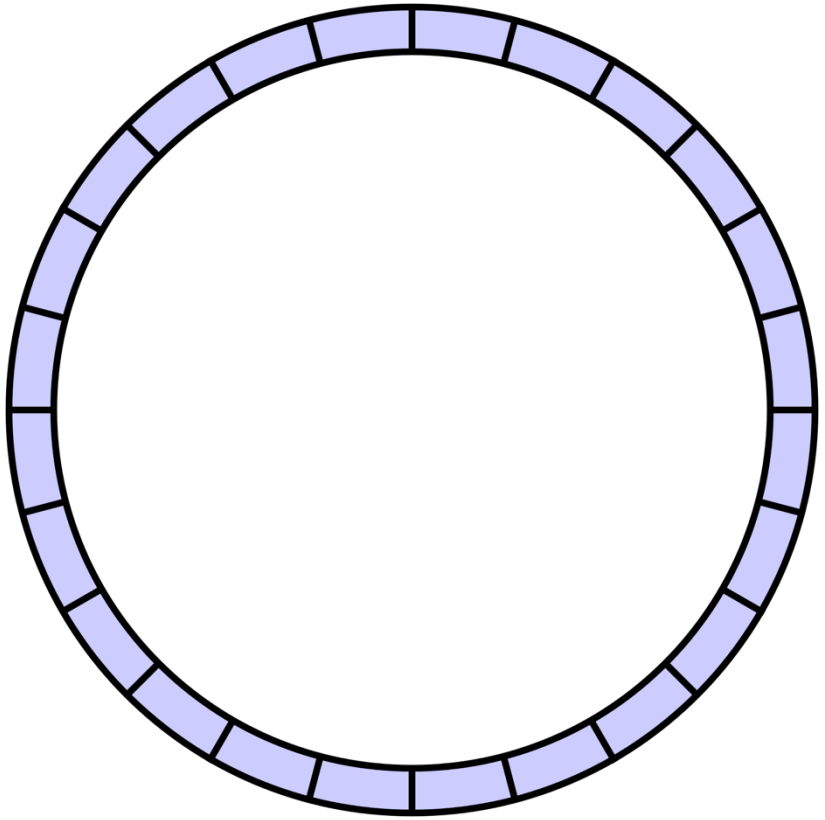
- Start with a fixed size array



We will use what is called a *circular buffer method*

# Producer Consumer Queues

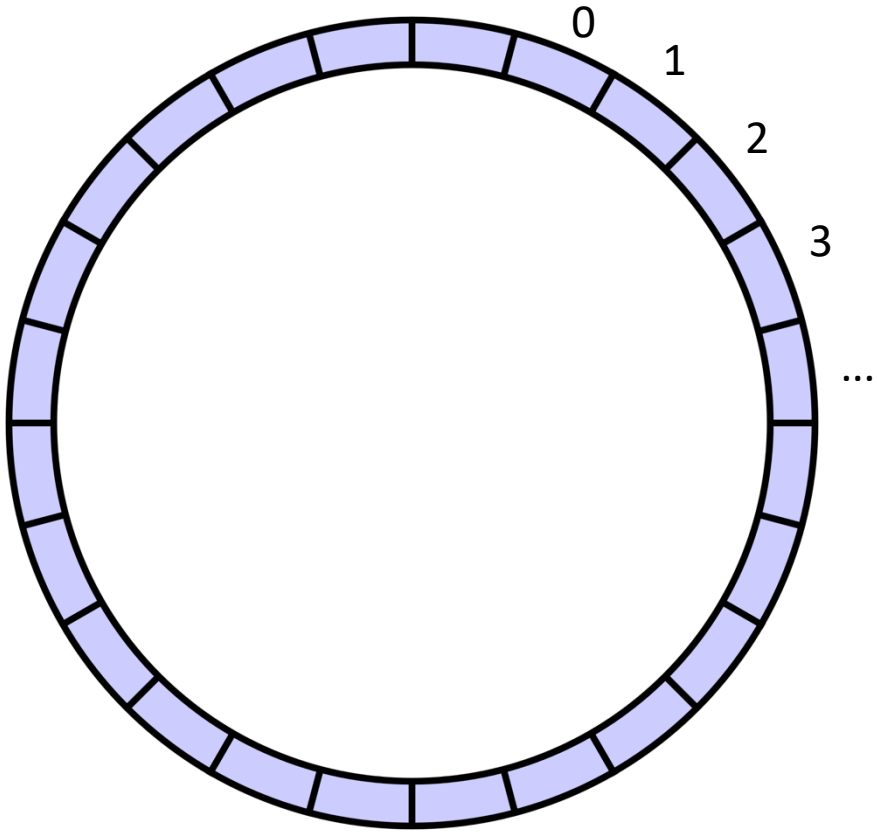
- Start with a fixed size array



conceptually it is a circle

# Producer Consumer Queues

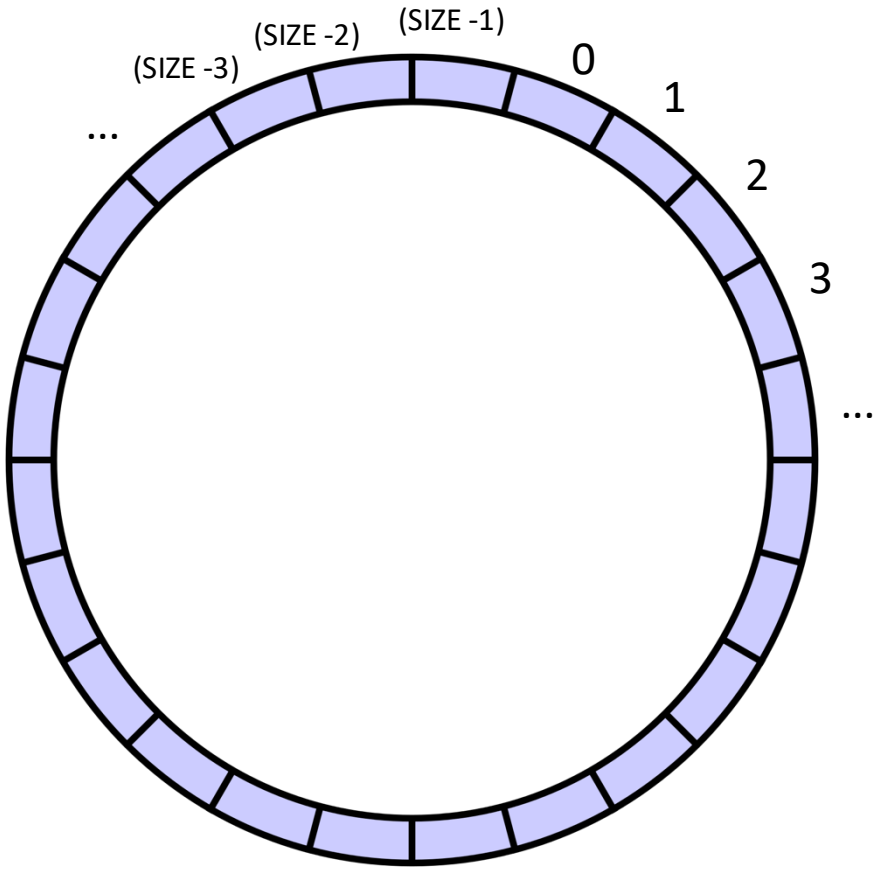
- Start with a fixed size array




conceptually it is a circle

# Producer Consumer Queues

- Start with a fixed size array

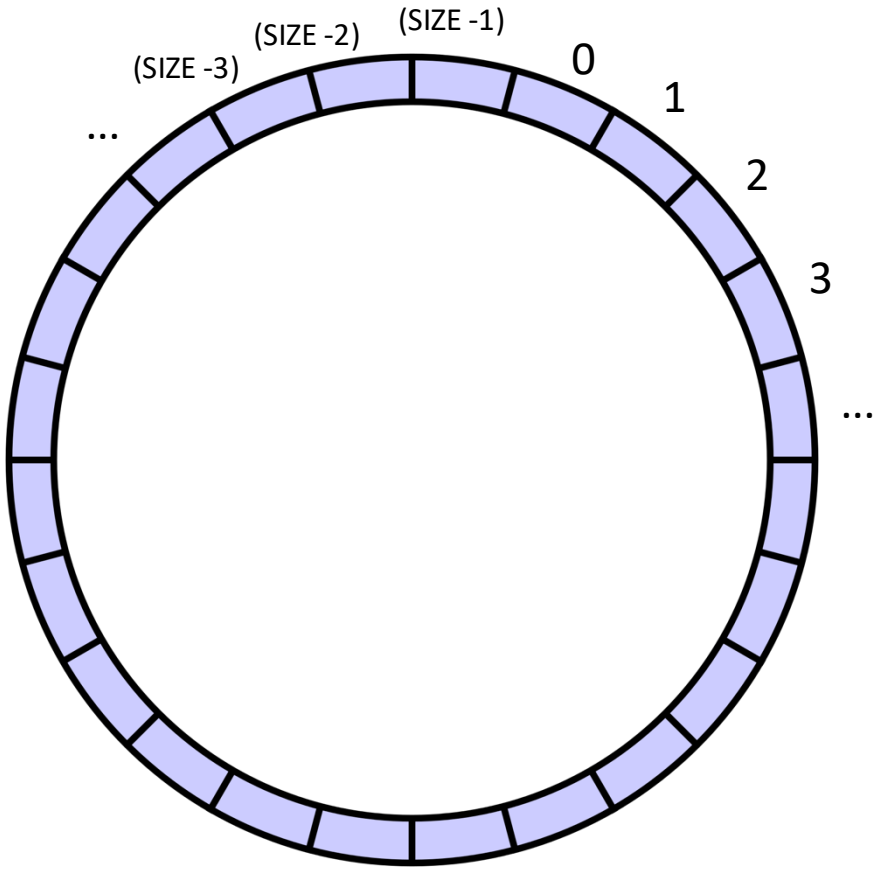



indexes will circulate in order and wrap around

conceptually it is a circle

# Producer Consumer Queues

- Start with a fixed size array




indexes will circulate in order and wrap around

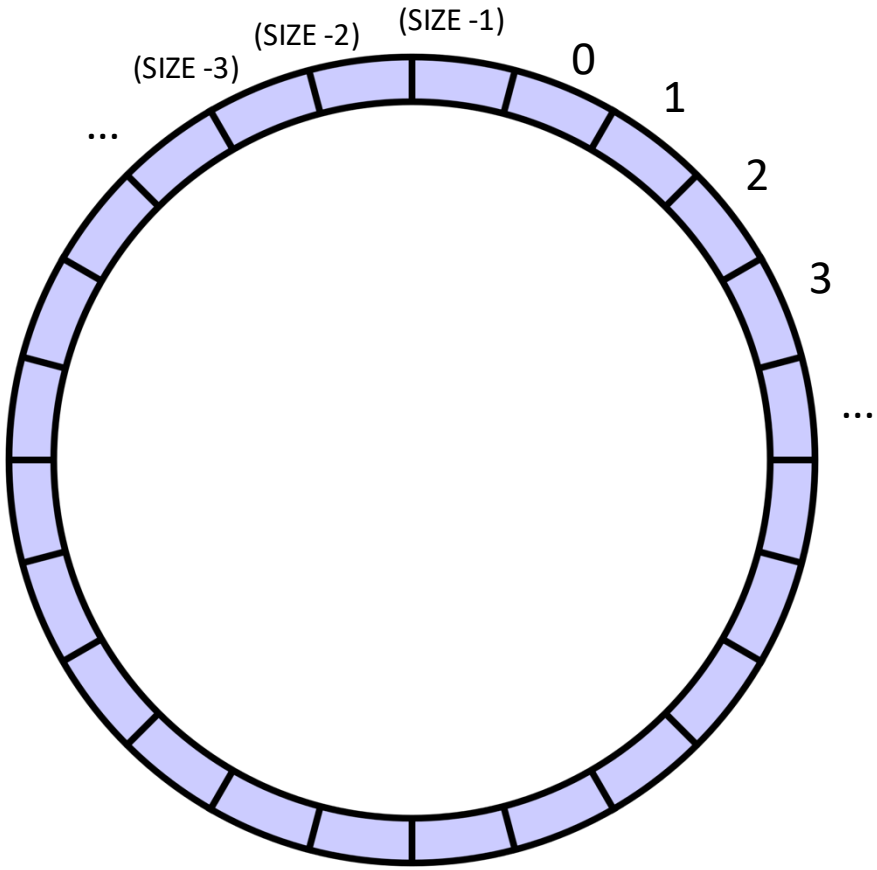
we will assume modular arithmetic:

if x = (SIZE - 1) then
x + 1 == 0;

conceptually it is a circle

# Producer Consumer Queues

- Start with a fixed size array




indexes will circulate in order and wrap around

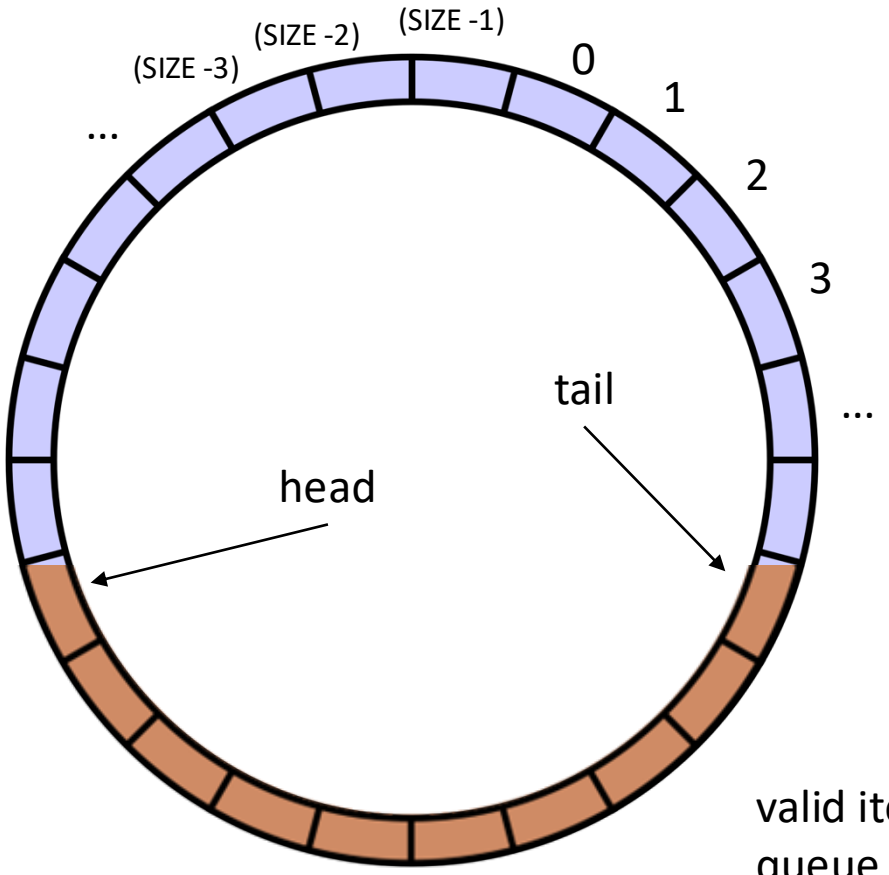
Two variables to keep track of where to deq and enq:

head and tail

conceptually it is a circle

# Producer Consumer Queues

- Start with a fixed size array




indexes will circulate in order and wrap around

Two variables to keep track of where to deq and enq:

head and tail:

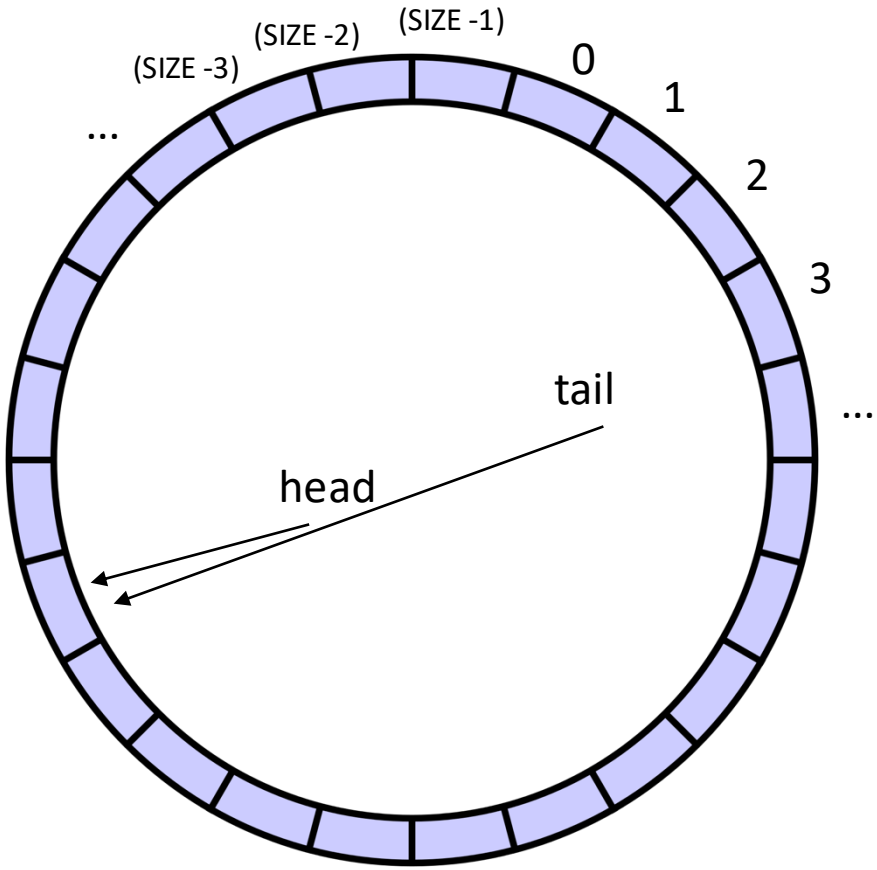
enq to the head, deq from the tail

valid items in the queue

conceptually it is a circle

# Producer Consumer Queues

- Start with a fixed size array




indexes will circulate in order and wrap around

Two variables to keep track of where to deq and enq:

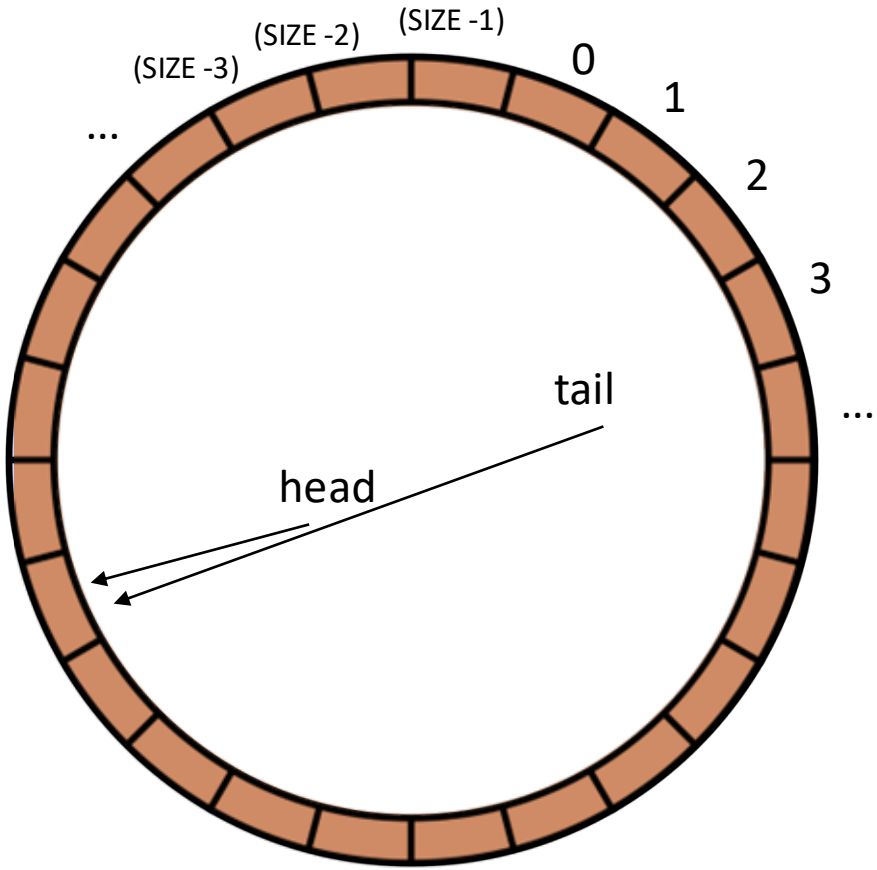
head and tail

Empty queue is when head == tail

conceptually it is a circle

# Producer Consumer Queues

- Start with a fixed size array




indexes will circulate in order and wrap around

Two variables to keep track of where to deq and enq:

head and tail

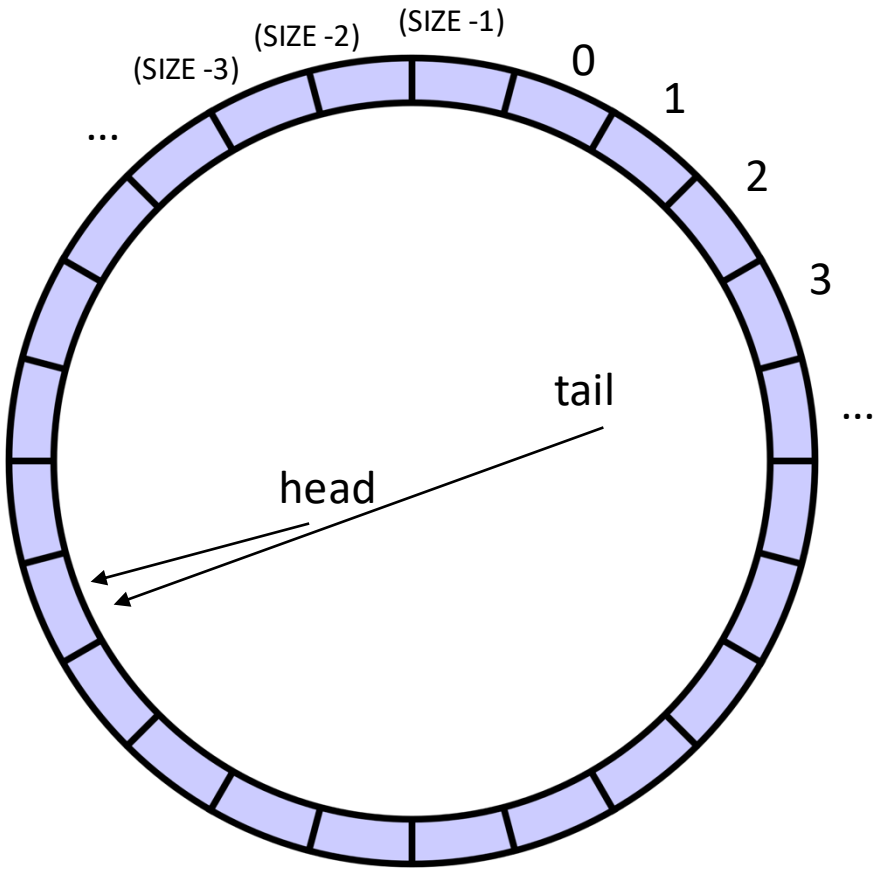
Empty queue is when
head == tail

Full queue is when
head == tail?

conceptually it is a circle

# Producer Consumer Queues

- Start with a fixed size array




indexes will circulate in order and wrap around

Two variables to keep track of where to deq and enq:

head and tail

Empty queue is when head == tail

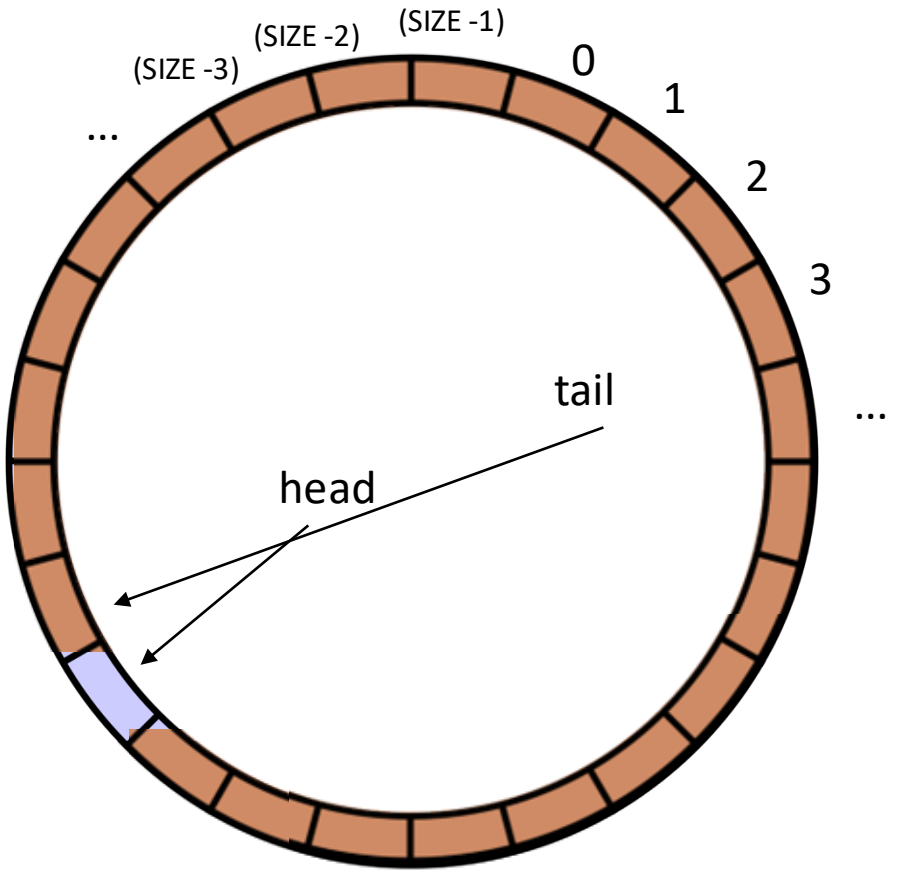
Full queue is when head == tail?

but then how to tell full queue from empty?

conceptually it is a circle

# Producer Consumer Queues

- Start with a fixed size array




indexes will circulate in order and wrap around

Two variables to keep track of where to deq and enq:

head and tail

Empty queue is when
head == tail

Full queue is when
head + 1 == tail

wasting one location, but its okay...

conceptually it is a circle

# Quiz 9.

A circular buffer is:

*Select one*

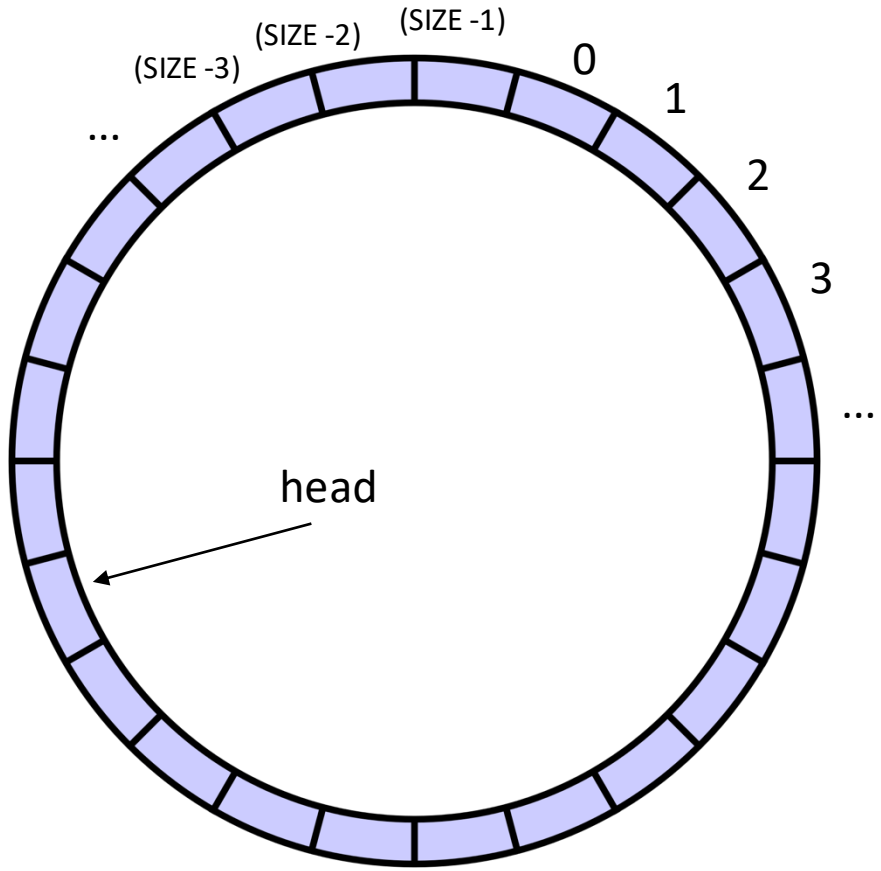
A. A useful data representation for fixed-length queues

B. Part of the C++ standard library

C. A special type of memory that is organized in circular patters

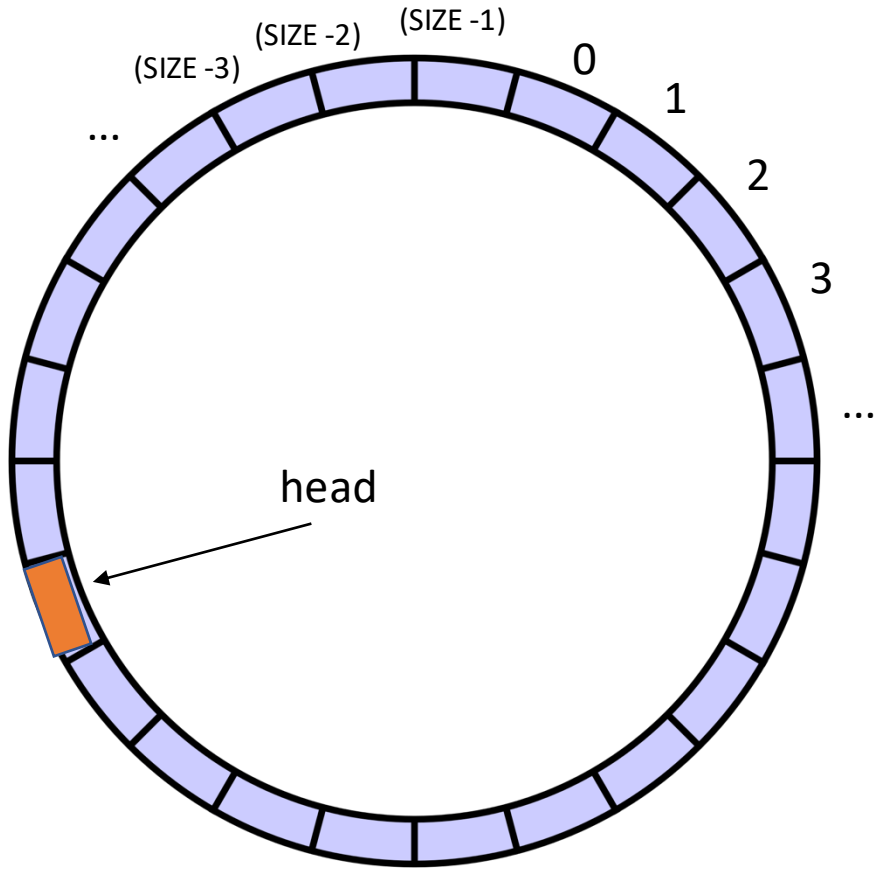
Scan to answer

**forms.gle/ZdqkrLPvwJfus3o3A**



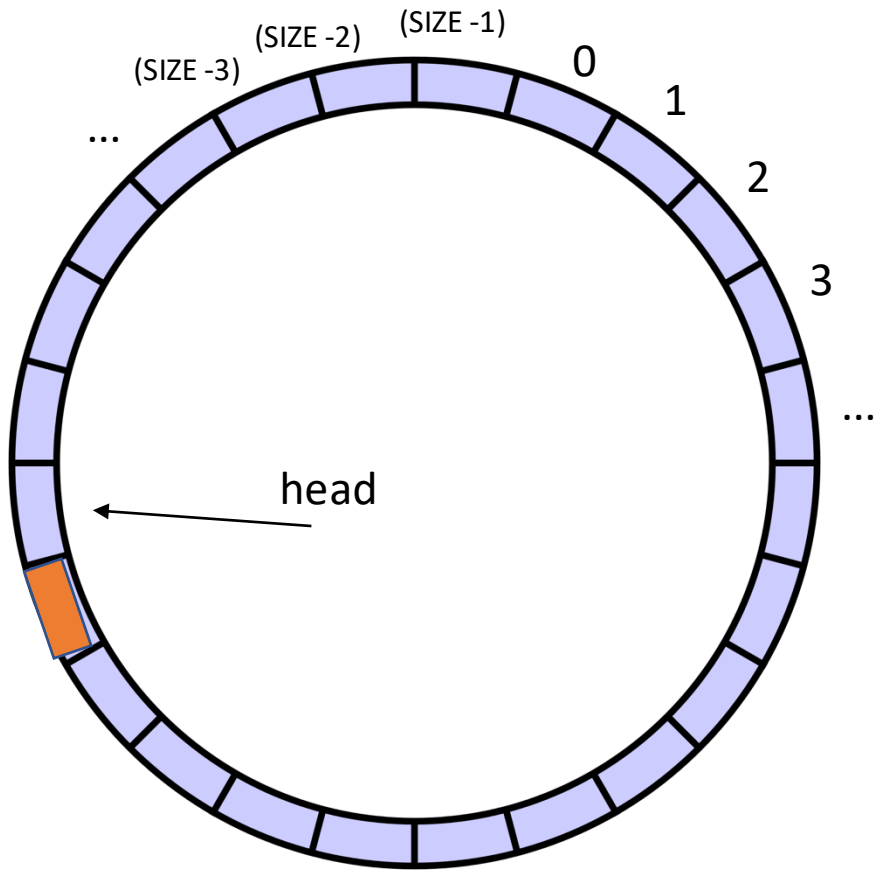

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
}
```



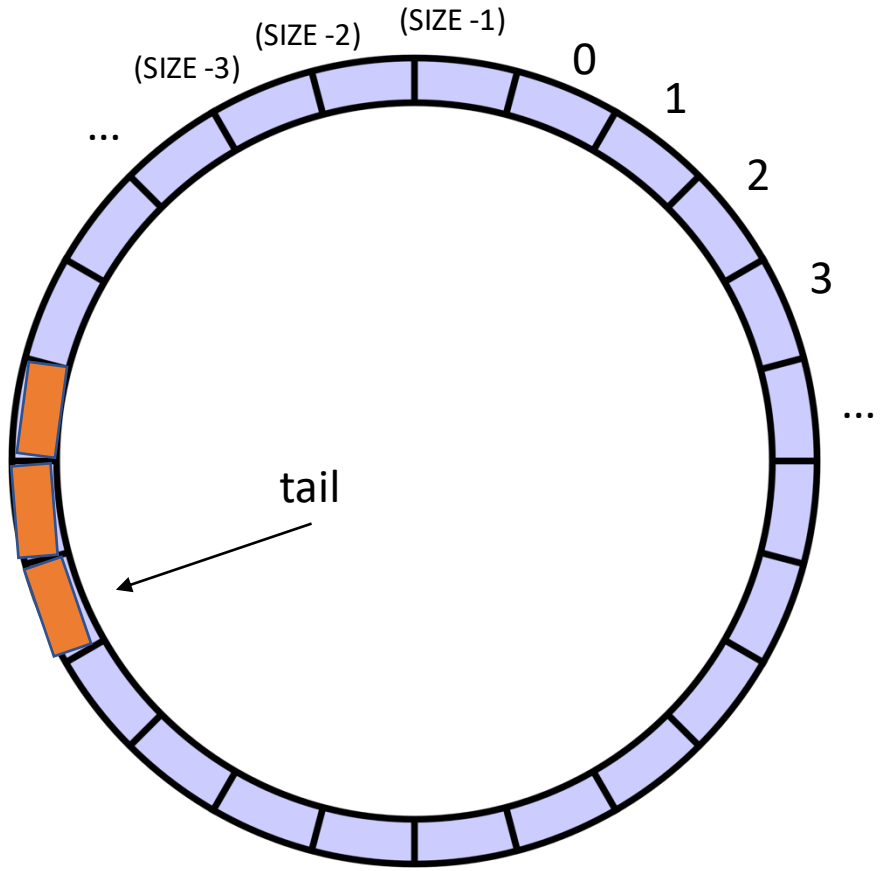

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
}
```



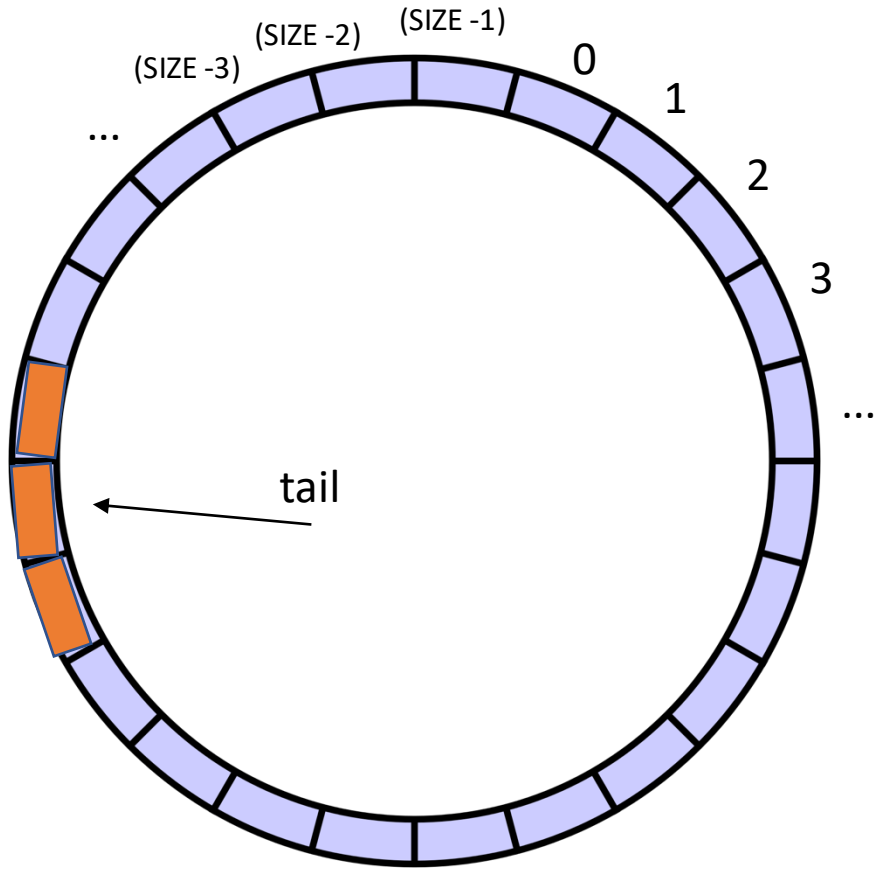

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
}
```




```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

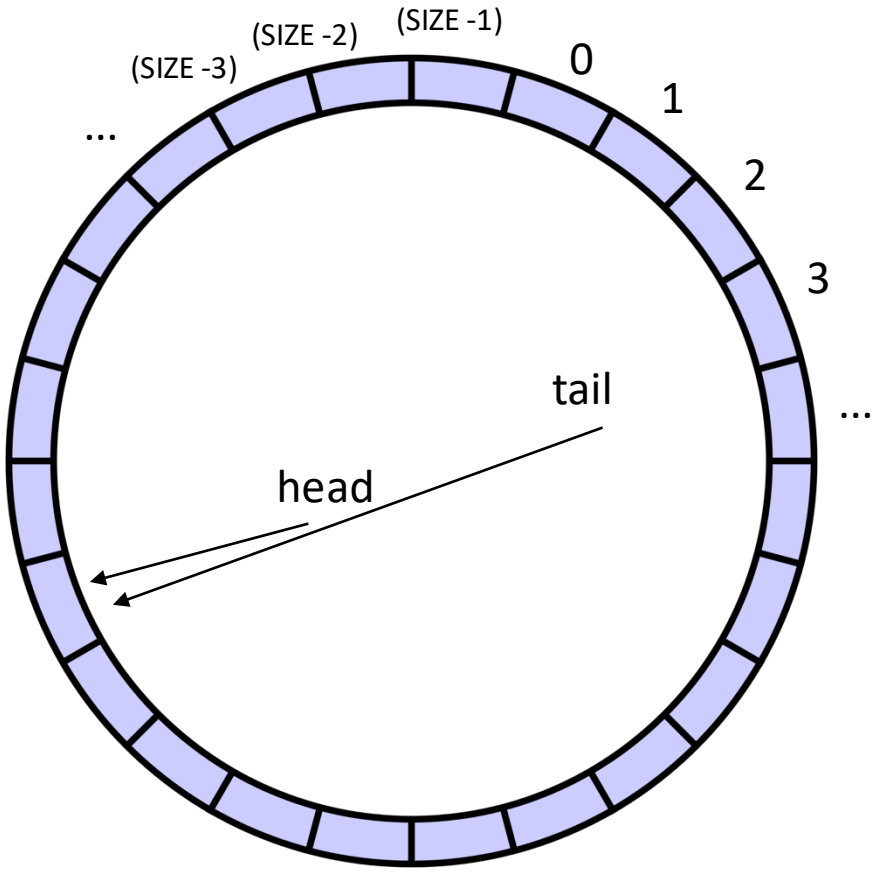
  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // get value at tail
      // increment tail
    }
}
```




```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // get value at tail
      // increment tail
    }
}
```

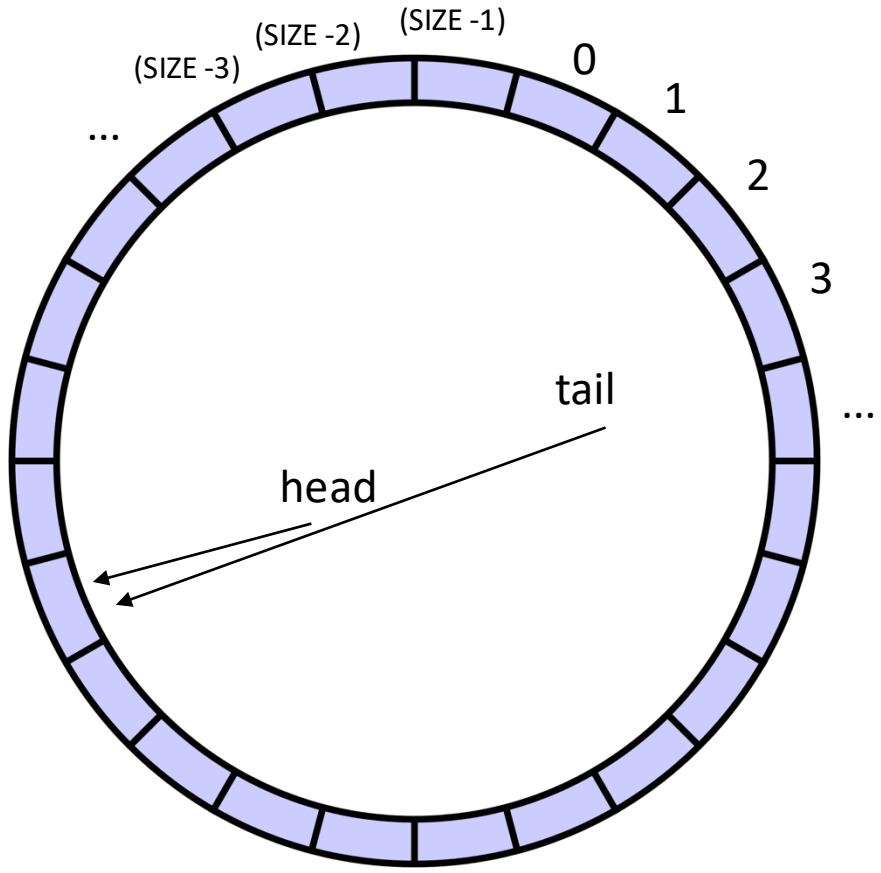
This looks like the two threads don't even share head and tail! What is missing?




What happens if we try to dequeue here?

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

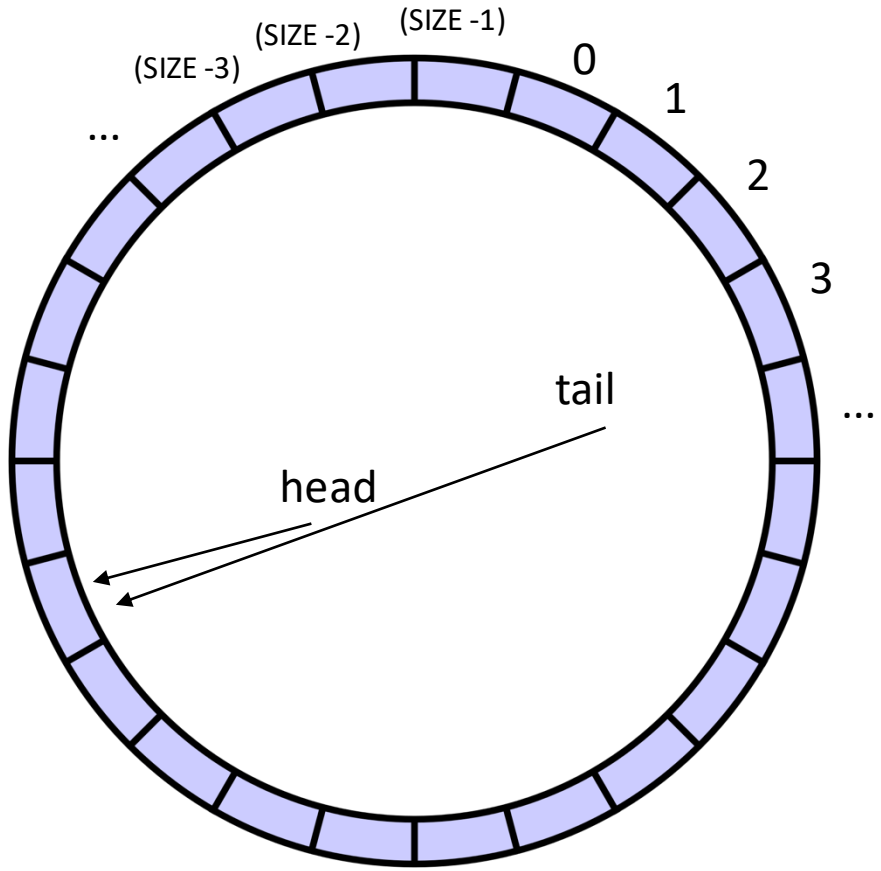
  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // get value at tail
      // increment tail
    }
}
```




What happens if we try to dequeue here?
Get a random value?

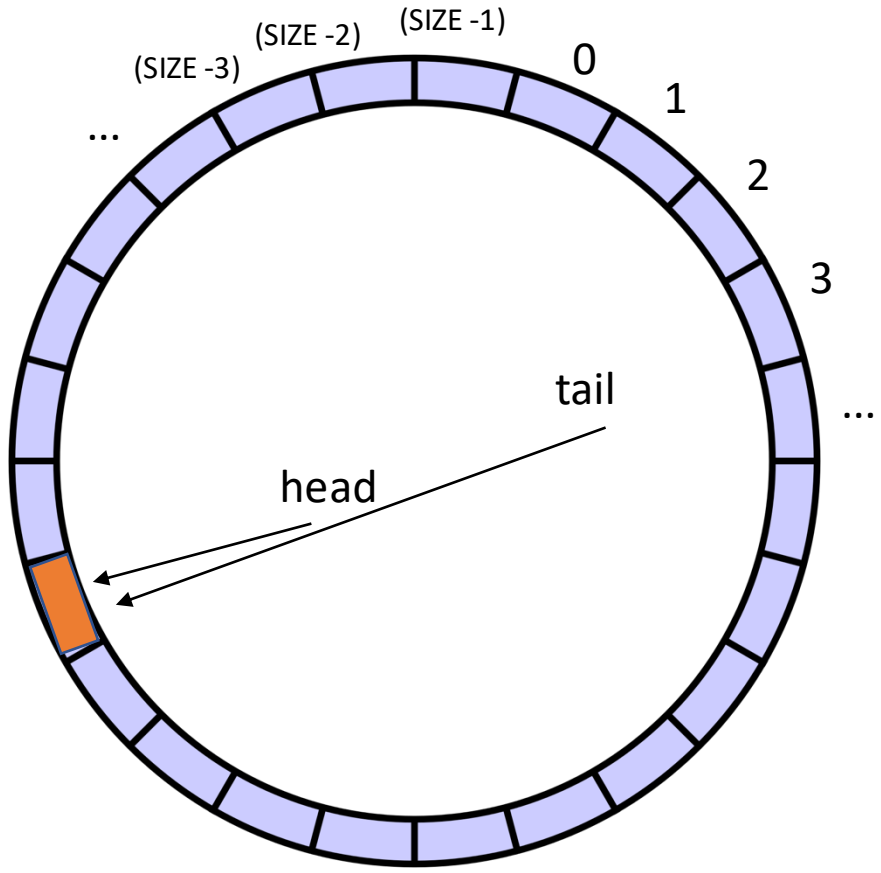
```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // get value at tail
      // increment tail
    }
}
```



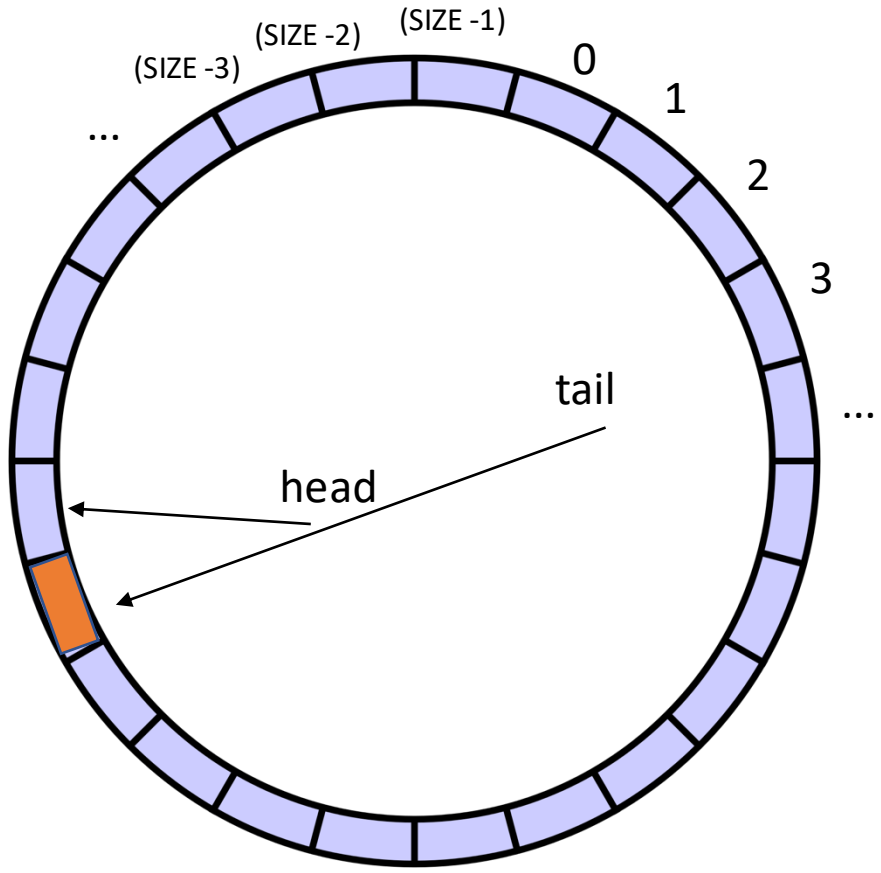

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```



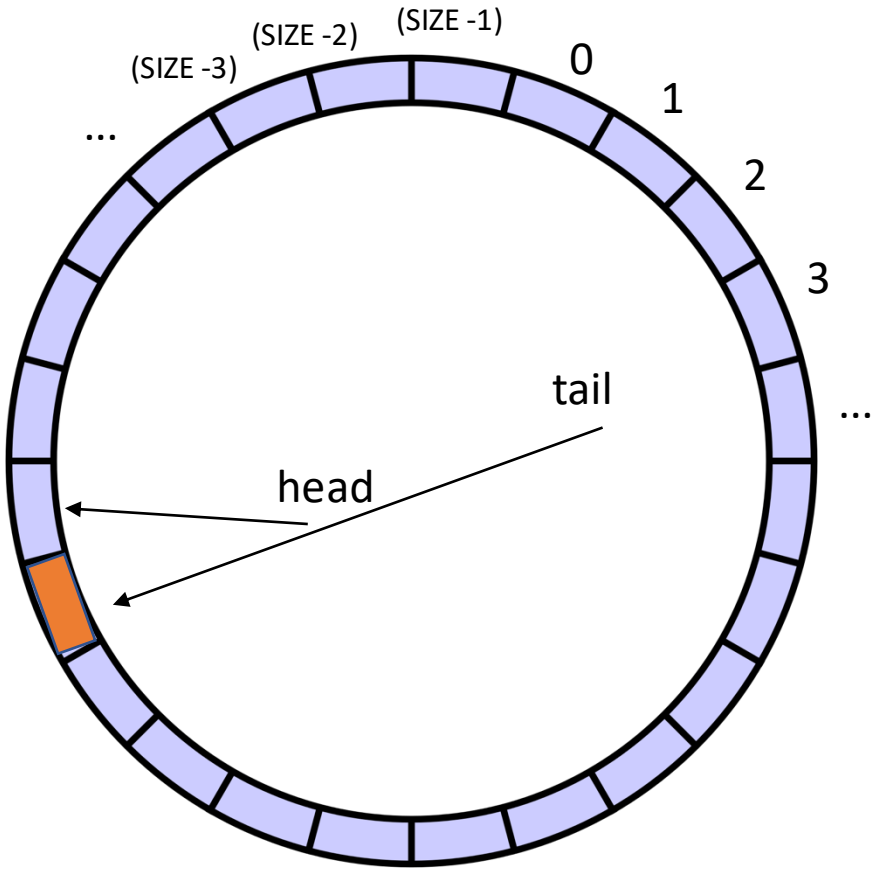

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```



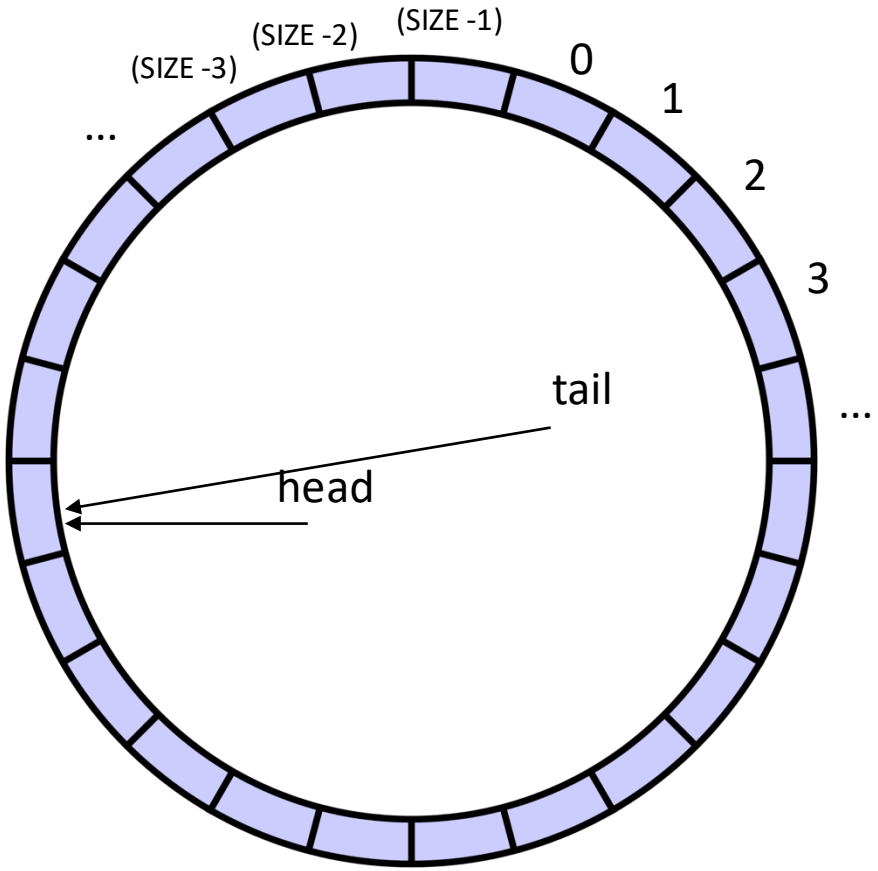

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```



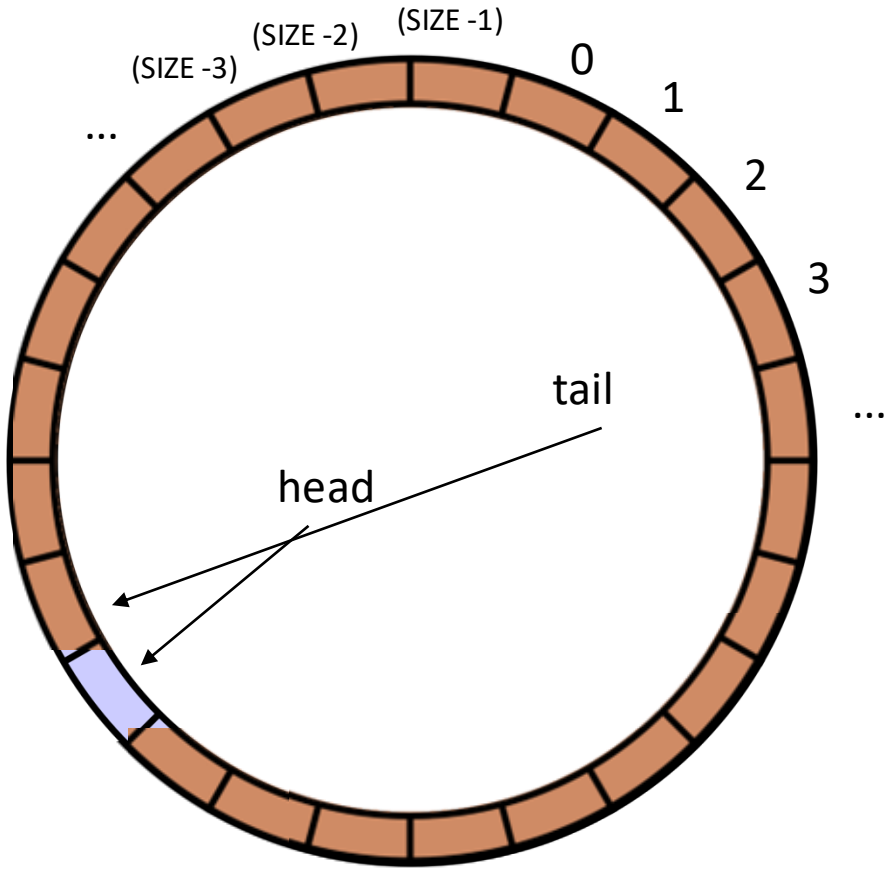

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```




```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```

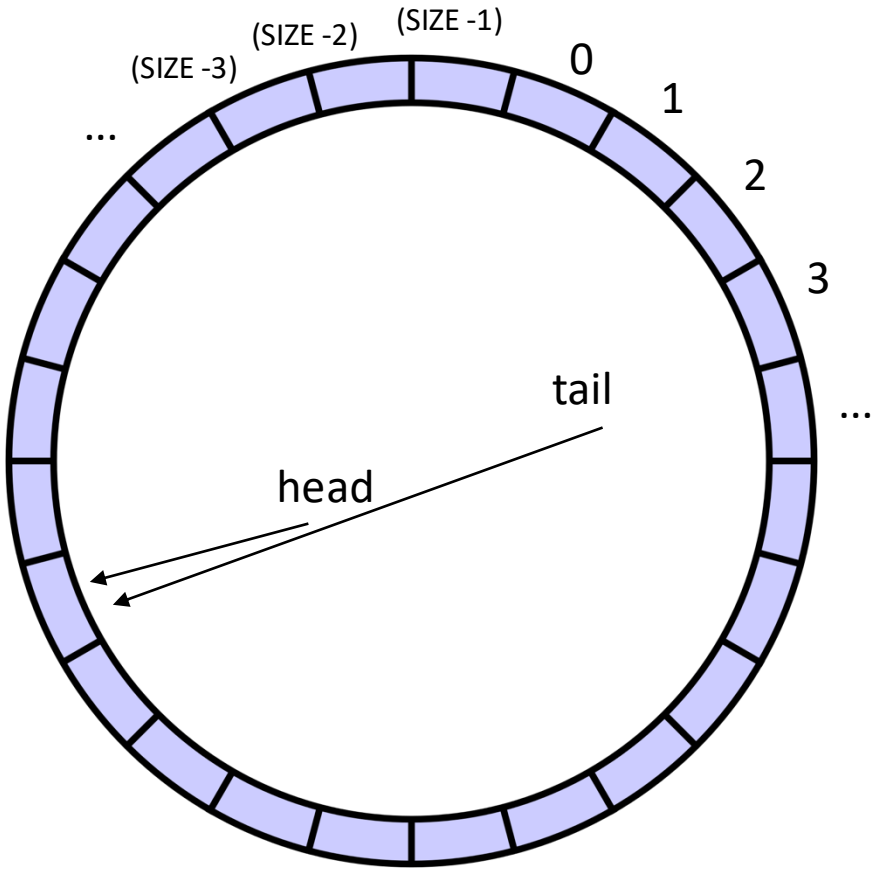



Similarly for enqueue

but why can't we enqueue?

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

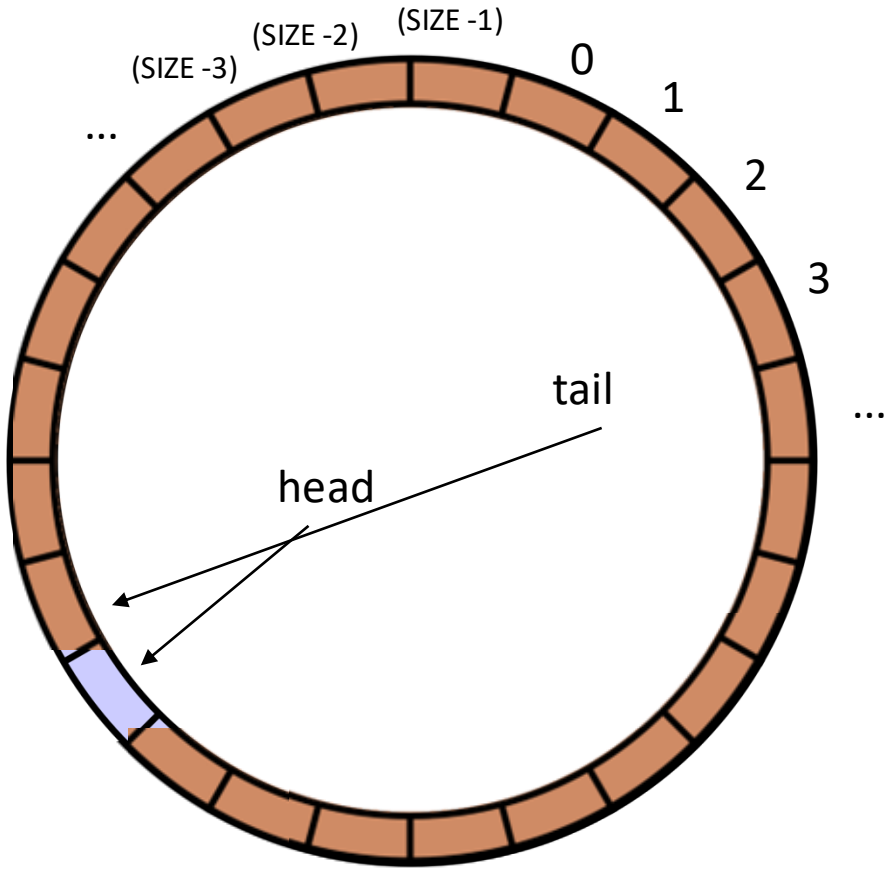
  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```




*Incrementing the head would make it empty!*

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```

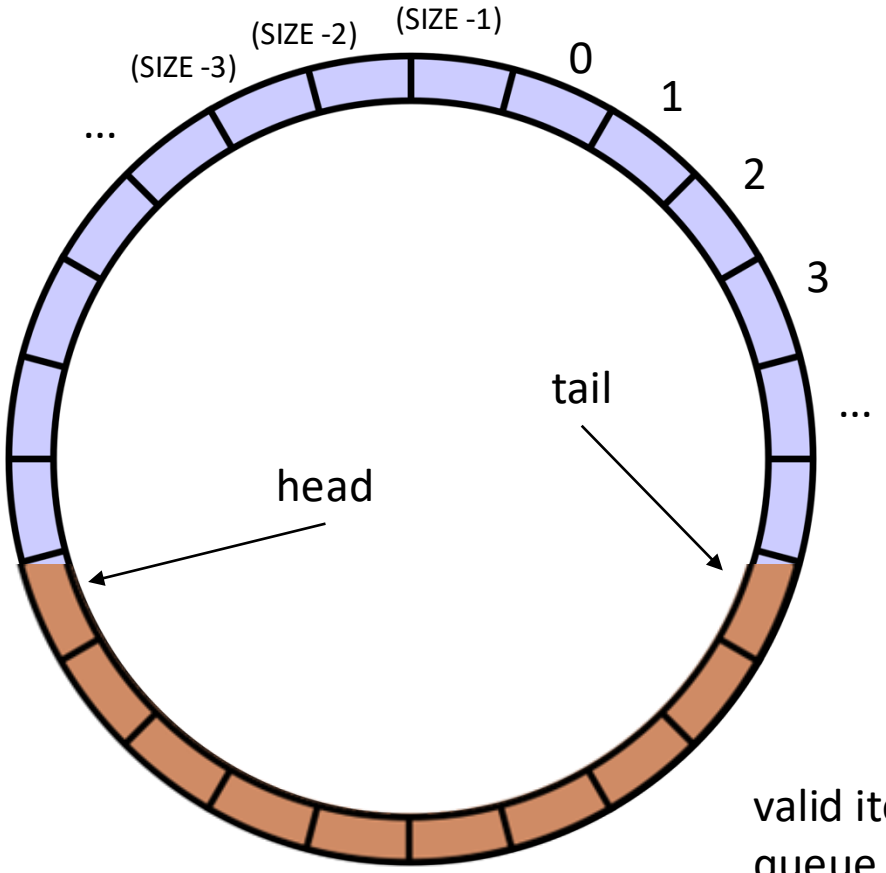



we need to wait for some room

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // wait for there to be room
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```

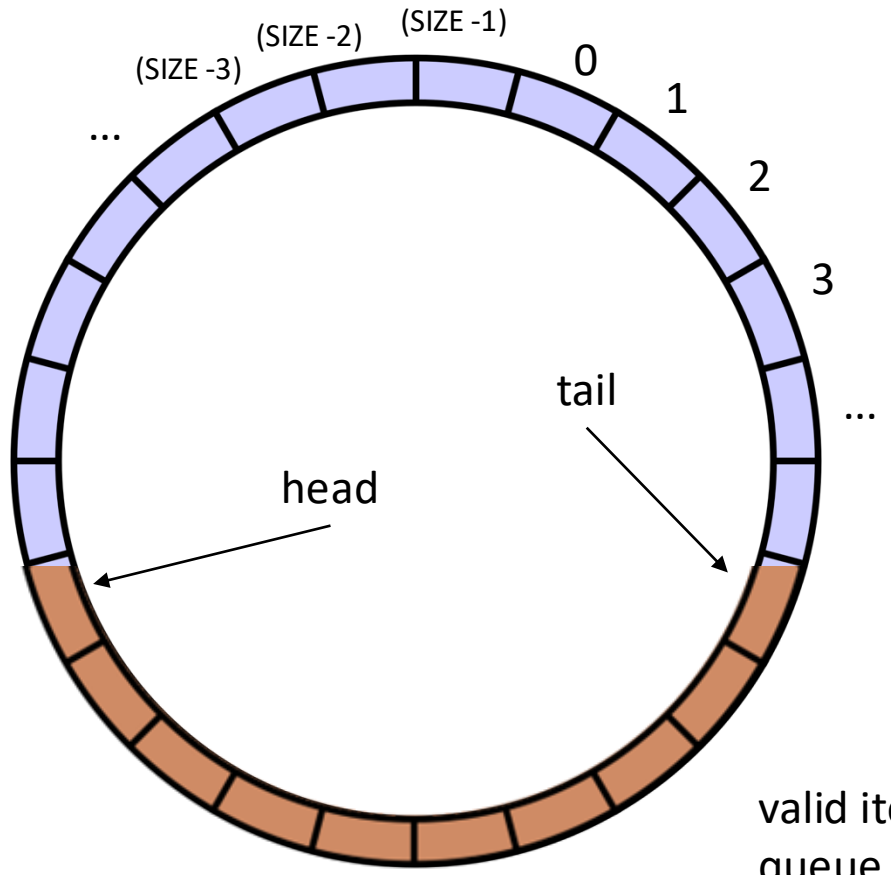
Other questions:




valid items in the queue

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // wait for there to be room
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```



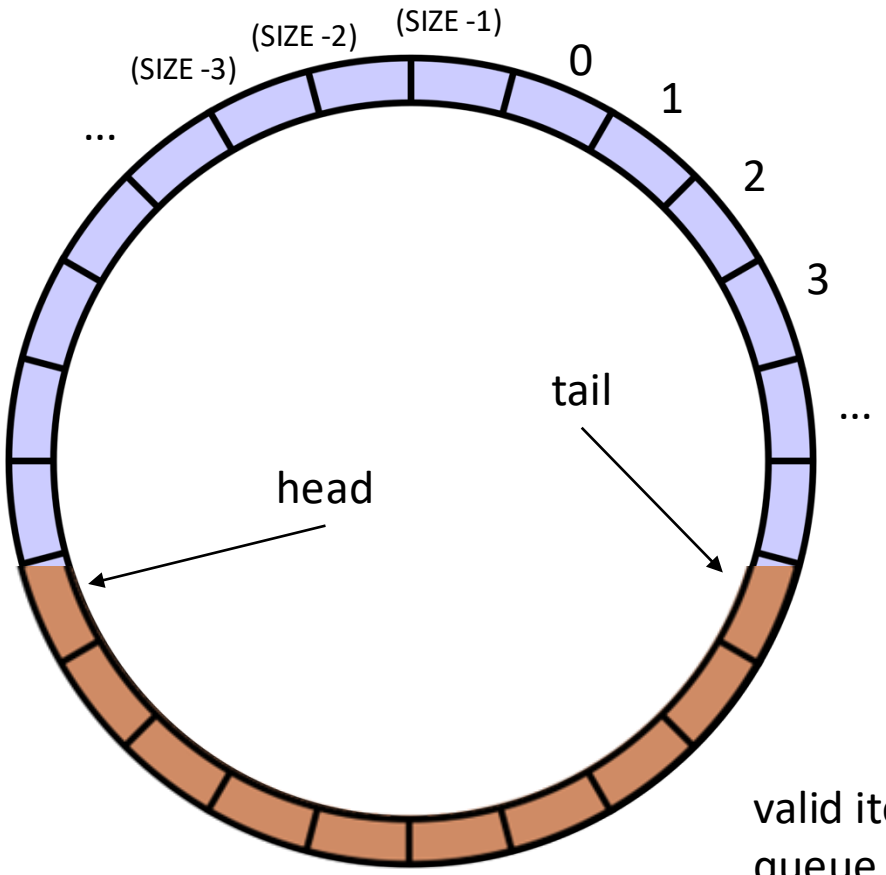

valid items in the queue

Other questions:

Do these need to be atomic RMWs?

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // wait for there to be room
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```




valid items in the queue

Other questions:

Do these need to be atomic RMWs?

No, remember 1 thread enqueues and 1 thread dequeues

```
class ProdConsQueue {
  private:
    atomic_int head;
    atomic_int tail;
    int buffer[SIZE];

  public:
    void enq(int x) {
      // wait for there to be room
      // store value at head
      // increment head
    }
    int deq() {
      // wait while queue is empty
      // get value at tail
      // increment tail
    }
}
```

# DOALL Loops

adds two arrays

```
for (int i = 0; i < SIZE; i++) {
  a[i] = b[i] + c[i];
}
```

adds elements with neighbors

```
for (int i = 0; i < SIZE; i++) {
  a[i] += a[i+1]
}
```

are they the same if you traverse them backwards?

adds two arrays

```
for (int i = 0; i < SIZE; i++) {
  a[i] = b[i] + c[i];
}
```

```
for (int i = SIZE-1; i >= 0; i--) {
  a[i] = b[i] + c[i];
}
```

adds elements with neighbors

```
for (int i = 0; i < SIZE; i++) {
  a[i] += a[i+1]
}
```

```
for (int i = SIZE-1; i >= 0; i--) {
  a[i] += a[i+1]
}
```

are they the same if you traverse them backwards?

adds two arrays

```
for (int i = 0; i < SIZE; i++) {
  a[i] = b[i] + c[i];
}
```

```
for (int i = SIZE-1; i >= 0; i--) {
  a[i] = b[i] + c[i];
}
```

adds elements with neighbors

```
for (int i = 0; i < SIZE; i++) {
  a[i] += a[i+1]
}
```

```
for (int i = SIZE-1; i >= 0; i--) {
  a[i] += a[i+1]
}
```

No!

adds two arrays

```
for (int i = 0; i < SIZE; i++) {
  a[i] = b[i] + c[i];
}
```

what about a random order?

```
for (pick i randomly) {
  a[i] = b[i] + c[i];
}
```

adds elements with neighbors

```
for (int i = 0; i < SIZE; i++) {
  a[i] += a[i+1]
}
```

```
for (pick i randomly) {
  a[i] += a[i+1]
}
```

adds two arrays

```
for (int i = 0; i < SIZE; i++) {
  a[i] = b[i] + c[i];
}
```

what about a random order?

```
for (pick i randomly) {
  a[i] = b[i] + c[i];
}
```

adds elements with neighbors

```
for (int i = 0; i < SIZE; i++) {
  a[i] += a[i+1]
}
```

```
for (pick i randomly) {
  a[i] += a[i+1]
}
```

No!

```
for (int i = 0; i < SIZE; i++) {
  a[i] = b[i] + c[i];
}
```

These are **DOALL** loops:

- Loop iterations are independent
- You can do them in ANY order and get the same results

```
for (int i = 0; i < SIZE; i++) {
  a[i] = b[i] + c[i];
}
```

These are **DOALL** loops:

- Loop iterations are independent
- You can do them in ANY order and get the same results

- Most importantly: you can do the iterations in parallel!
- Assign each thread a set of indices to compute

# DOALL Loops

- Given a nest of For loops, can we make the outer-most loop parallel?
  - Safely
  - Efficiently

# DOALL Loops

- We will consider a special type of for loop, common in scientific applications:
  - Operates on N dimensional arrays (only side-effects are array writes)
  - Array bases are disjoint and constant
  - Bounds, indexes are a function of loop variables, input variables and constants
  - Loops Increment by 1

```
for (int i = 0; i < dim1; i++) {
  for (int j = 0; j < dim3; j++) {
    for (int k = 0; k < dim2; k++) {
      a[i][j] += b[i][k] * c[k][j];
    }
  }
}
```

matrix multiplication example

# DOALL Loops

- We will consider a special type of for loop, common in scientific applications:
  - Operates on N dimensional arrays (only side-effects are array writes)
  - Array bases are disjoint and constant
  - Bounds, indexes are a function of loop variables, input variables and constants
  - Loops Increment by 1 (optional). Transform to 1.

# DOALL Loops

- Given a nest of ***candidate*** *For* loops, determine if we can we make the outer-most loop parallel?
  - Safely
  - efficiently

- Criteria: every iteration of the outer-most loop must be *independent*
  - The loop can execute in any order, and produce the same result

# Safety Criteria

- How do we check this?
  - If the property doesn't hold then there exists 2 iterations, such that if they are re-ordered, it causes different outcomes for the loop.
  - **Write-Write conflicts**: two distinct iterations write different values to the same location
  - **Read-Write conflicts**: two distinct iterations where one iteration reads from the location written to by another iteration.

# Safety Criteria

- Criteria: every iteration of the outer-most loop must be *independent*
- the loop must produce the same result for any order of the iterations

```
for (i = 0; i < size; i++) {
   a[index(i)] = loop(i);
}
```

# Safety Criteria

- Criteria: every iteration of the outer-most loop must be *independent*
- the loop must produce the same result for any order of the iterations

```
for (i = 0; i < size; i++) {
   a[index(i)] = loop(i);
}
```

Index calculation based on the loop variable

# Safety Criteria

- Criteria: every iteration of the outer-most loop must be *independent*
- the loop must produce the same result for any order of the iterations

```
for (i = 0; i < size; i++) {
   a[index(i)] = loop(i);
}
```

Index calculation based on the loop variable
Computation to store in the memory location

# Safety Criteria

- Criteria: every iteration of the outer-most loop must be *independent*
- the loop must produce the same result for any order of the iterations

```
for (i = 0; i < size; i++) {
   a[index(i)] = loop(i);
}
```

**Write-write conflicts:**

for two distinct iteration variables:

$i_x$ != $i_y$

Check:

index($i_x$) != index($i_y$)

# Safety Criteria

- Criteria: every iteration of the outer-most loop must be *independent*
- the loop must produce the same result for any order of the iterations

```
for (i = 0; i < size; i++) {
   a[index(i)] = loop(i);
}
```

**Write-write conflicts:**

for two distinct iteration variables:
$i_x$ != $i_y$
Check:
index($i_x$) != index($i_y$)

**Why?**
Because if
index($i_x$) == index($i_y$)
then:
a[index($i_x$)] will equal
either loop($i_x$) or loop($i_y$)
depending on the order

# Safety Criteria

- Criteria: every iteration of the outer-most loop must be *independent*

```
for (i = 0; i < size; i++) {
    a[write_index(i)] = a[read_index(i)] + loop(i);
}
```

**Read-write conflicts:**

for two distinct iteration variables:

$i_x$ != $i_y$

Check:

write_index($i_x$) != read_index($i_y$)

# Safety Criteria

- Criteria: every iteration of the outer-most loop must be *independent*

```
for (i = 0; i < size; i++) {
    a[write_index(i)] = a[read_index(i)] + loop(i);
}
```

**Read-write conflicts:**

for two distinct iteration variables:
$i_x$ != $i_y$
Check:
write_index($i_x$) != read_index($i_y$)

**Why?**

if $i_x$ iteration happens first, then iteration $i_y$ reads an updated value.

if $i_y$ happens first, then it reads the original value

# Examples:

```
for (i = 0; i < 128; i++) {
   a[i]= a[i]*2;
}
```

# Examples:

```
for (i = 0; i < 128; i++) {
   a[i]= a[i]*2;
}
```

Yes

```
for (i = 0; i < 128; i++) {
   a[i]= a[0]*2;
}
```

# Examples:

```
for (i = 0; i < 128; i++) {
   a[i]= a[i]*2;
}
```

```
for (i = 0; i < 128; i++) {
   a[i]= a[0]*2;
}
```

No

```
for (i = 1; i < 128; i++) {
   a[i]= a[0]*2;
}
```

# Examples:

```
for (i = 0; i < 128; i++) {
   a[i]= a[i]*2;
}
```

```
for (i = 0; i < 128; i++) {
   a[i]= a[0]*2;
}
```

```
for (i = 0; i < 128; i++) {
   a[i%64]= a[i]*2;
}
```

```
for (i = 1; i < 128; i++) {
   a[i]= a[0]*2;
}
```

Yes

# Examples:

```
for (i = 0; i < 128; i++) {
   a[i]= a[i]*2;
}
```

```
for (i = 0; i < 128; i++) {
   a[i]= a[0]*2;
}
```

```
for (i = 1; i < 128; i++) {
   a[i]= a[0]*2;
}
```

```
for (i = 0; i < 128; i++) {
   a[i%64]= a[i]*2;
}
```

No

```
i1 = 0,  i2 = 1 both writing to 0
```

# Quiz 10.

A DOALL Loop must have:

*Select one*

A. A loop variable that starts at 0 and is incremented by 1

B. loop iterations that are independent

C. be unrolled and interleaved

D. not access any memory locations

Scan to answer

**forms.gle/K5CFRh6ekk3EADUs5**

chart: docs.google.com/forms/d/e/1FAIpQLSdjt9CWaBmyCgVdo9zDPY-ERa0OntalQPaUeG1ucS5MBrB-5A/viewanalytics

# Quiz 10.

A DOALL Loop must have:

*Select one*

A. A loop variable that starts at 0 and is incremented by 1

B. loop iterations that are independent

C. be unrolled and interleaved

D. not access any memory locations

Scan to answer

**forms.gle/K5CFRh6ekk3EADUs5**

chart: docs.google.com/forms/d/e/1FAIpQLSdjt9CWaBmyCgVdo9zDPY-ERa0OntalQPaUeG1ucS5MBrB-5A/viewanalytics

# Parallel Schedules

- Consider the following program:

There are 3 arrays: `a, b, c.`

We want to compute:

```
for (int i = 0; i < SIZE; i++) {
  c[i] = a[i] + b[i];
}
```

Is this a DOALL loop?

# Parallel Schedules

- Consider the following program:

There are 3 arrays: `a, b, c.`

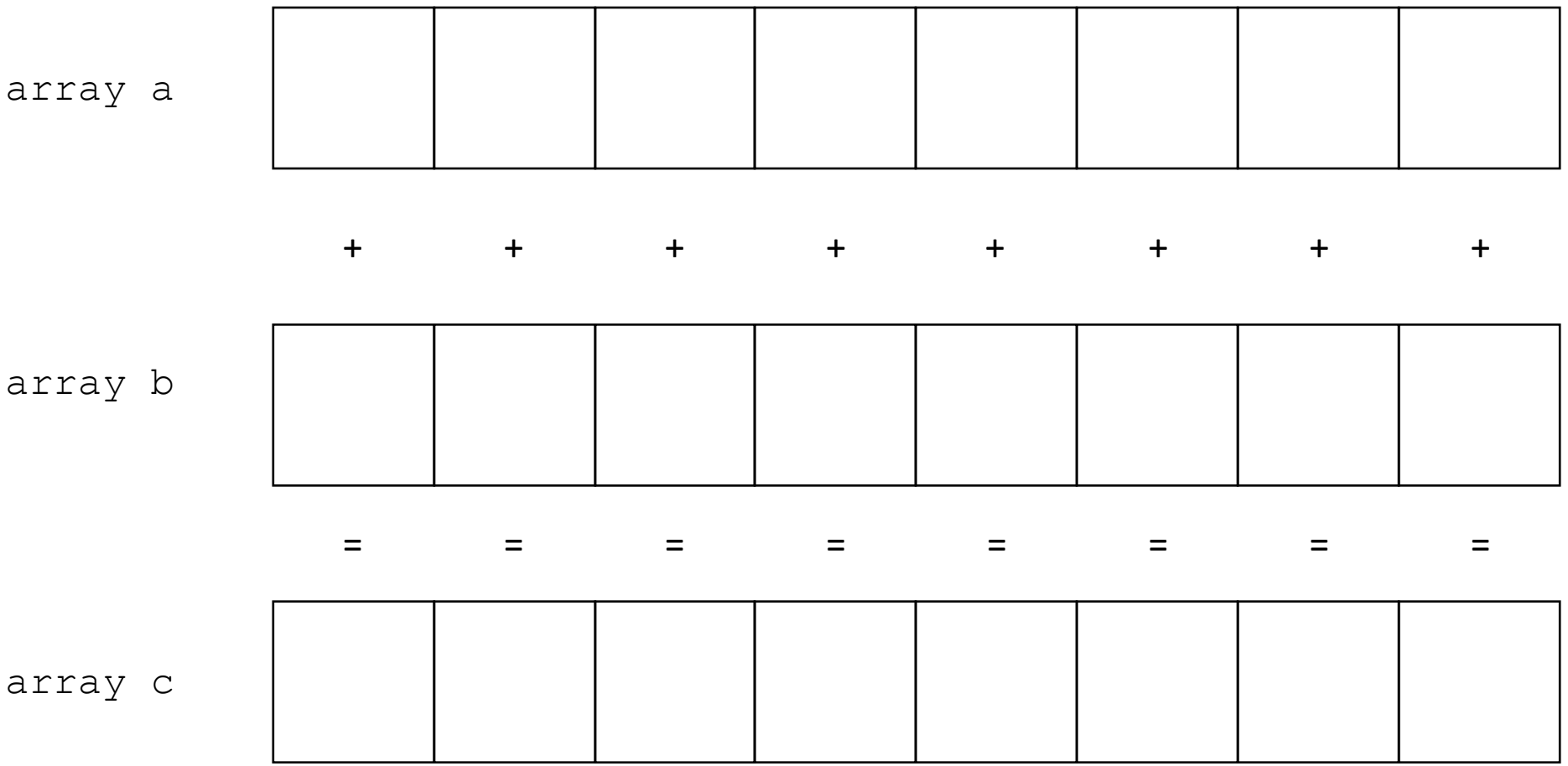
We want to compute:

```
for (int i = 0; i < SIZE; i++) {
  c[i] = a[i] + b[i];
}
```

Is this a DOALL loop?

How should we parallelize it?

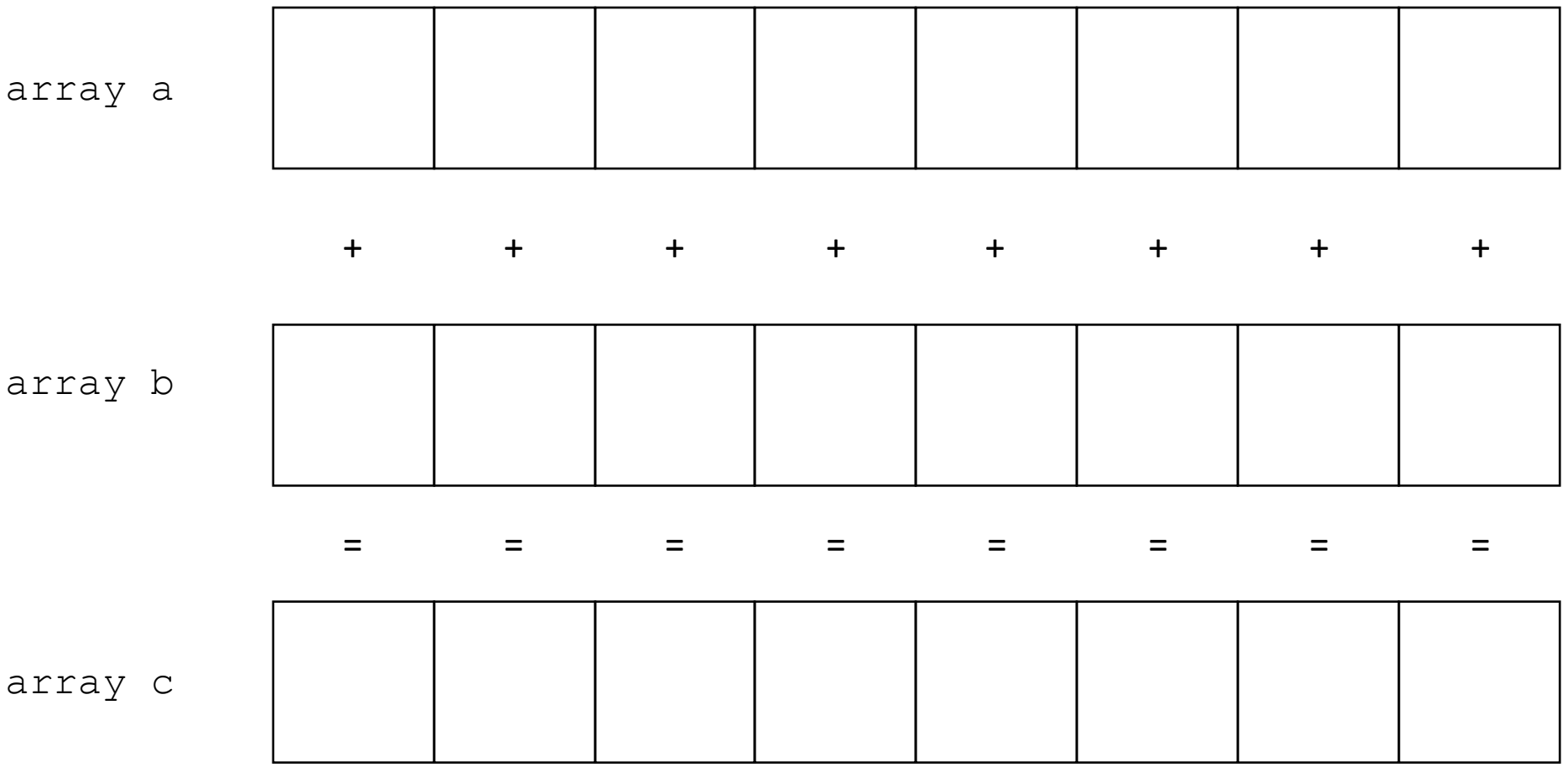
# Parallel Schedules



# Parallel Schedules

Computation can easily be divided into threads

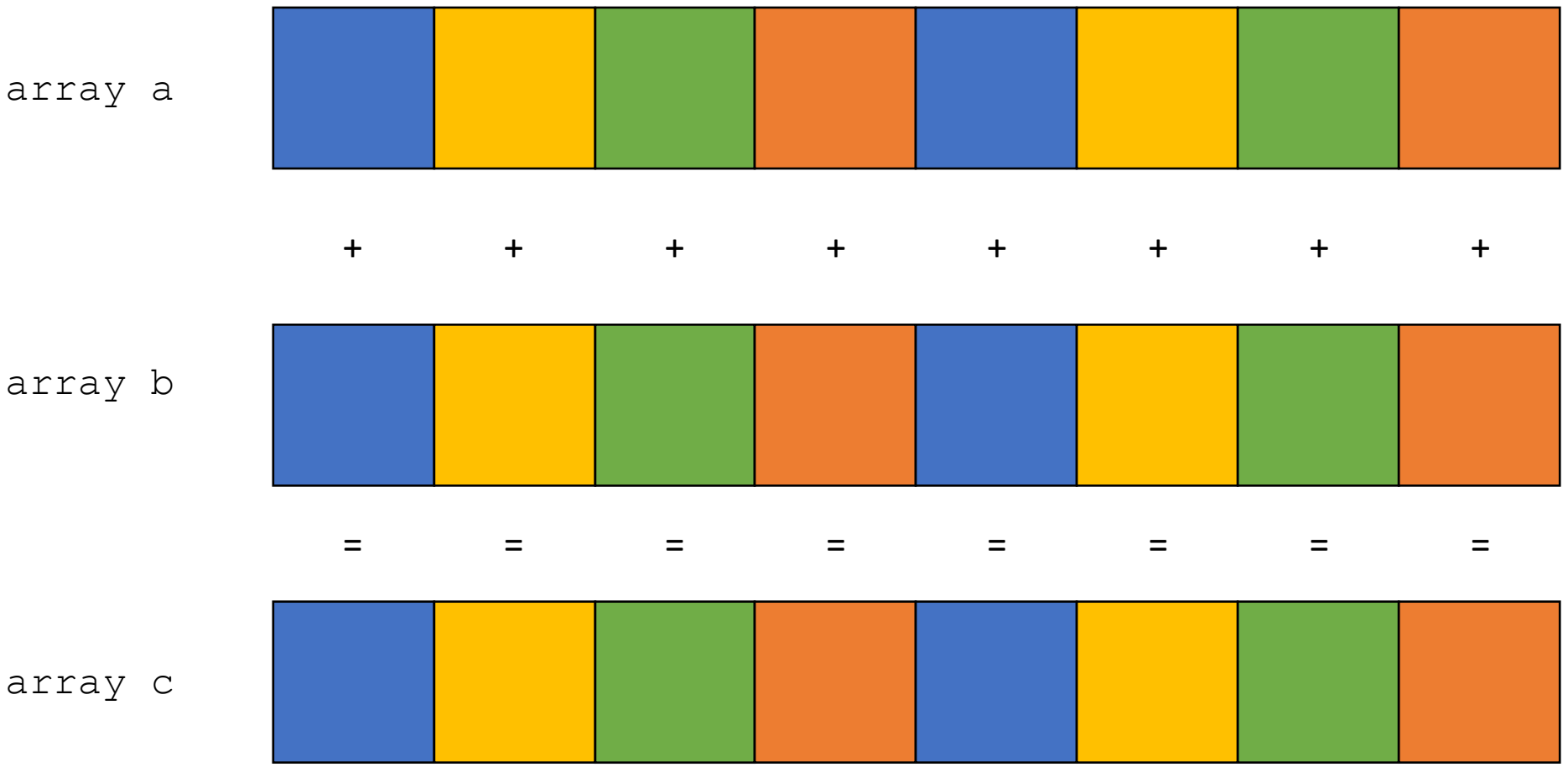
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



# Parallel Schedules

Computation can easily be divided into threads

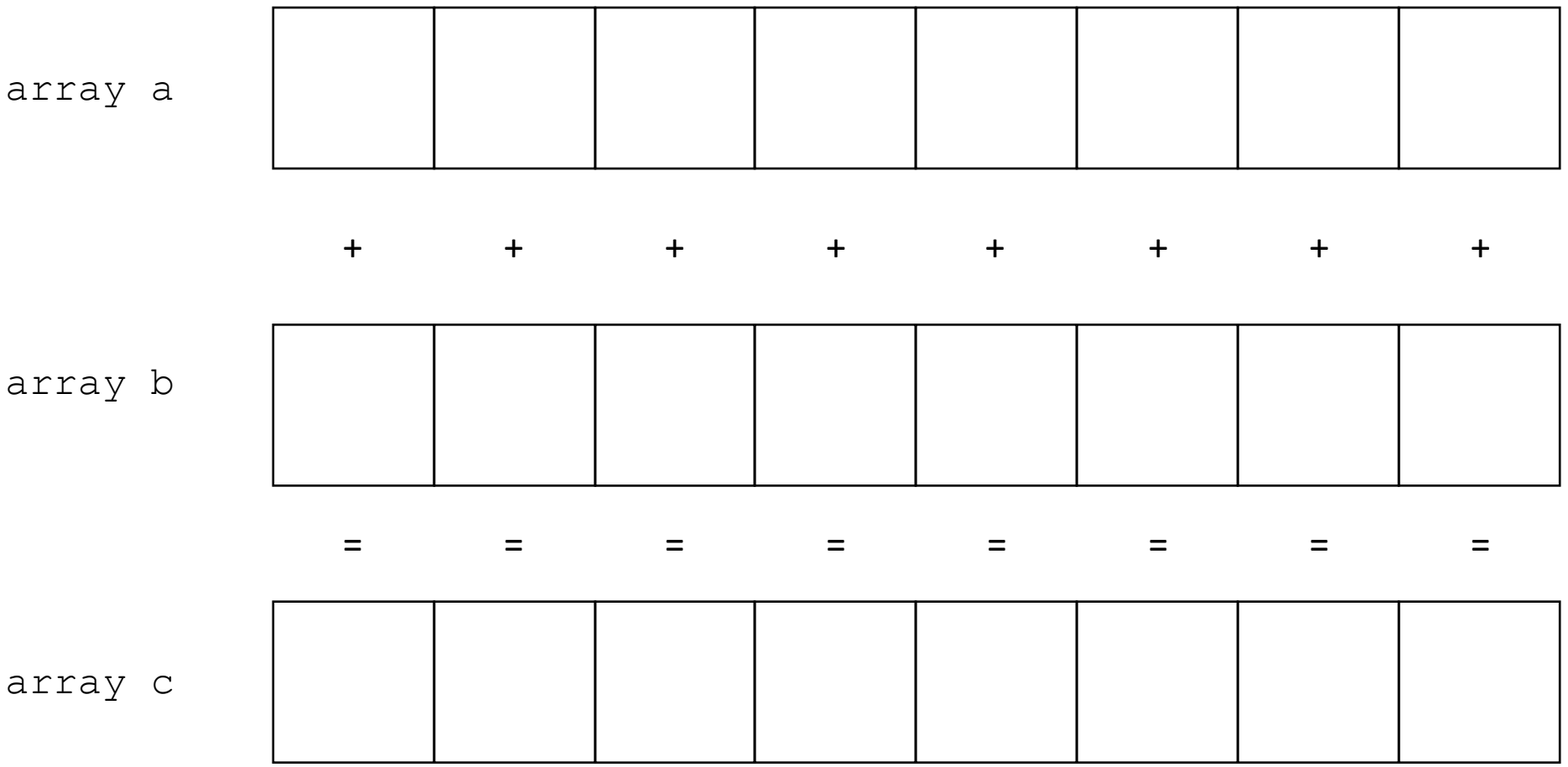
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



# Parallel Schedules

Computation can easily be divided into threads

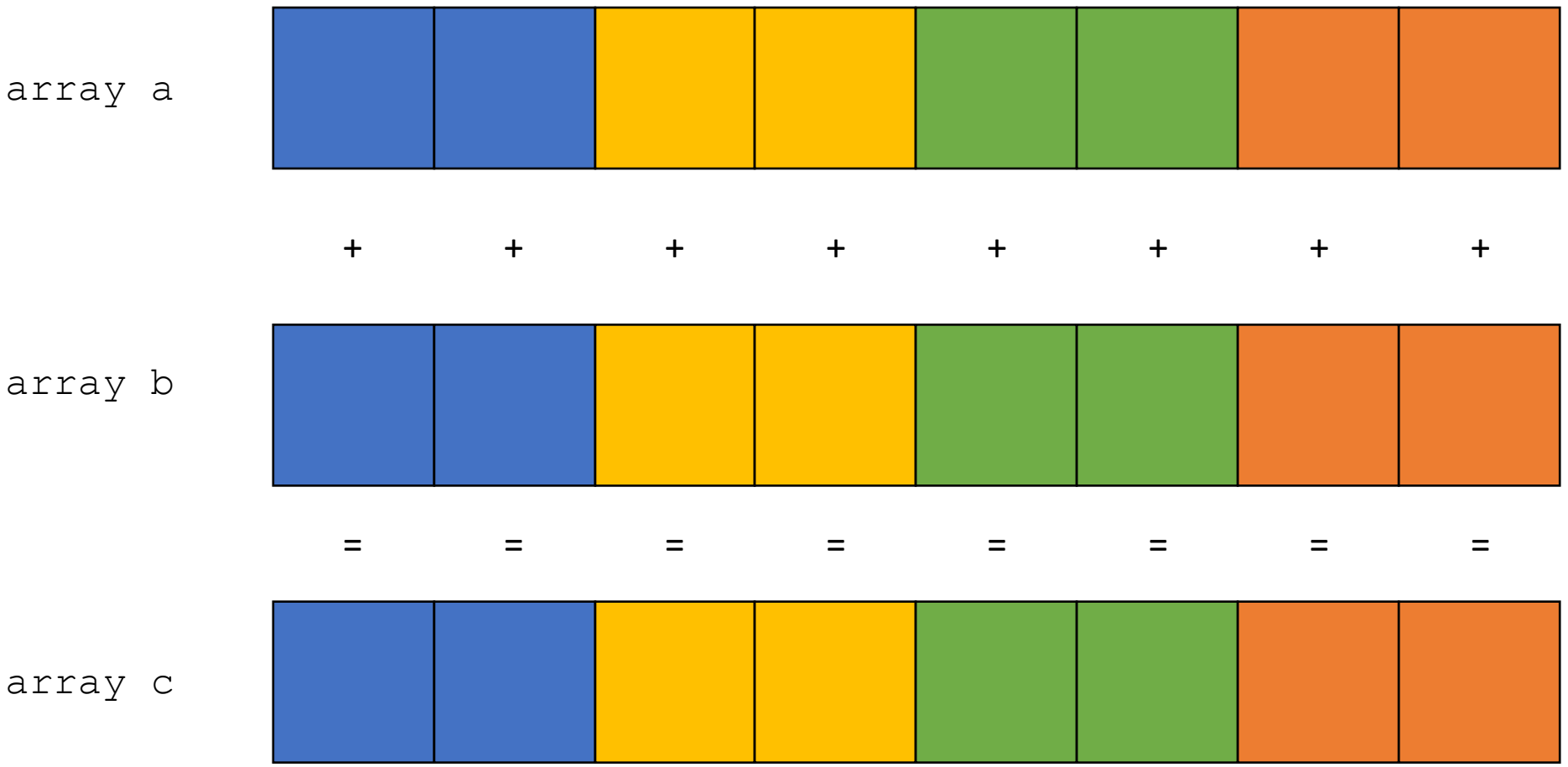
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



# Parallel Schedules

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



# Parallel Schedules

- Which one is more efficient?

# Parallel Schedules

- Which one is more efficient? The second. Avoid false sharing.

# Parallel Schedules

- Which one is more efficient?

- These are called Parallel Schedules for DOALL Loops
- We will discuss several of them.

# Quiz 10.

Given what we've learned: what role do you believe the compiler should play in parallelizing DOALL loops?

For example, should it: (1) identify them? (2) parallelize them? (3) pick a parallel schedule?

There is no right or wrong answer here, but it is interesting to think about!

*Free response: a few sentences on the form*

Scan to answer

**forms.gle/ET667AQ9wAWS43fg8**

chart: docs.google.com/forms/d/e/1FAIpQLSfIoFOfBDcozeGpa-6VvO3wHytVe0-Saylp4ChvOmsrNhgj1w/viewanalytics

# Schedule

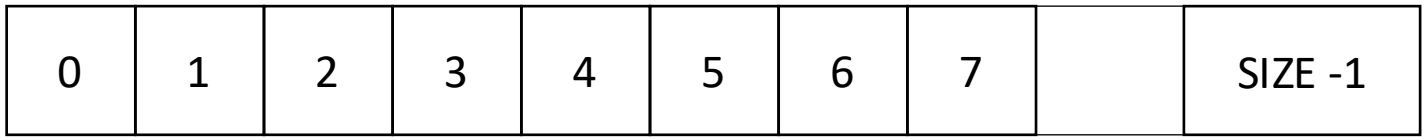
- DOALL Loops

- **Parallel Schedules:**
  - **Static**
  - Global Worklists
  - Local Worklists

# Static schedule

- Works well when loop iterations take similar amounts of time

```
void foo() {
...
  for (int x = 0; x < SIZE; x++) {
  // Each iteration takes roughly
  // equal time
  }
...
}
```

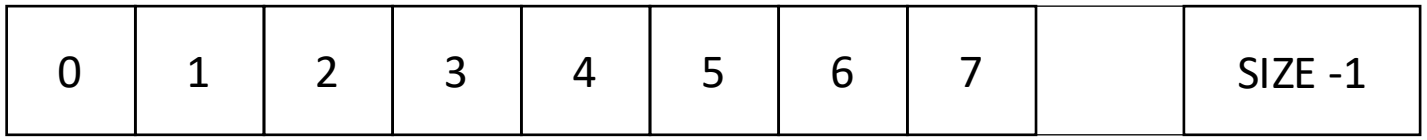


# Static schedule

- Works well when loop iterations take similar amounts of time

```
void foo() {
...
  for (int x = 0; x < SIZE; x++) {
  // Each iteration takes roughly
  // equal time
  }
...
}
```

*say SIZE / NUM_THREADS = 4*

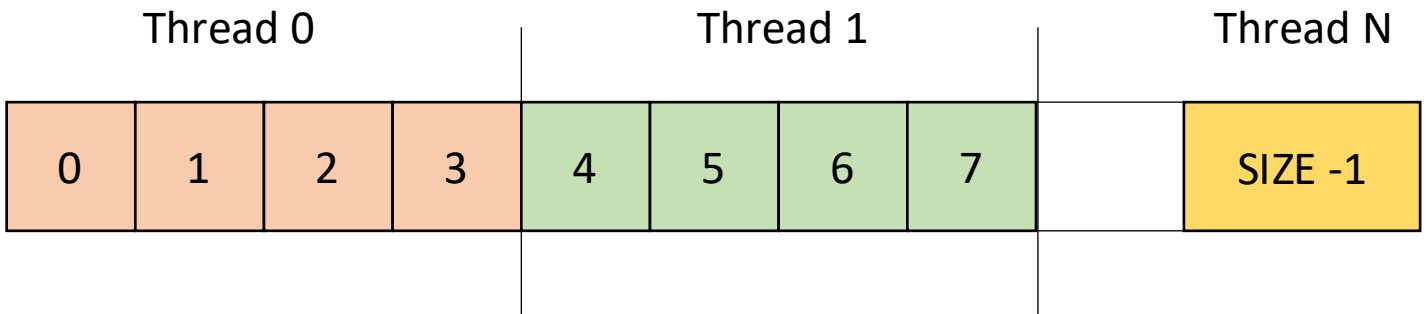


# Static schedule

- Works well when loop iterations take similar amounts of time

```
void foo() {
...
  for (int x = 0; x < SIZE; x++) {
  // Each iteration takes roughly
  // equal time
  }
...
}
```

*say SIZE / NUM_THREADS = 4*



# Static schedule

- Works well when loop iterations take similar amounts of time

```
void foo() {
...
  for (int x = 0; x < SIZE; x++) {
  // Each iteration takes roughly
  // equal time
  }
...
}
```

make a new function with the for loop inside. Pass all needed variables as arguments. Take an extra argument for a thread id

# Static schedule

- Works well when loop iterations take similar amounts of time

```
void foo() {
...
  for (int x = 0; x < SIZE; x++) {
  // Each iteration takes roughly
  // equal time
  }
...
}
```

```
void parallel_loop(..., int tid, int num_threads)
{

  for (int x = 0; x < SIZE; x++) {
    // work based on x
  }
}
```

make a new function with the for loop inside. Pass all needed variables as arguments. Take an extra argument for a thread id

# Static schedule

- Works well when loop iterations take similar amounts of time

```
void foo() {
...
  for (int x = 0; x < SIZE; x++) {
  // Each iteration takes roughly
  // equal time
  }
...
}
```

```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  for (int x = 0; x < SIZE; x++) {
    // work based on x
  }
}
```

determine chunk size in new function

# Static schedule

- Works well when loop iterations take similar amounts of time

```
void foo() {
...
  for (int x = 0; x < SIZE; x++) {
  // Each iteration takes roughly
  // equal time
  }
...
}
```

```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

Set new loop bounds

# Static schedule

- Works well when loop iterations take similar amounts of time

```
void foo() {
...
  for (int t = 0; t < NUM_THREADS; t++) {
    spawn(parallel_loop(..., t, NUM_THREADS))
  }
  join();
...
}
```

```
void parallel_loop(..., int tid, int num_threads)
{

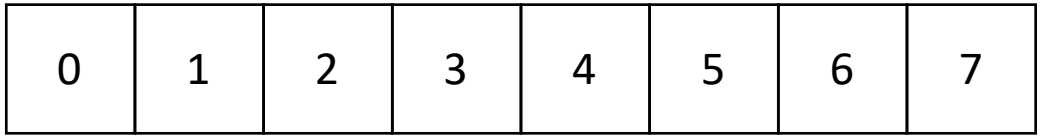
  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

You will need to adapt the thread spawn, join to C++

Spawn threads

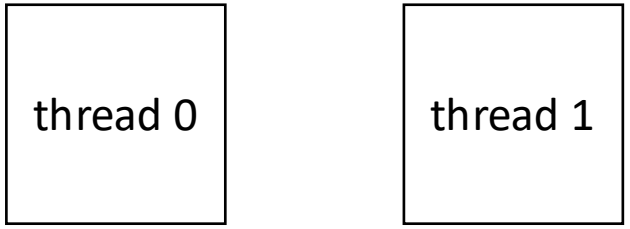
# Static schedule

- Example, 2 threads/cores, array of size 8




```
chunk_size = ?

0: start = ?    1: start = ?
0: end = ?      1: end = ?
```




```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

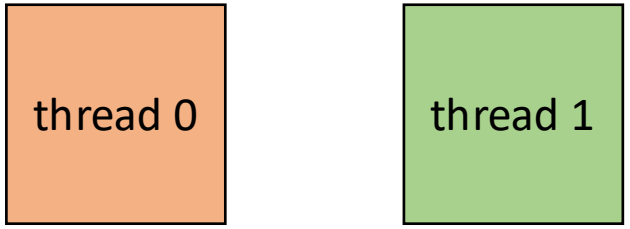
- Example, 2 threads/cores, array of size 8




```
chunk_size = 4
```

```
0: start = 0
0: end = 4
```

```
1: start = 4
1: end = 8
```





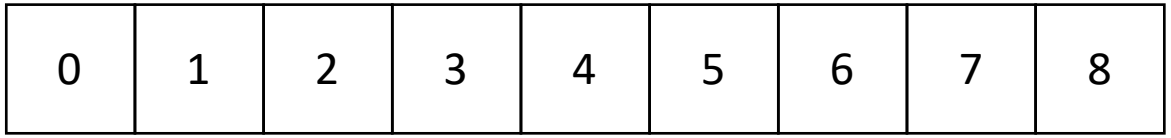


```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

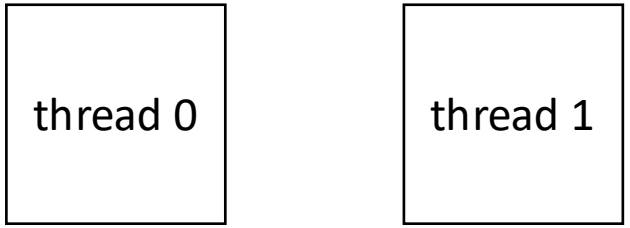
# Static schedule

- Example, 2 threads/cores, array of size 9




```
chunk_size = ?

0: start = ?    1: start = ?
0: end = ?      1: end = ?
```

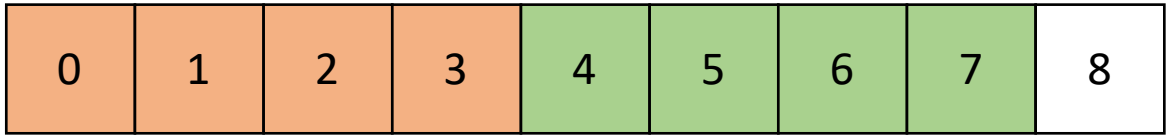



```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

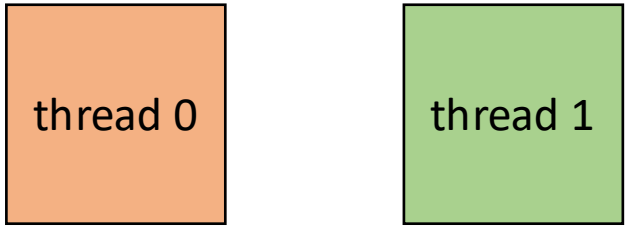
- Example, 2 threads/cores, array of size 9




```
chunk_size = 4
```

```
0: start = 0
0: end = 4
```

```
1: start = 4
1: end = 8
```





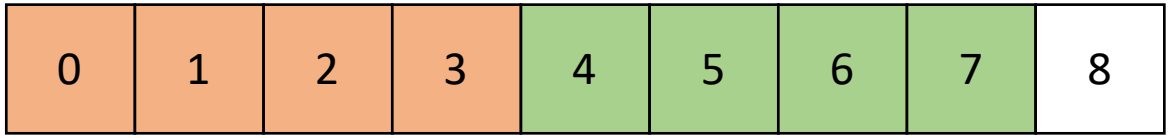


```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

- Example, 2 threads/cores, array of size 9




chunk_size = 4

0: start = 0

0: end = 4

1: start = 4

1: end = 8

thread 0

thread 1

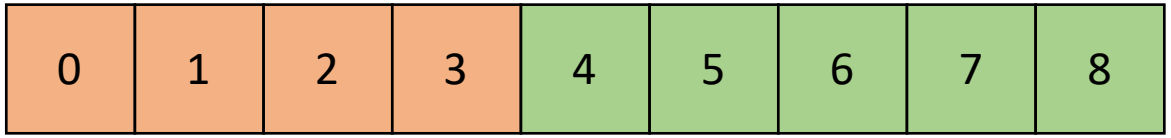
```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  if (tid == num_threads - 1) {
    end = SIZE;
  }
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

last thread gets more work

- Example, 2 threads/cores, array of size 9




```
chunk_size = 4
```

```
0: start = 0
0: end = 4
```

```
1: start = 4
1: end = ?
```

thread 0

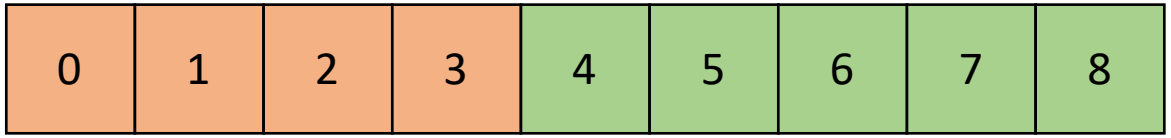
thread 1

```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  if (tid == num_threads - 1) {
    end = SIZE;
  }
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

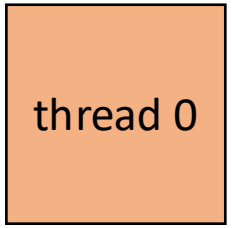
- Example, 2 threads/cores, array of size 9




```
chunk_size = 4
```

```
0: start = 0
0: end = 4
```

```
1: start = 4
1: end = 9
```







last thread gets more work

What is the worst case?

```
void parallel_loop(..., int tid, int num_threads)
{

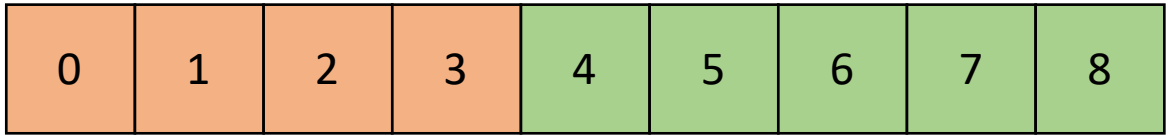
  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  if (tid == num_threads - 1) {
    end = SIZE;
  }
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

- Example, 2 threads/cores, array of size 9

last thread gets more work

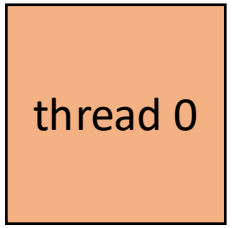
What is the worst case?
3 items




```
chunk_size = 4
```

```
0: start = 0
0: end = 4
```

```
1: start = 4
1: end = 9
```





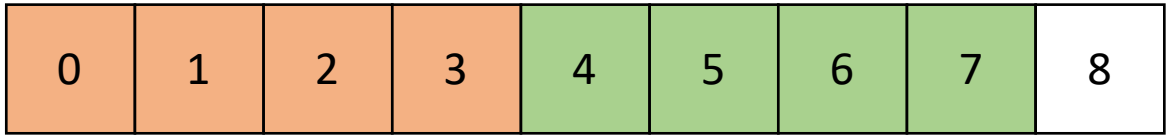


```
void parallel_loop(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  if (tid == num_threads - 1) {
    end = SIZE;
  }
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

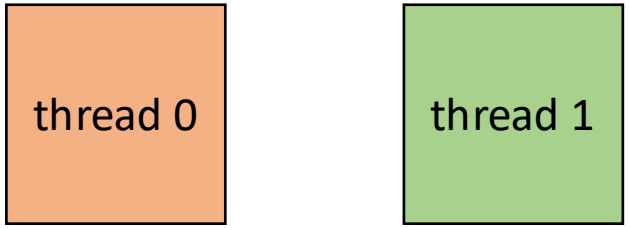
- Example, 2 threads/cores, array of size 9




```
chunk_size = 4
```

```
0: start = 0
0: end = 4
```

```
1: start = 4
1: end = 8
```

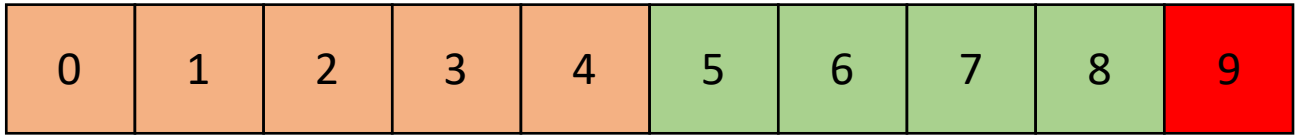



ceiling division, this will distribute uneven work in the last thread to all other threads

```
void parallel_loop(..., int tid, int num_threads)
{
  int chunk_size =
   (SIZE+(NUM_THREADS-1))/NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

- Example, 2 threads/cores, array of size 9

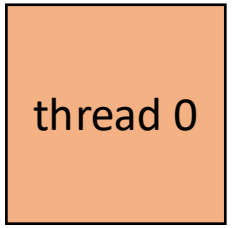



*out of bounds*

```
chunk_size = 5
```

```
0: start = 0
0: end = 5
```

```
1: start = 5
1: end = 10
```



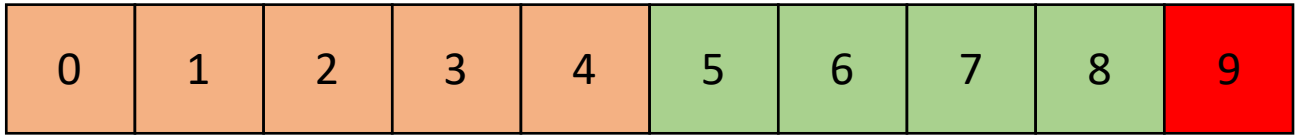




```
void parallel_loop(..., int tid, int num_threads)
{
  int chunk_size =
   (SIZE+(NUM_THREADS-1))/NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

- Example, 2 threads/cores, array of size 9




*out of bounds*

```
chunk_size = 5
```

```
0: start = 0
0: end = 5
```

```
1: start = 5
1: end = 10
```

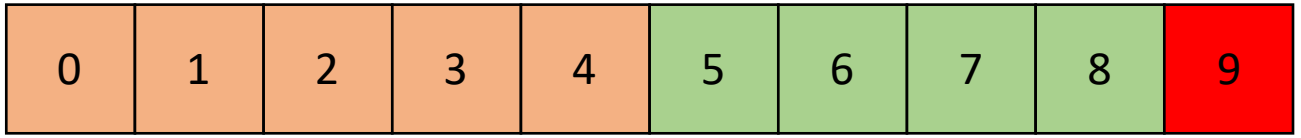
thread 0

thread 1

```
void parallel_loop(..., int tid, int num_threads)
{
  int chunk_size =
   (SIZE+(NUM_THREADS-1))/NUM_THREADS;
  int start = chunk_size * tid;
  int end =
    min(start+chunk_size, SIZE)
for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Static schedule

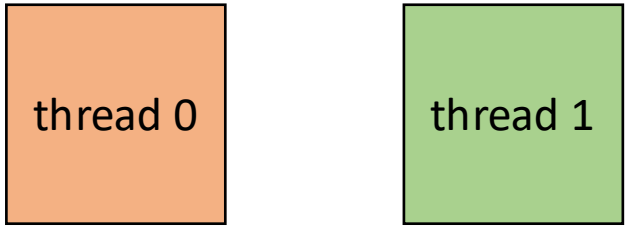
- Example, 2 threads/cores, array of size 9




```
chunk_size = 5
```

```
0: start = 0
0: end = 5
```

```
1: start = 5
1: end = 9
```




most threads do equal amounts of work, last thread may do less.

Which one is better/worse?
Approach 1:
Max slowdown for last thread does all the extra work? 3 items
Approach 2: (with ceiling)
Max slowdown is 1 item.

```
void parallel_loop(..., int tid, int num_threads)
{
  int chunk_size =
   (SIZE+(NUM_THREADS-1))/NUM_THREADS;
  int start = chunk_size * tid;
  int end =
    min(start+chunk_size, SIZE)
for (int x = start; x < end; x++) {
    // work based on x
  }
}
```

# Quiz 10.

Which of the following solutions can guarantee that a static schedule will not be out of bounds?

*Select one*

A. The last thread always checks to get the minimum between the end of the array and the last location assigned to it.

B. The last thread always get the end of the array

C. The last thread never receives more than N tasks

Scan to answer

**forms.gle/2o8uR7DYsVavisBr9**

chart: docs.google.com/forms/d/e/1FAIpQLSeA2mK90xWGCJP7xpnu-qq_ygBHzHD0do_Gn8LOCArbSPpnEA/viewanalytics

# Quiz 10.

Which of the following solutions can guarantee that a static schedule will not be out of bounds?

*Select one*

A. The last thread always checks to get the minimum between the end of the array and the last location assigned to it.

B. The last thread always get the end of the array

C. The last thread never receives more than N tasks

Scan to answer

**forms.gle/2o8uR7DYsVavisBr9**

chart: docs.google.com/forms/d/e/1FAIpQLSeA2mK90xWGCJP7xpnu-qq_ygBHzHD0do_Gn8LOCArbSPpnEA/viewanalytics

# Schedule

- DOALL Loops

- **Parallel Schedules:**
  - Static
  - **Global Worklists**
  - Local Worklists

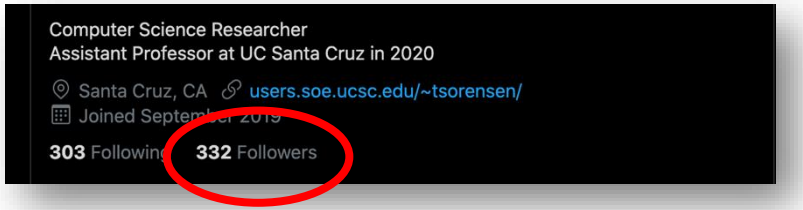
# Irregular parallelism in loops

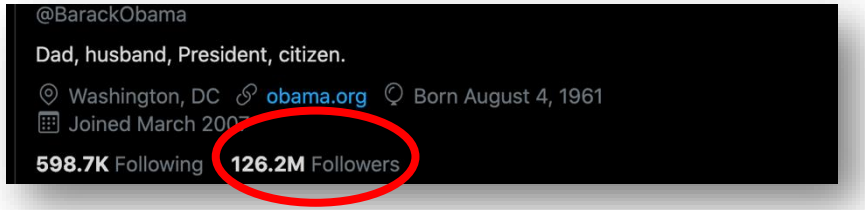
- Tasks are not balanced
- Appears in lots of emerging workloads

# Irregular parallelism in loops

- Tasks are not balanced
- Appears in lots of emerging workloads

  social network analytics where threads are parallel across users



# Irregular parallelism in loops

- Independent iterations have different amount of work to compute
- Threads with longer tasks take longer to compute.
- Threads with shorter tasks are under utilized.

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

irregular (or unbalanced) parallelism: each x iteration performs different amount of work.

# Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

# Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

# Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by first thread:

$$\text{t1_work} = \sum_{n=0}^{SIZE/2} n$$

# Irregular parallelism in loops

- Calculate imbalance cost if x is chunked:
  - Thread 1 takes iterations 0 - SIZE/2
  - Thread 2 takes iterations SIZE/2 - SIZE

```
for (x = 0; x < SIZE; x++) {
  for (y = x; y < SIZE; y++) {
    a[x,y] = b[x,y] + c[x,y];
  }
}
```

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by second thread:

$$\text{t2_work} = \sum_{n=0}^{SIZE/2} n$$

Calculate work work done by first thread:

t1_work = total_work − t2_work

# Irregular parallelism in loops

Example: SIZE = 64

total_work = 2016
t2_work = 496
t1_work = 1520

t1 does ~3x more work than t2

Only provides ~1.3x speedup

*Potential solution:*
Have T1 do only ¼ of the iterations
Gives a better speedup of 1.77x

*Not a feasible solution because often times load imbalance is not given by a static equation on loop bounds!*

Calculate how much total work:

$$\text{total_work} = \sum_{n=0}^{SIZE} n$$

Calculate work done by first thread:

$$\text{t1_work} = \sum_{n=0}^{SIZE/2} n$$

Calculate work work done by second thread:

t2_work = total_work − t1_work

# Work stealing

- Tasks are dynamically assigned to threads.

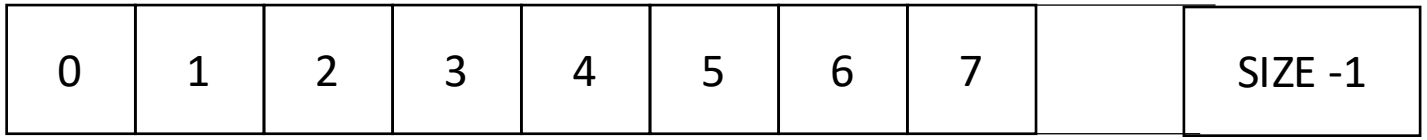
# Work stealing - global worklist

- Pros
  - Simple to implement

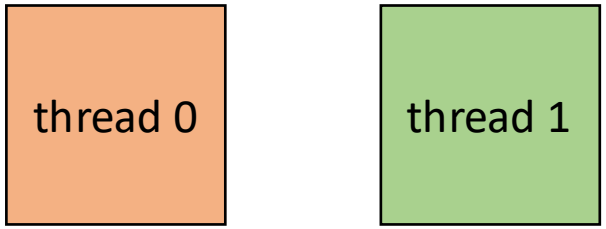
- Cons:
  - High contention on global counter
  - Potentially bad memory locality.

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically

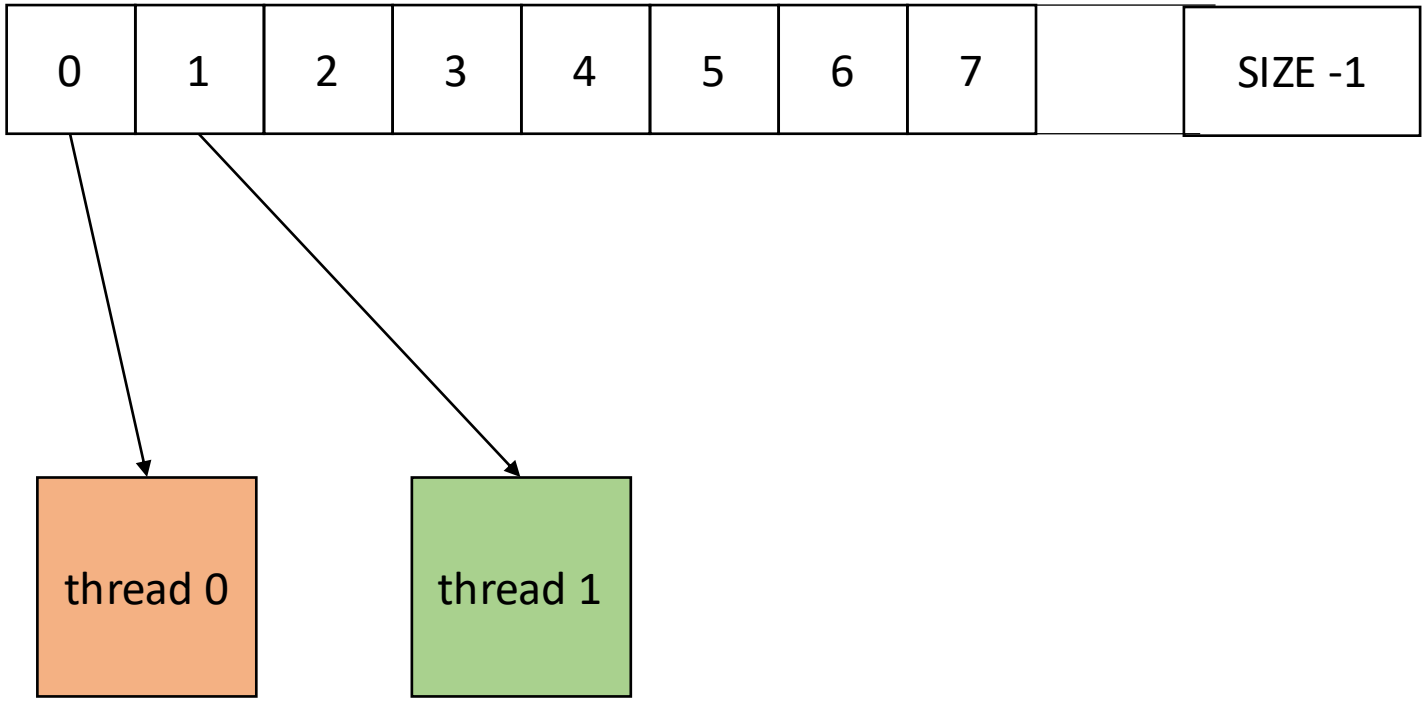



*cannot color initially!*



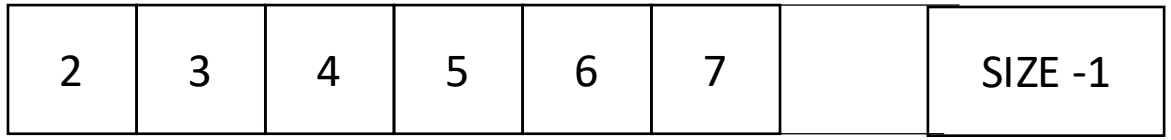
# Work stealing - global worklist

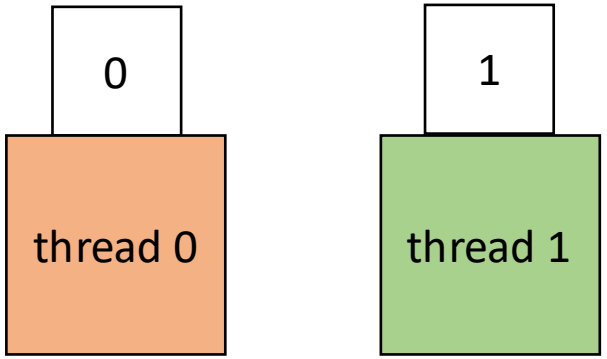
- Global worklist: threads take tasks (iterations) dynamically



# Work stealing - global worklist

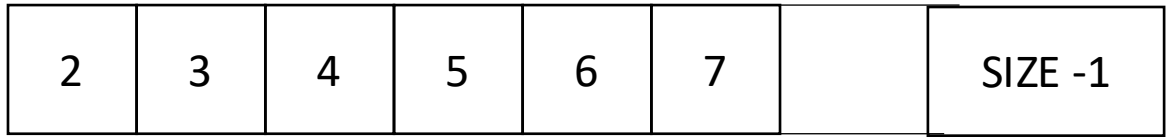
- Global worklist: threads take tasks (iterations) dynamically

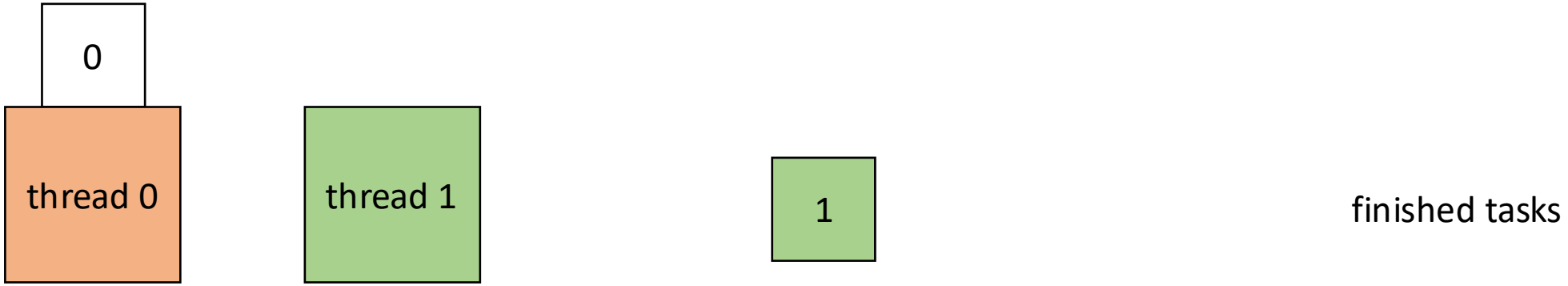


# Work stealing - global worklist

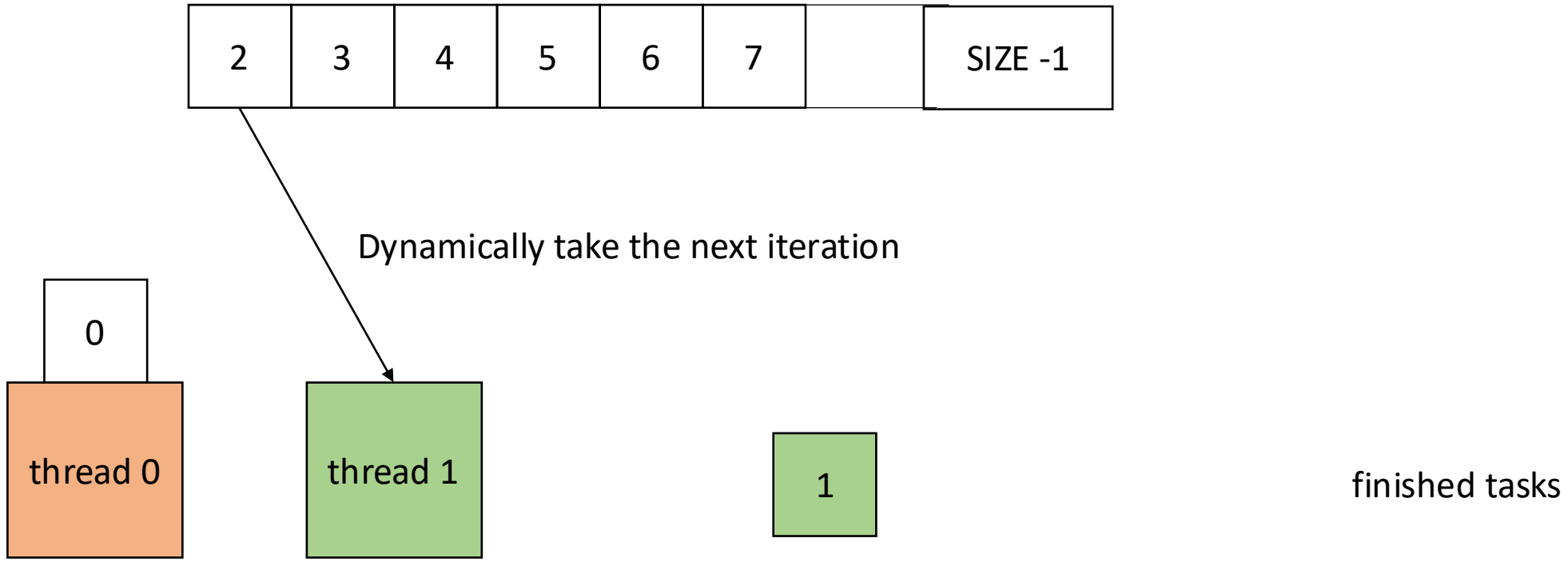
- Global worklist: threads take tasks (iterations) dynamically



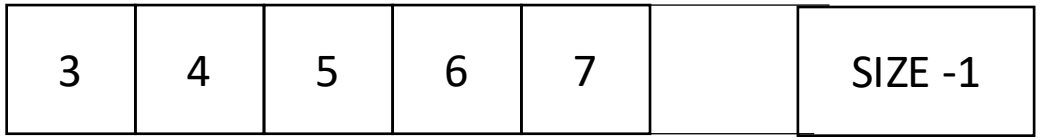
# Work stealing - global worklist

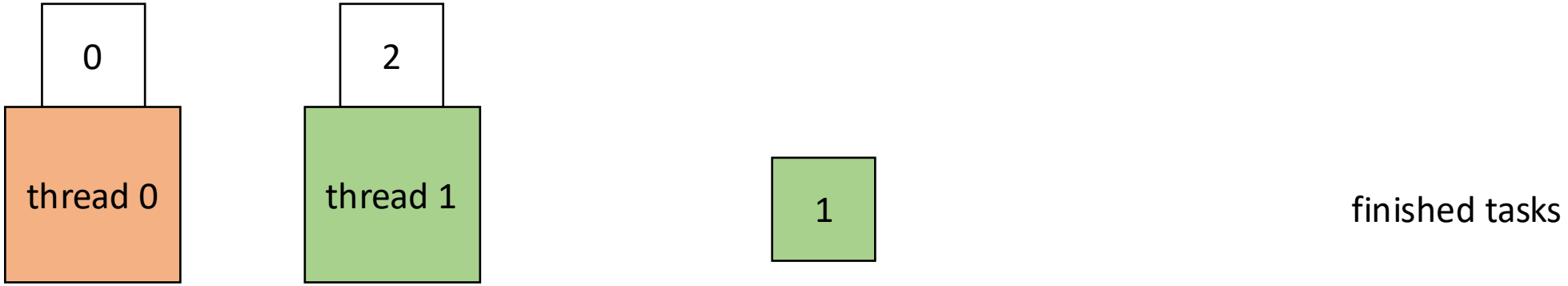
- Global worklist: threads take tasks (iterations) dynamically



# Work stealing - global worklist

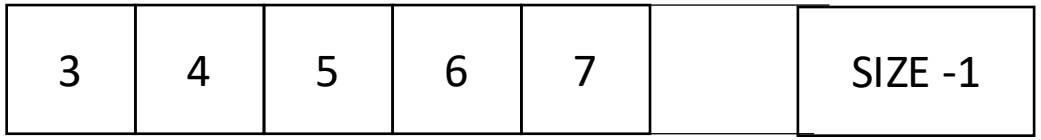
- Global worklist: threads take tasks (iterations) dynamically

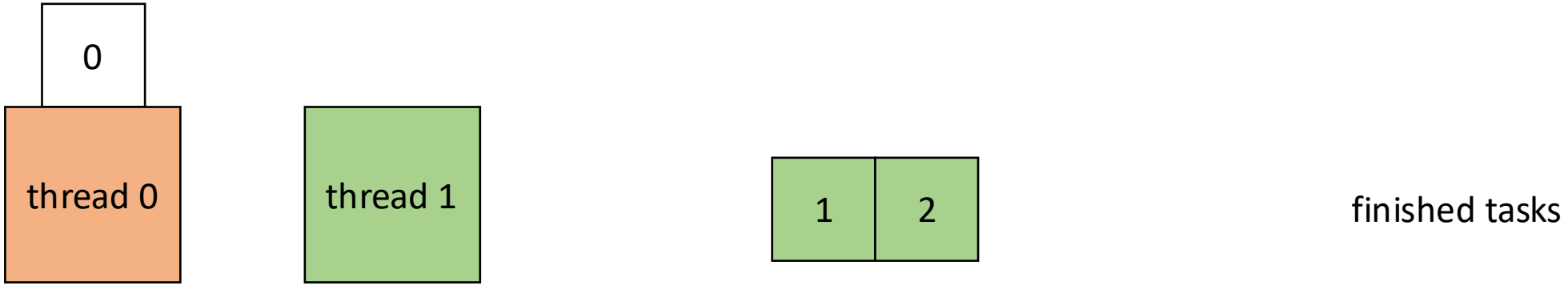


# Work stealing - global worklist

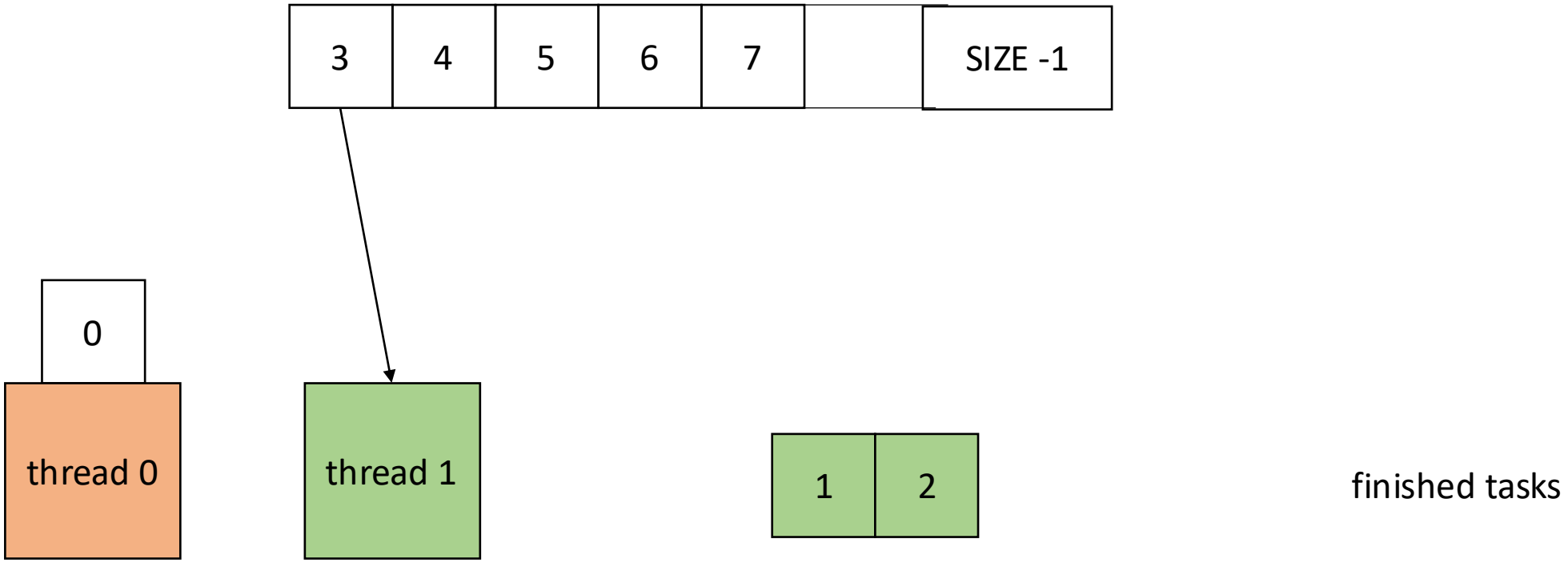
- Global worklist: threads take tasks (iterations) dynamically



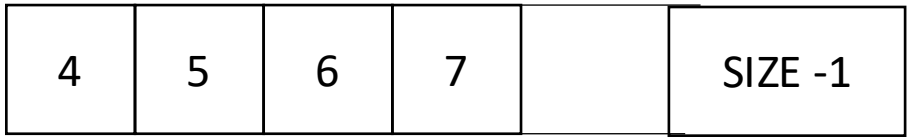
# Work stealing - global worklist

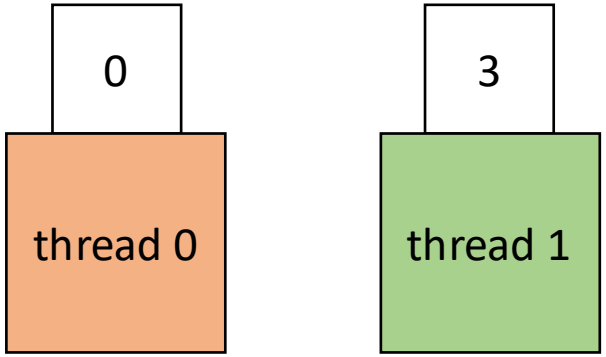
- Global worklist: threads take tasks (iterations) dynamically



# Work stealing - global worklist

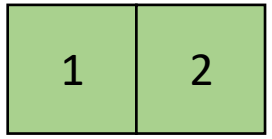
- Global worklist: threads take tasks (iterations) dynamically





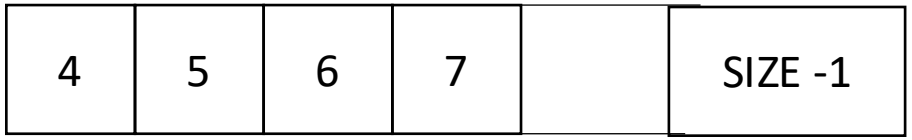


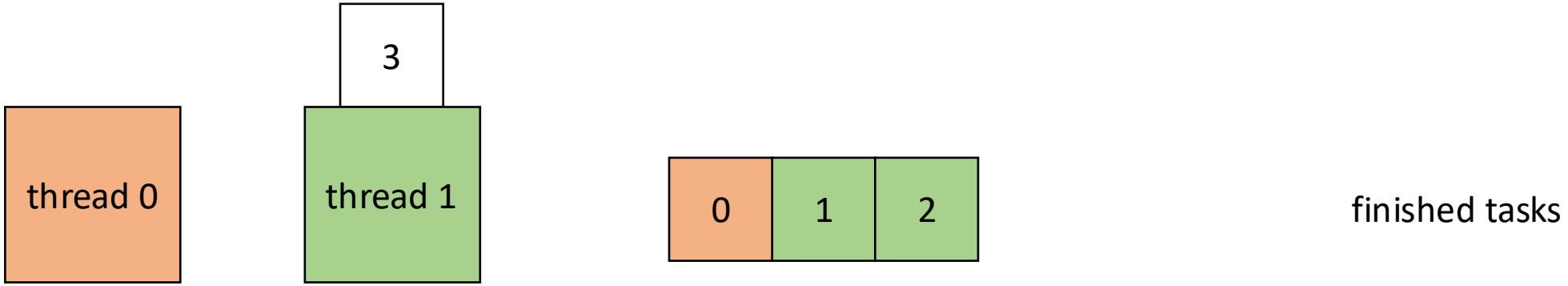



finished tasks

# Work stealing - global worklist

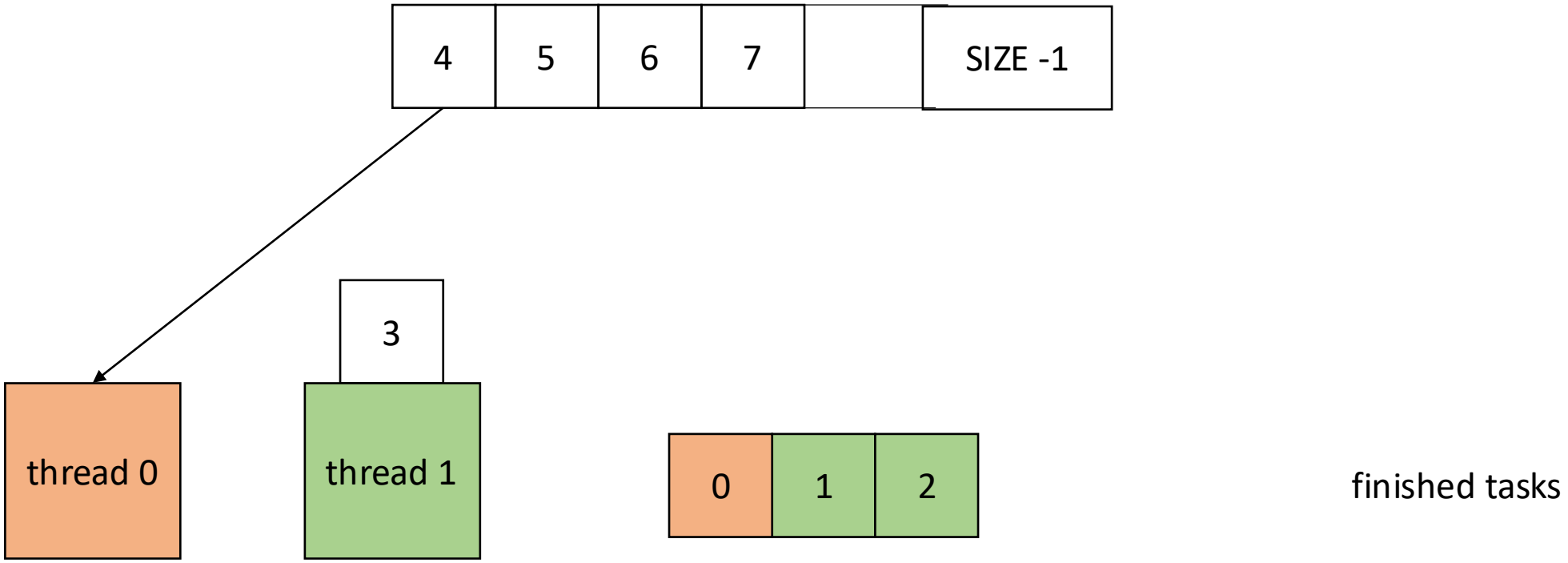
- Global worklist: threads take tasks (iterations) dynamically



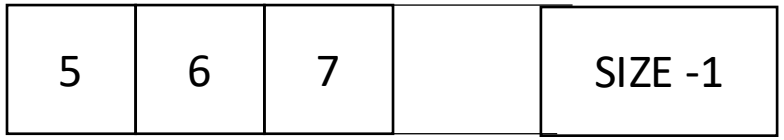
# Work stealing - global worklist

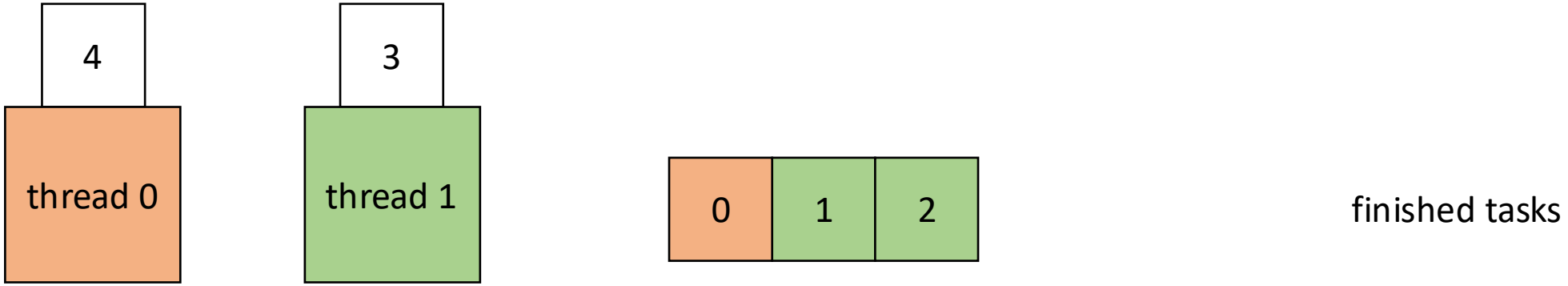
- Global worklist: threads take tasks (iterations) dynamically



# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically



# Work stealing - global worklist

- How to implement

```
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

# Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
void parallel_loop(...) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Replicate code in a new function. Pass all needed variables as arguments.

# Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

move loop variable to be a global atomic variable

# Work stealing - global worklist

- How to implement

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
      // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

   for (int local_x = ??
            local_x < SIZE;
            local_x = ??) {

      // dynamic work based on x
   }
}
```

change loop bounds in new function to use a local variable using global variable.

# Work stealing - global worklist

- How to implement

*These must be atomic updates!*

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
      // dynamic work based on x
   }
   ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

   for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

      // dynamic work based on x
   }
}
```

change loop bounds in new function to use a local variable using global variable.

# Work stealing - global worklist

- How to implement

```
void foo() {
  ...
  for (t = 0; x < THREADS; t++) {
    spawn(parallel_loop);
  }
  join();
  ...
}
```

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

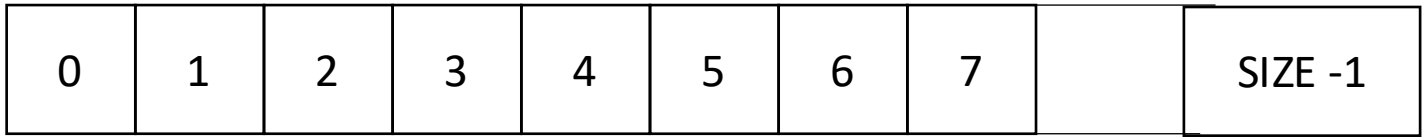
     // dynamic work based on x
  }
}
```

Spawn threads in original function and join them afterwards

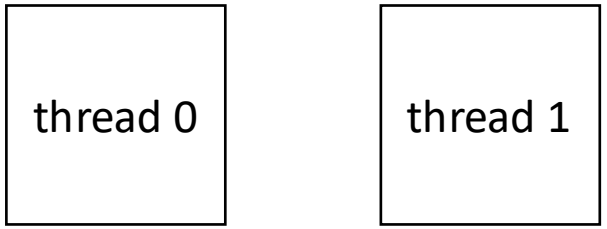
You will have to change to C++ syntax for the homework!

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 0
0 - local_x - UNDEF
1 - local_x - UNDEF
```







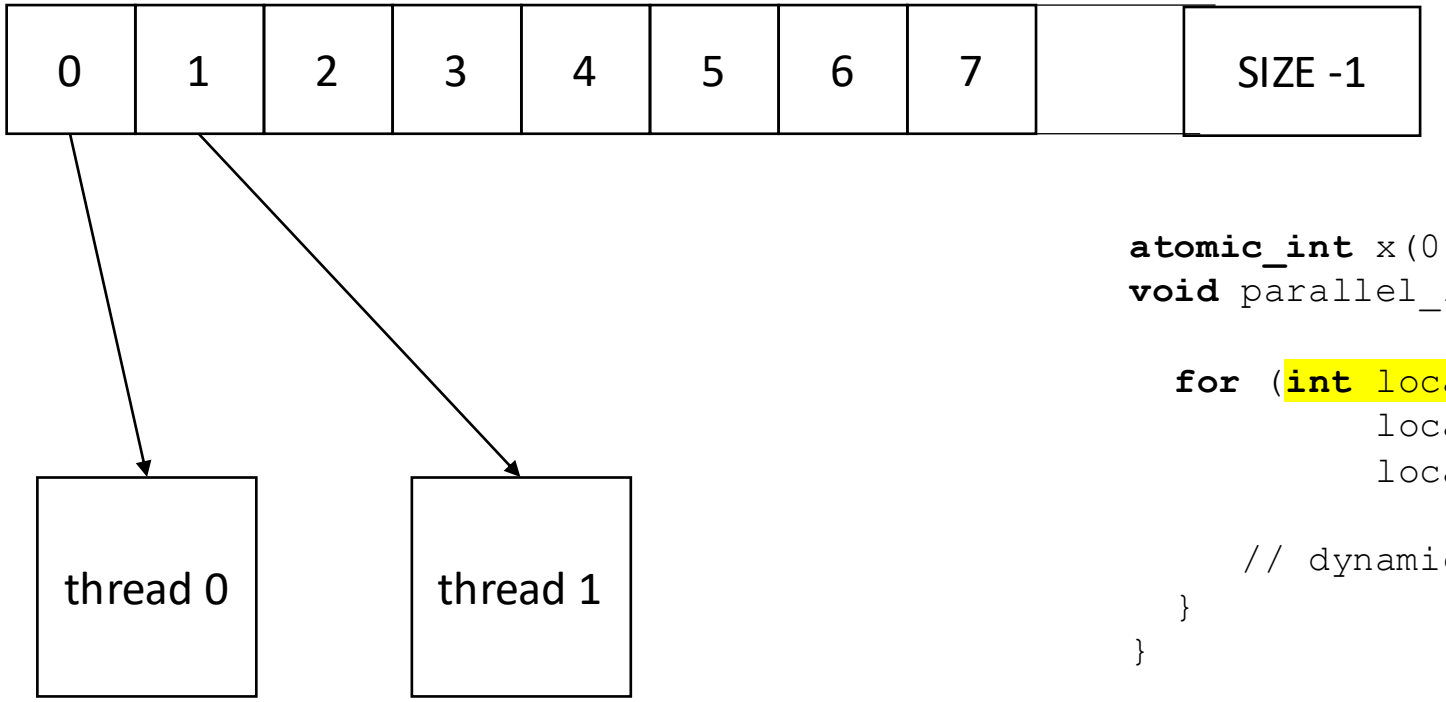
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
           local_x < SIZE;
           local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```

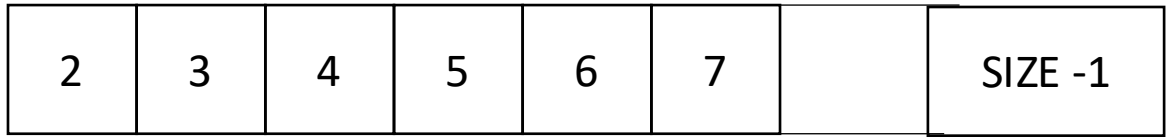
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

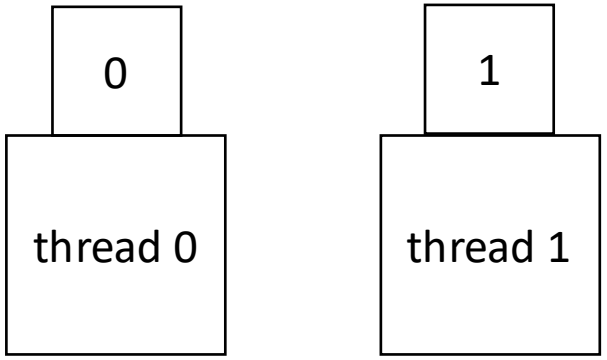
     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```







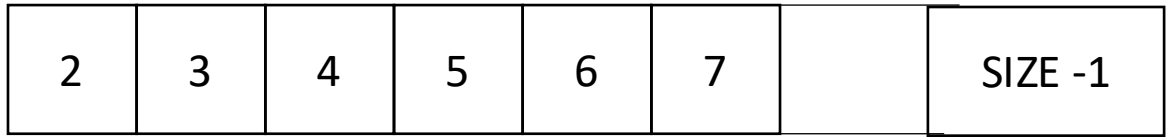
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
           local_x < SIZE;
           local_x = atomic_fetch_add(&x,1)) {

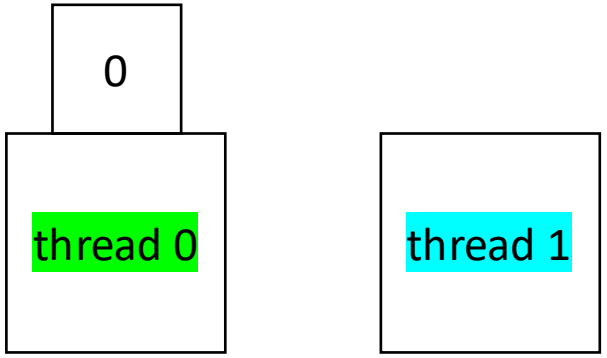
     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 2
0 - local_x - 0
1 - local_x - 1
```



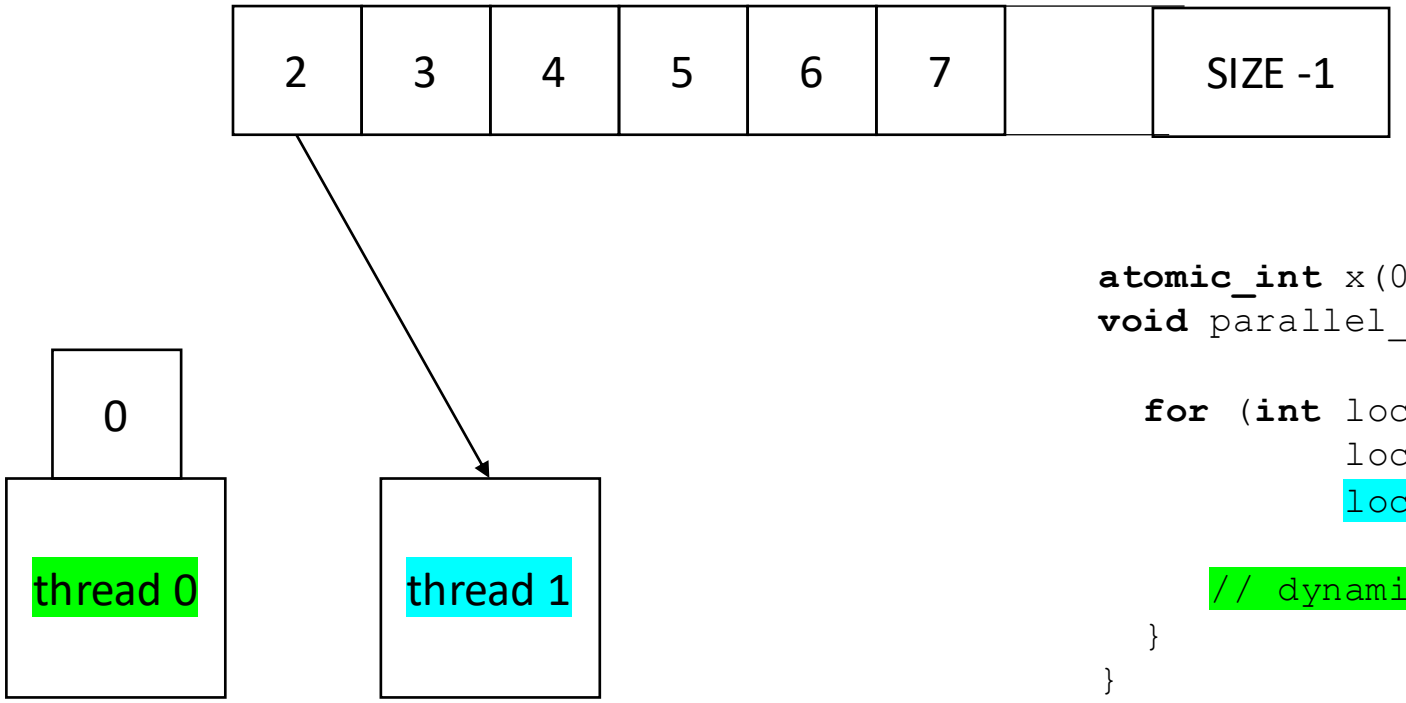

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```

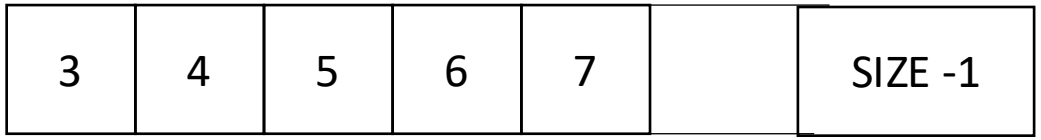
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

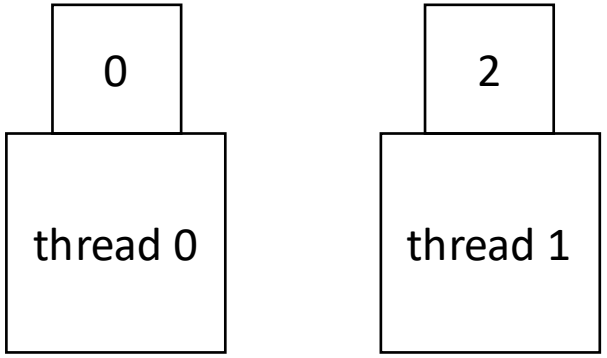
      // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```



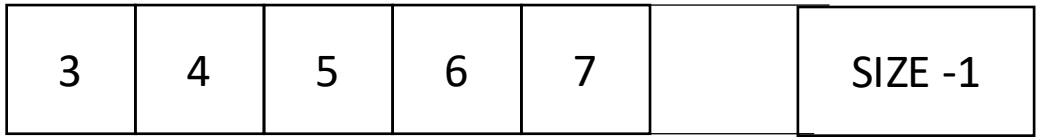

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

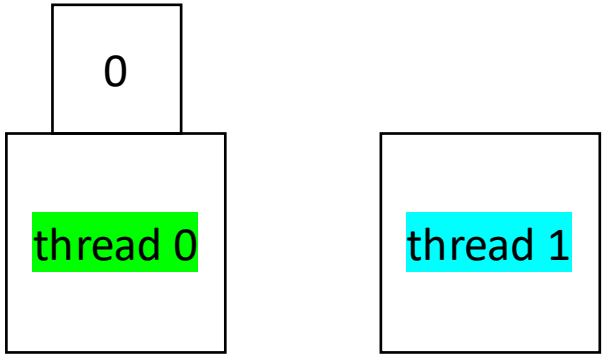
     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 3
0 - local_x - 0
1 - local_x - 2
```



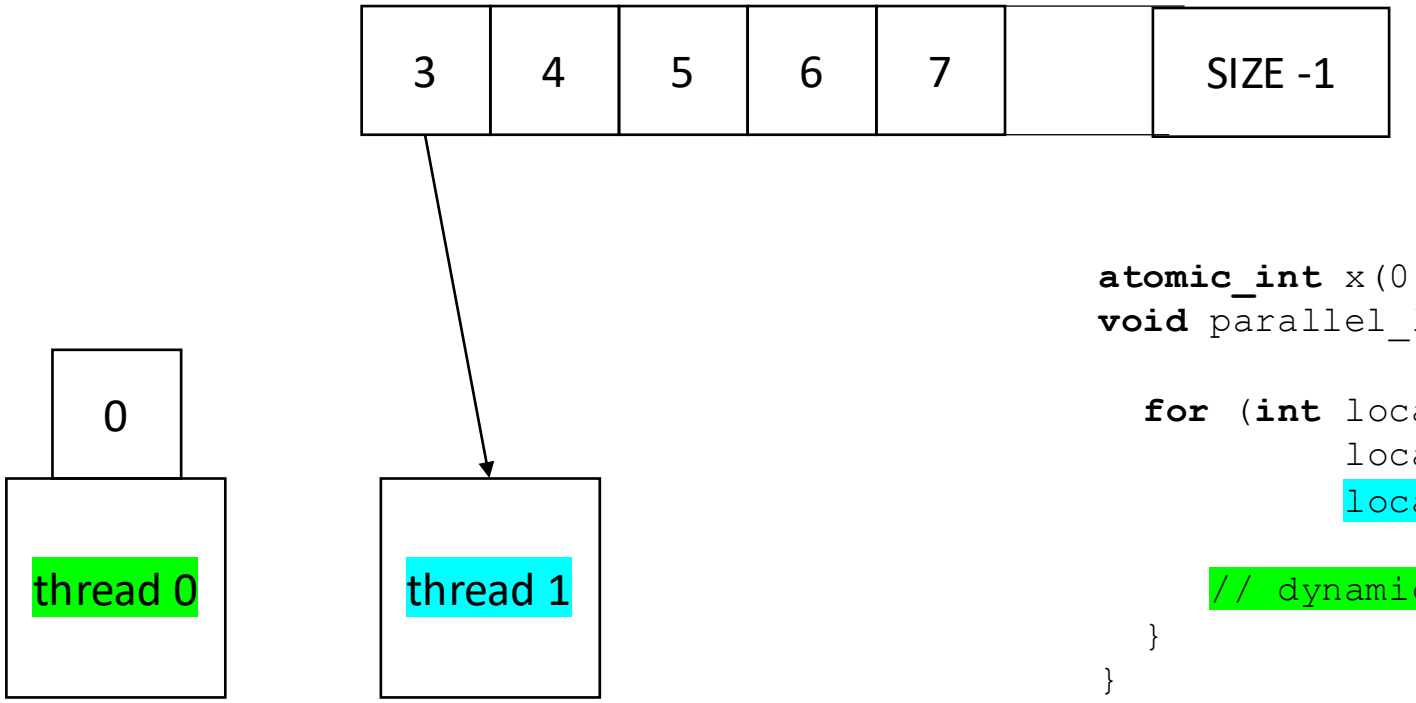

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```

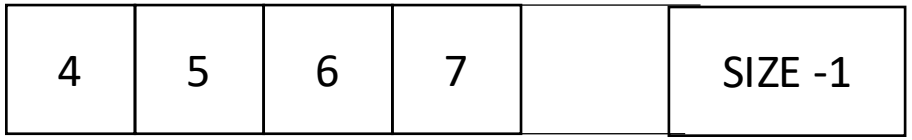
```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

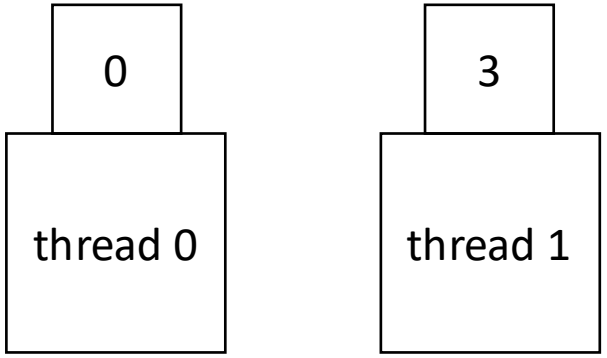
     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```



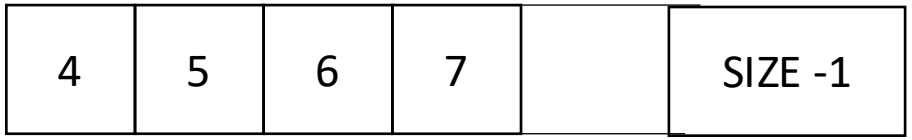

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

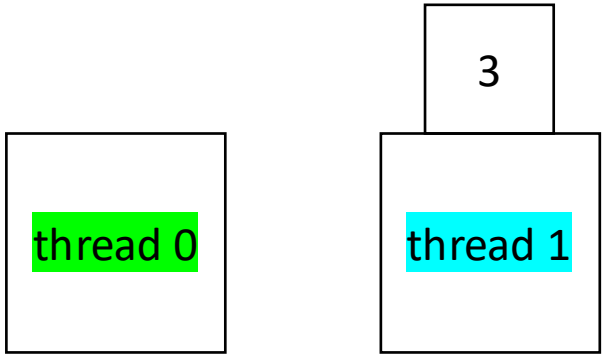
     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 4
0 - local_x - 0
1 - local_x - 3
```



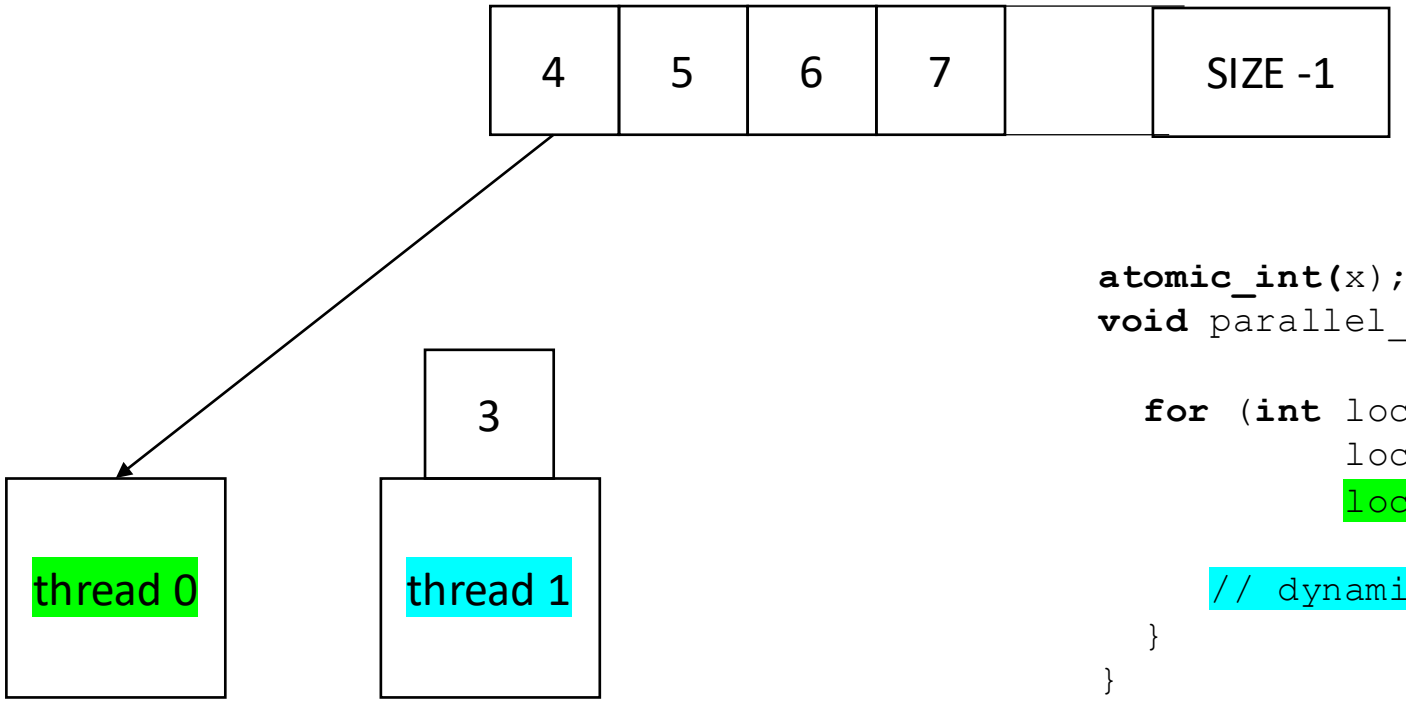

```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 5
0 - local_x - 4
1 - local_x - 3
```

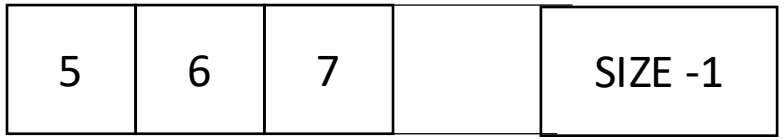
```
atomic_int(x);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

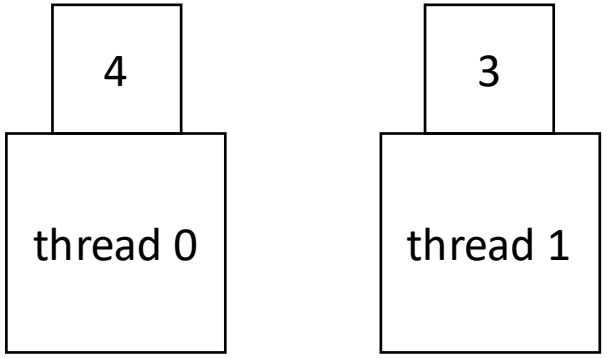
     // dynamic work based on x
  }
}
```

# Work stealing - global worklist

- Global worklist: threads take tasks (iterations) dynamically




```
x: 5
0 - local_x - 4
1 - local_x - 3
```




```
atomic_int x(0);
void parallel_loop(...) {

  for (int local_x = atomic_fetch_add(&x,1);
            local_x < SIZE;
            local_x = atomic_fetch_add(&x,1)) {

     // dynamic work based on x
  }
}
```

# Quiz 10.

Which one of the following is NOT a drawback of a global workstealing parallel schedule

*Select one*

A. Requires a concurrent data structure

B. Contention on shared cache lines

C. Contention on a single location with RMWs

Scan to answer

**forms.gle/HrhkkgQiYqML2jcB9**

chart: docs.google.com/forms/d/e/1FAIpQLSckAO0lpZ5Ng4HW5K7ZxxB4dclr8g6V9sTUHZoEJXz1CXRvHw/viewanalytics

# Quiz 10.

Which one of the following is NOT a drawback of a global workstealing parallel schedule

*Select one*

A. Requires a concurrent data structure

B. Contention on shared cache lines

C. Contention on a single location with RMWs

A concurrent data structure is needed.

Scan to answer

**forms.gle/HrhkkgQiYqML2jcB9**

chart: docs.google.com/forms/d/e/1FAIpQLSckAO0lpZ5Ng4HW5K7ZxxB4dclr8g6V9sTUHZoEJXz1CXRvHw/viewanalytics

# Schedule

- DOALL Loops

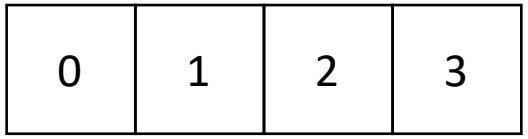
- **Parallel Schedules:**
  - Static
  - Global Worklists
  - **Local Worklists**

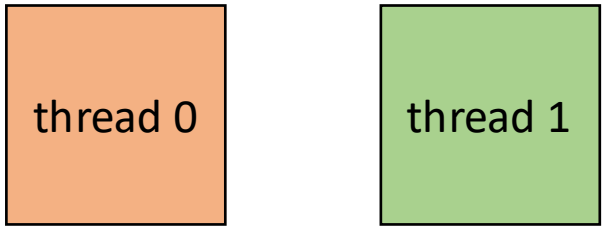
# Work stealing - local worklists

- More difficult to implement
- low contention on local data-structures
- potentially better cache locality

# Work stealing - local worklists

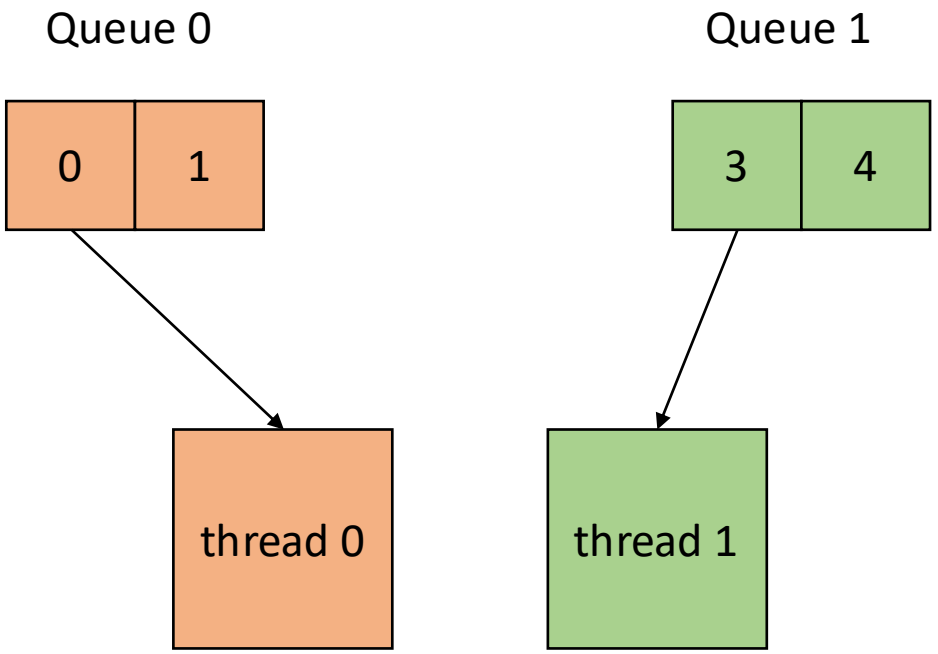
- local worklists: divide tasks into different worklists for each thread



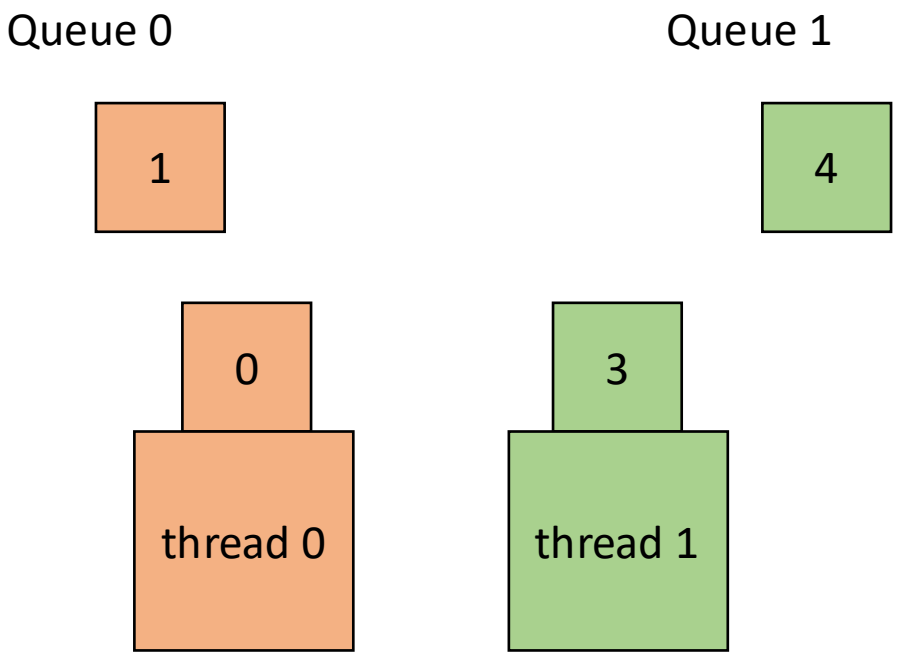
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



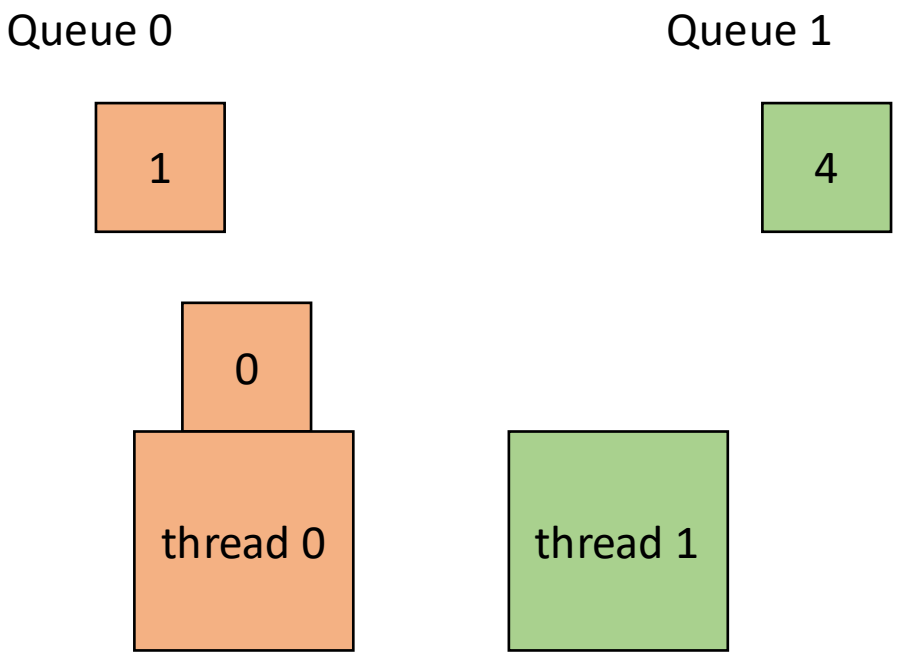
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



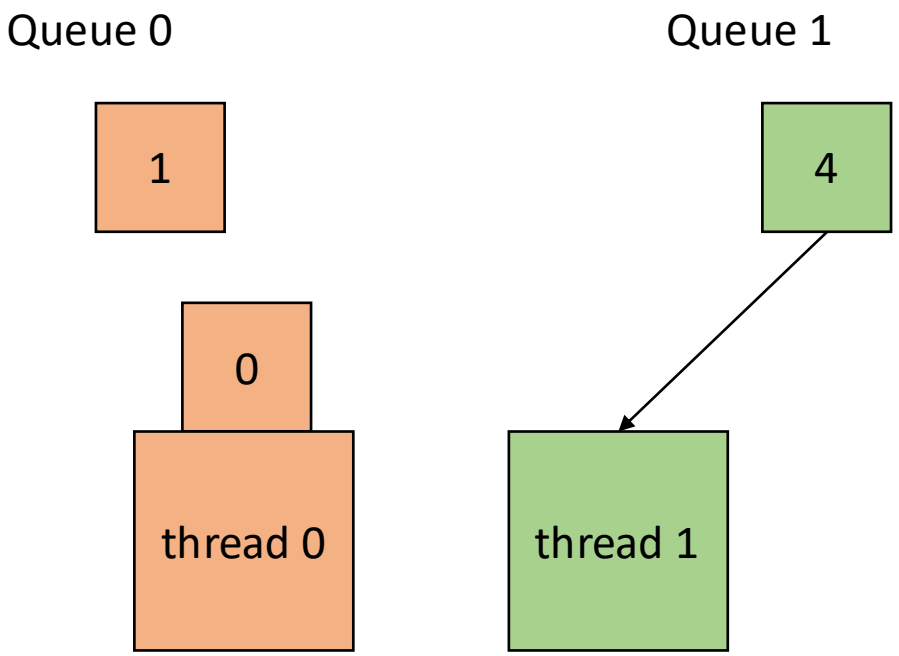
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



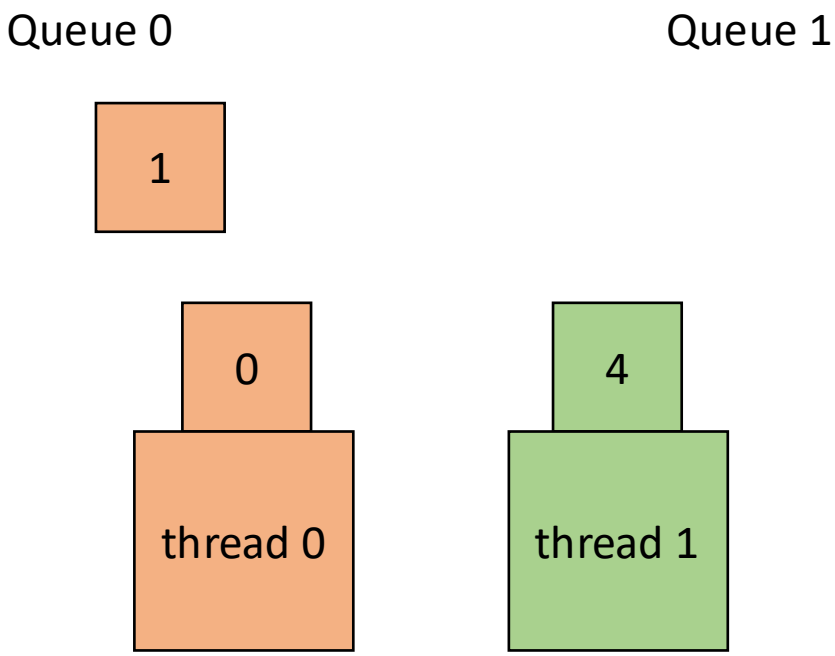
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



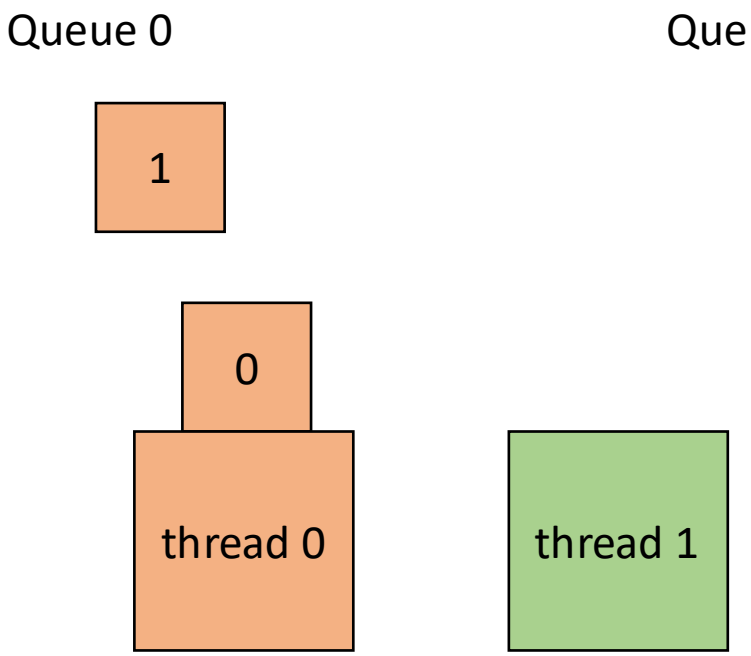
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



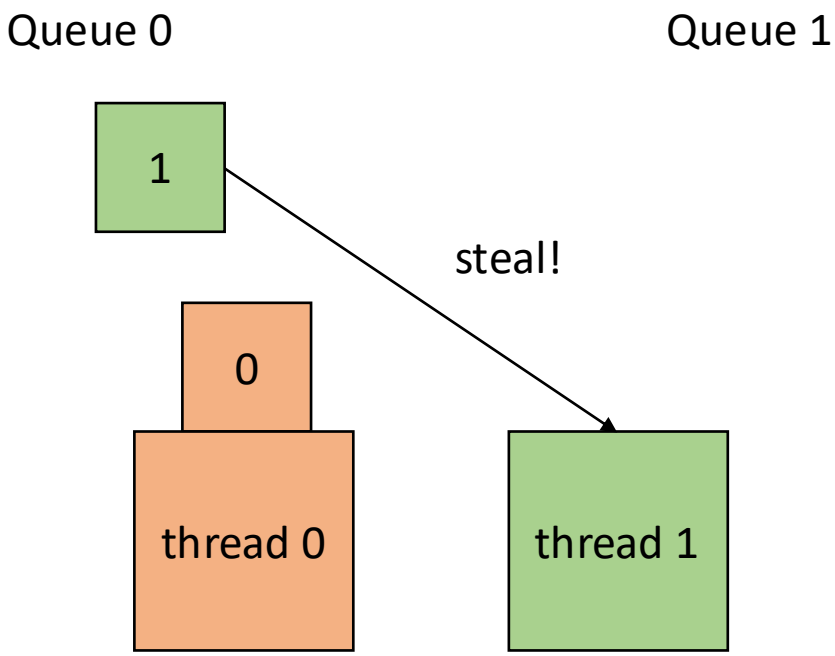
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread

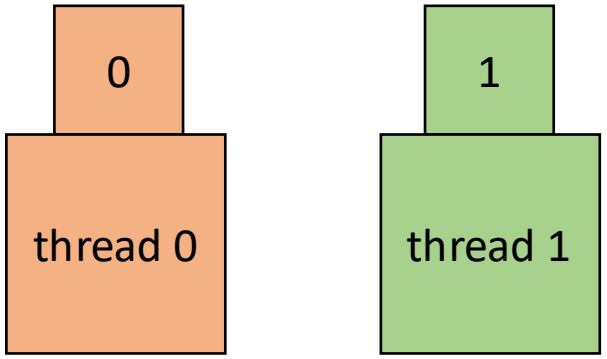


# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread

Queue 0

Queue 1



# Work stealing - local worklists

- How to implement:

```
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

# Work stealing - local worklists

- How to implement:

```
void foo() {
   ...
   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
   ...
}
```

```
void parallel_loop(..., int tid) {

   for (x = 0; x < SIZE; x++) {
     // dynamic work based on x
   }
}
```

Make a new function, taking any variables used in loop body as args. Additionally take in a thread id

# Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Make a global array of concurrent queues

# Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

What type of queues?

Make a global array of concurrent queues

# Work stealing - local worklists

- How to implement:

```
concurrent_queue cq[NUM_THREADS];
void foo() {
  ...
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
  ...
}
```

```
void parallel_loop(..., int tid) {

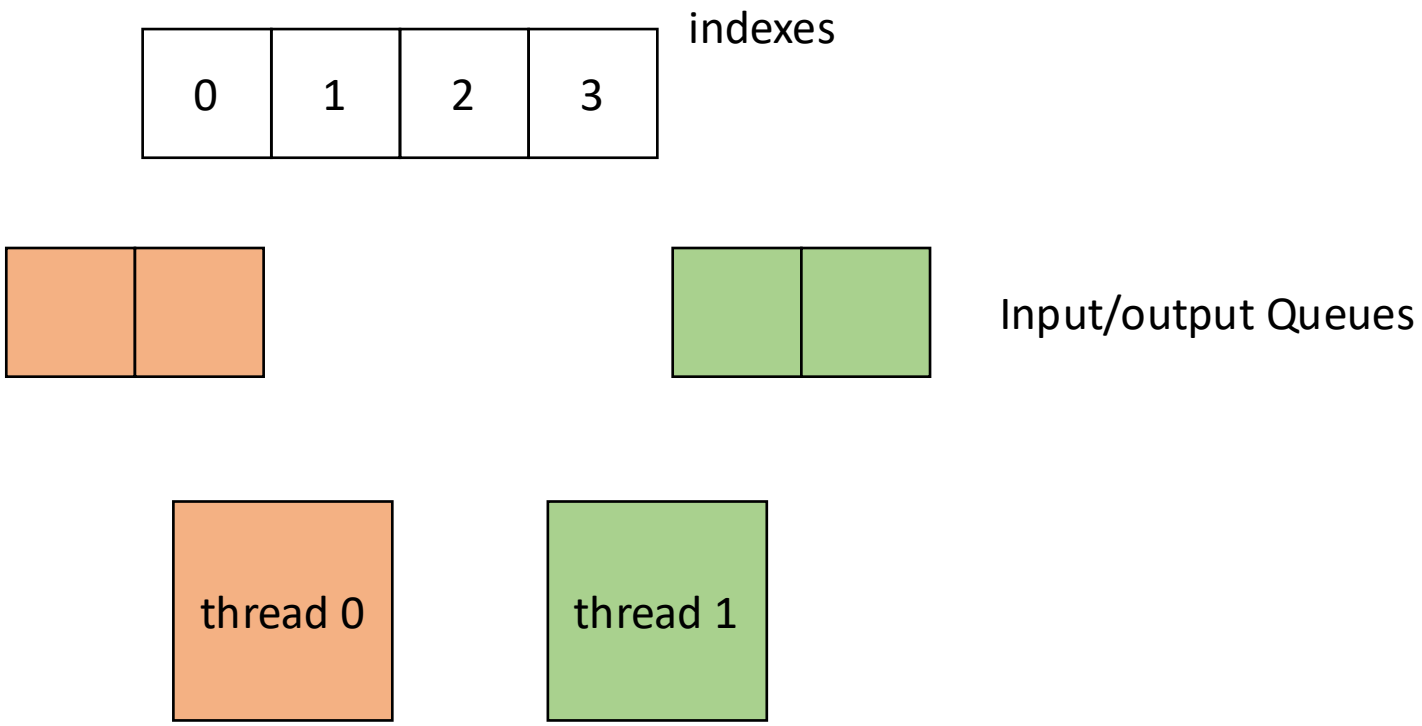
  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

Make a global array of concurrent queues

What type of queues?
We're going to use InputOutput Queues!

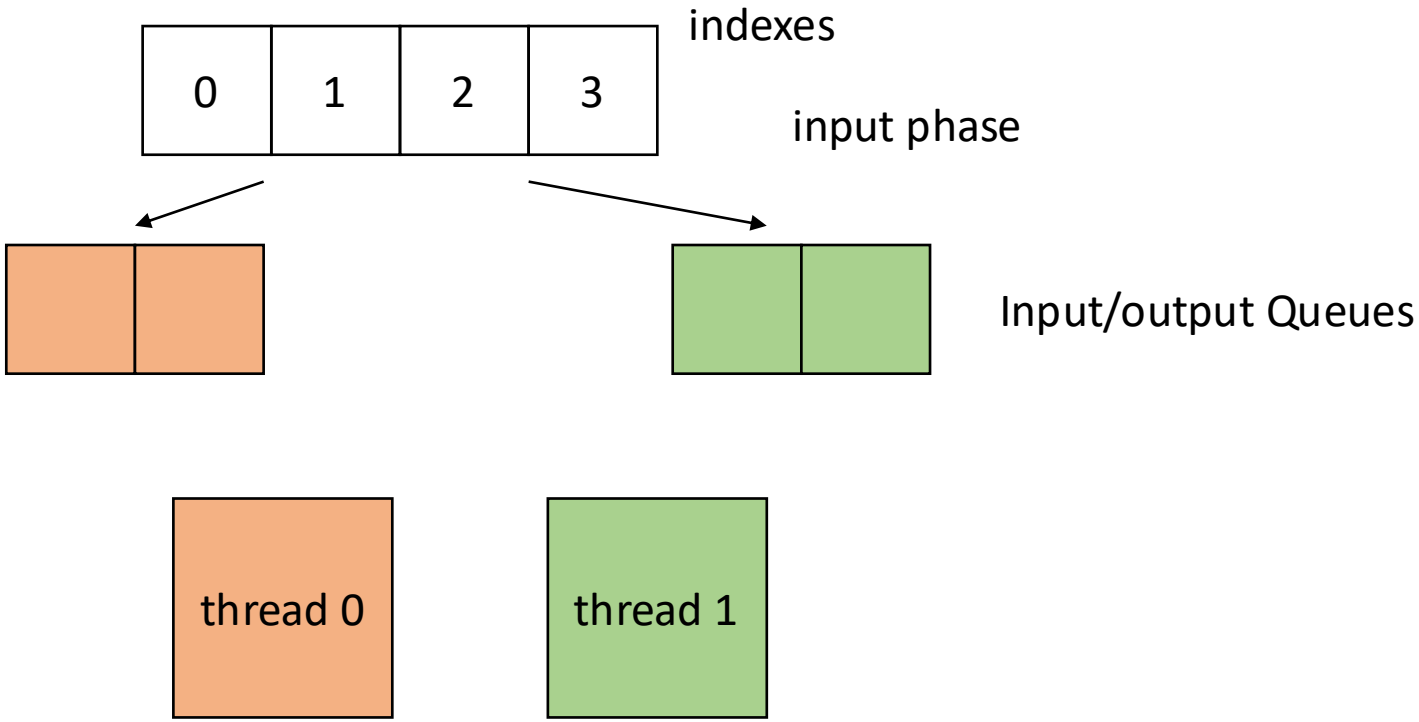
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



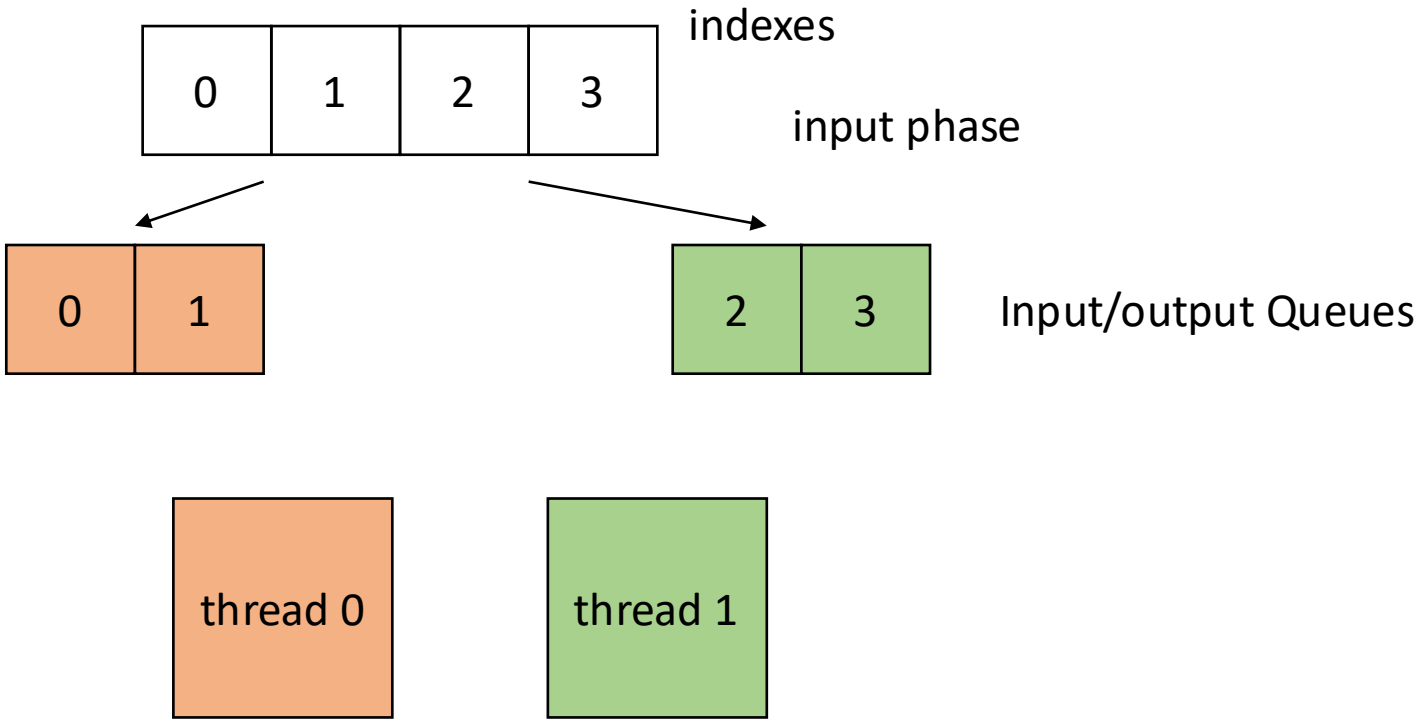
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



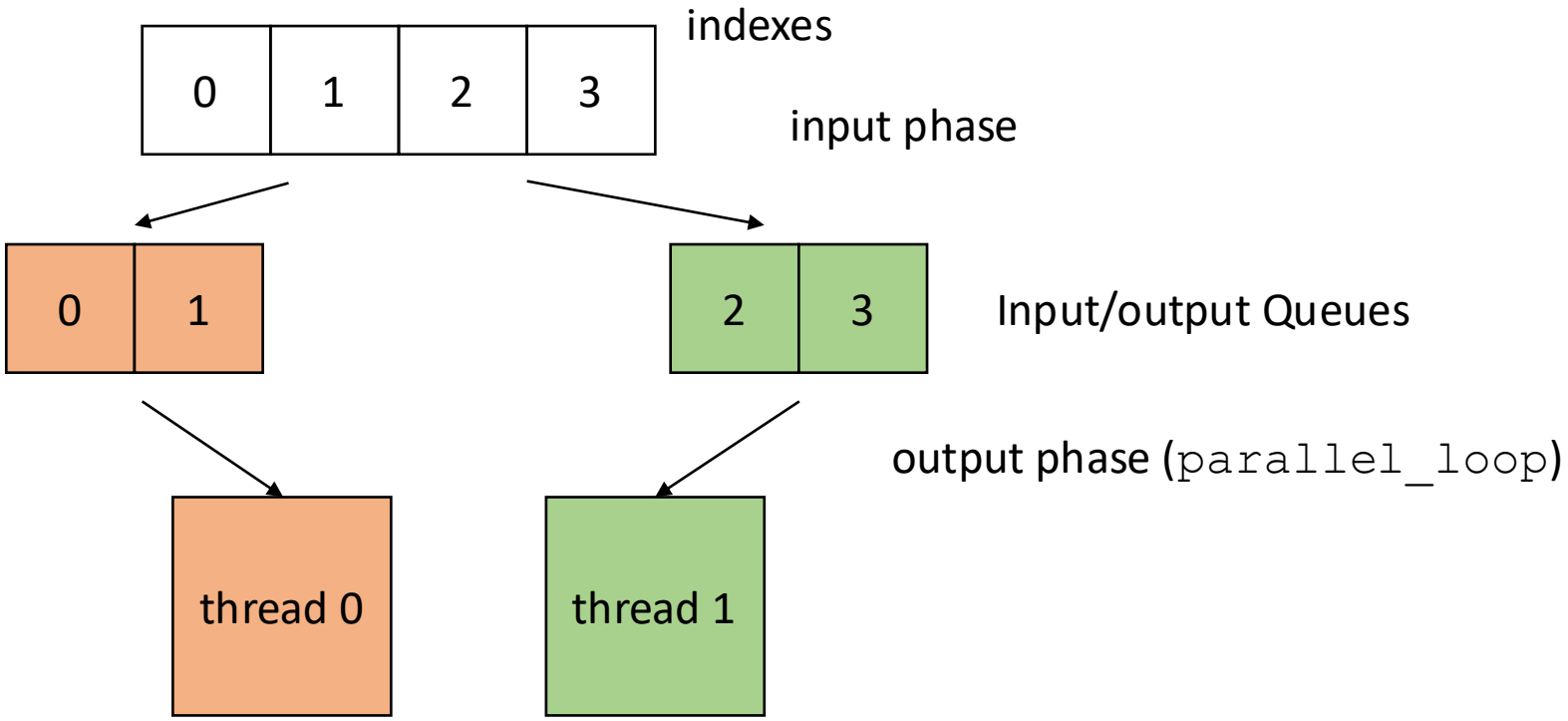
# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



# Work stealing - local worklists

- local worklists: divide tasks into different worklists for each thread



# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...


   ...
}
```

First, we need to initialize the queues

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // Spawn threads to initialize
   // join initializing threads

   ...
}
```

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // Spawn threads to initialize
   // join initializing threads

   ...
}
```

Make sure to account for boundary conditions!

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

# Work stealing - local worklists

- How to implement in a compiler:

```
NUM_THREADS = 2;
SIZE = 4;
CHUNK = 2;
```

| x | 0 | 1 | 2 | 3 |
|---|---|---|---|---|

| tid | 0 | 0 | 1 | 1 |
|---|---|---|---|---|

Make sure to account for boundary conditions!

```
void parallel_enq(..., int tid, int num_threads)
{

  int chunk_size = SIZE / NUM_THREADS;
  int start = chunk_size * tid;
  int end = start + chunk_size;
  for (int x = start; x < end; x++) {
    cq[tid].enq(x);
  }
}
```

Just like the static schedule, except we are enqueuing

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  for (x = 0; x < SIZE; x++) {
    // dynamic work based on x
  }
}
```

How do we modify the parallel loop?

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
}
```

loop until the queue is empty

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  ...
}
```

```
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
}
```

loop until the queue is empty
Are we finished?

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
      // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
}
```

Track how many threads are finished

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
     // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {

   }
}
```

While there are threads that are still working

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

   // launch loop function
   ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
     // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {
     int target = // pick a thread to steal from
     int task = cq[target].deq();
   }
}
```

pick a random target and steal a task

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
  ...
  // initialize queues
  // join threads

  // launch loop function
  // join loop threads
  ...
}
```

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
      // perform task
    }
  }
}
```

# Work stealing - local worklists

```
concurrent_queues cq[NUM_THREADS];
void foo() {
   ...
   // initialize queues
   // join threads

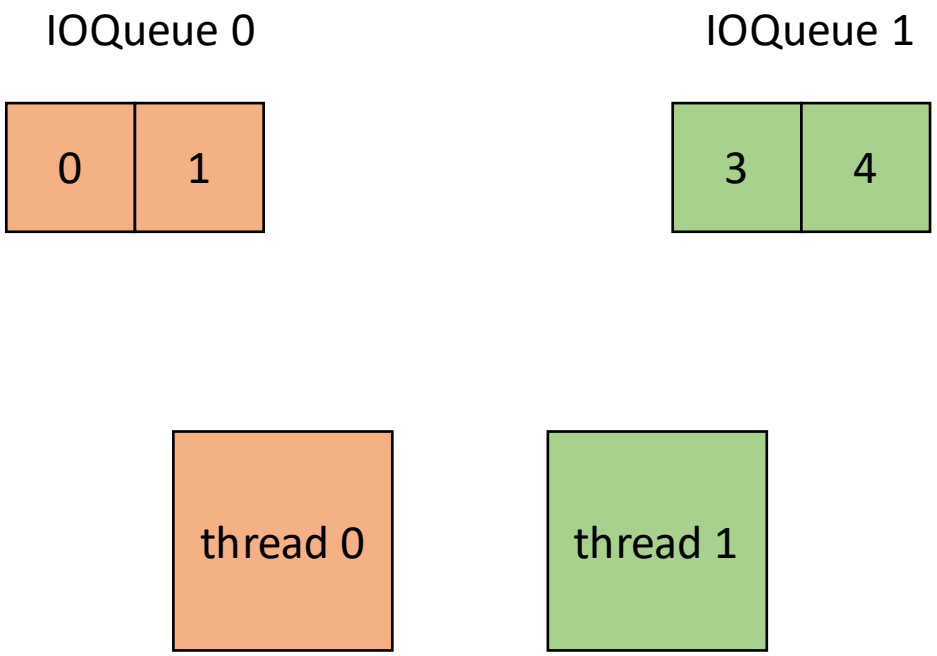
   // launch loop function
   // join loop threads
   ...
}
```

join the threads

```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

   int task = 0;
   for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
   {
      // dynamic work based on task
   }
   atomic_fetch_add(&finished_threads,1);
   while (finished_threads.load() != num_threads) {
      int target = // pick a thread to steal from
      int task = cq[target].deq();
      if (task != -1) {
         // perform task
      }
   }
}
```

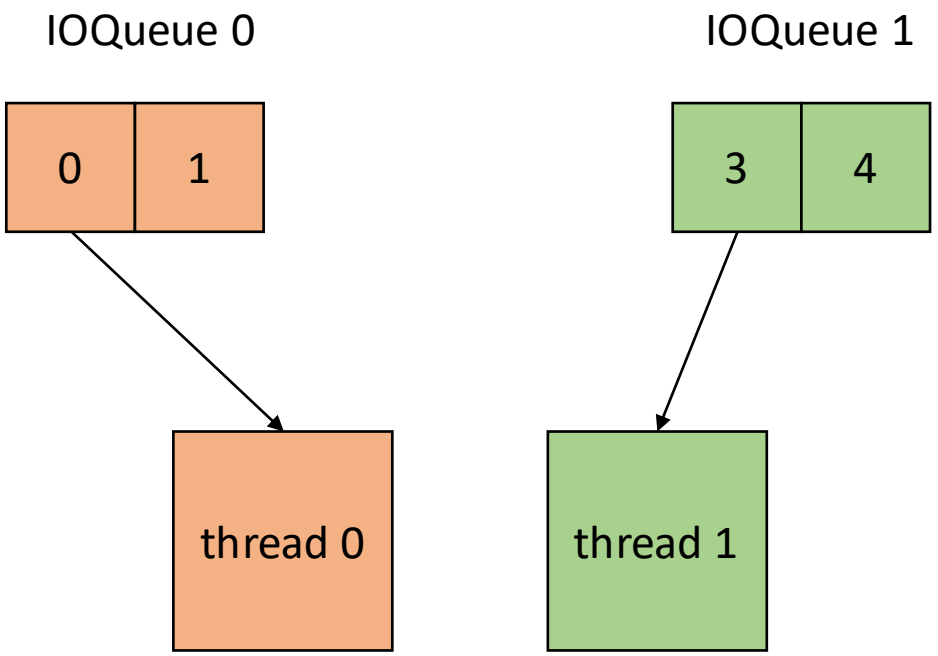
# Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

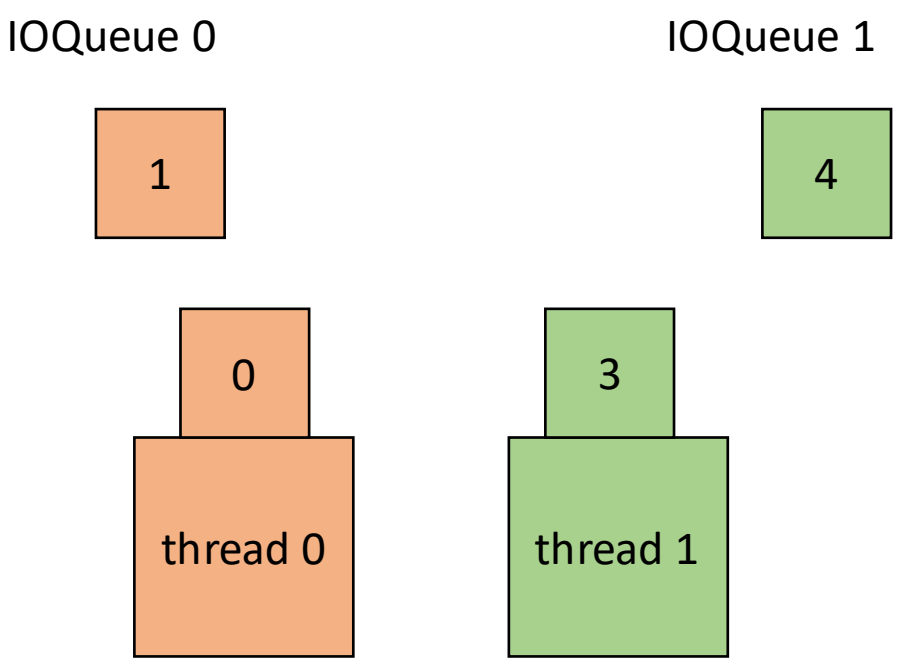
# Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

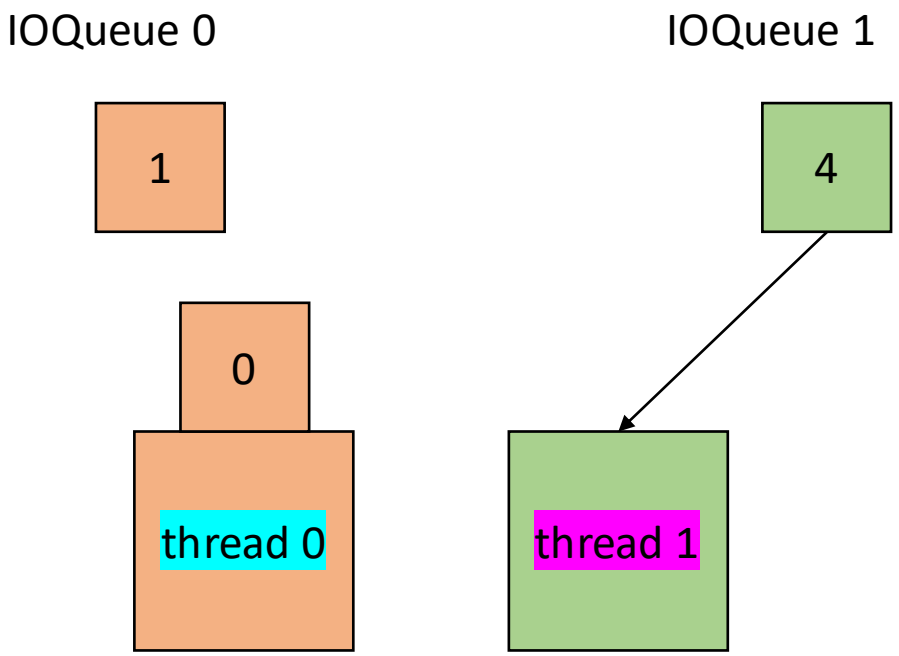
# Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

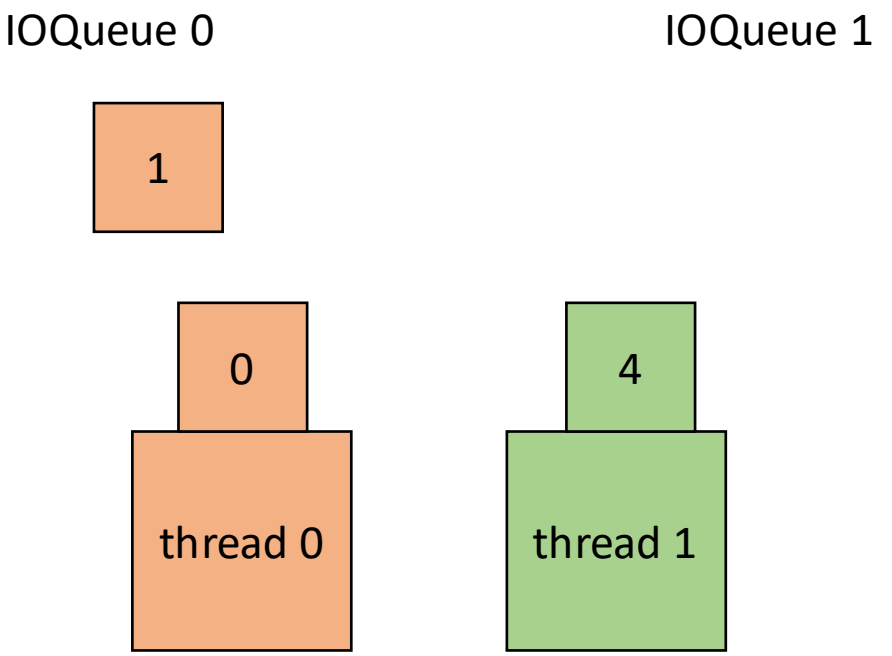
# Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

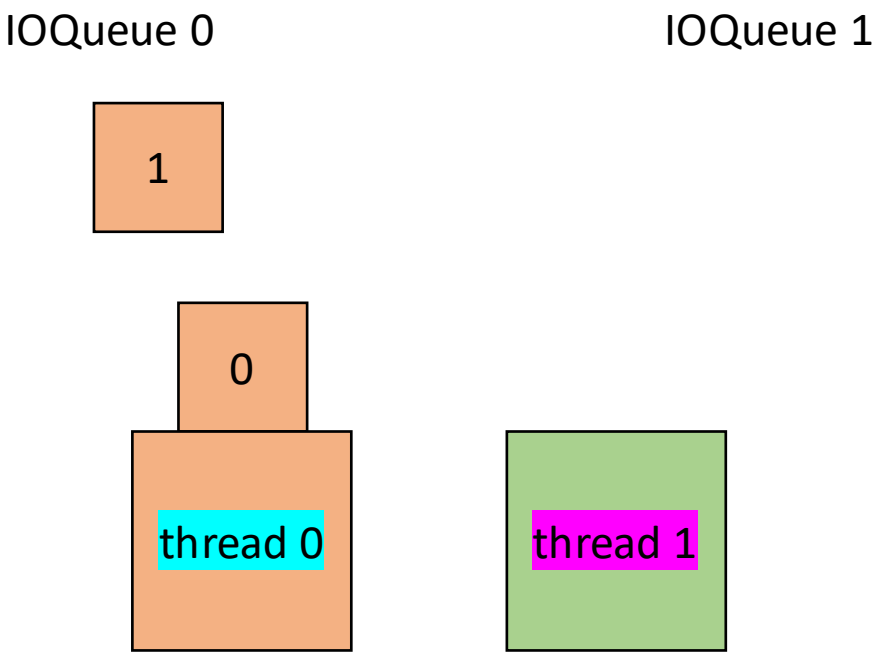
# Work stealing - local worklists




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

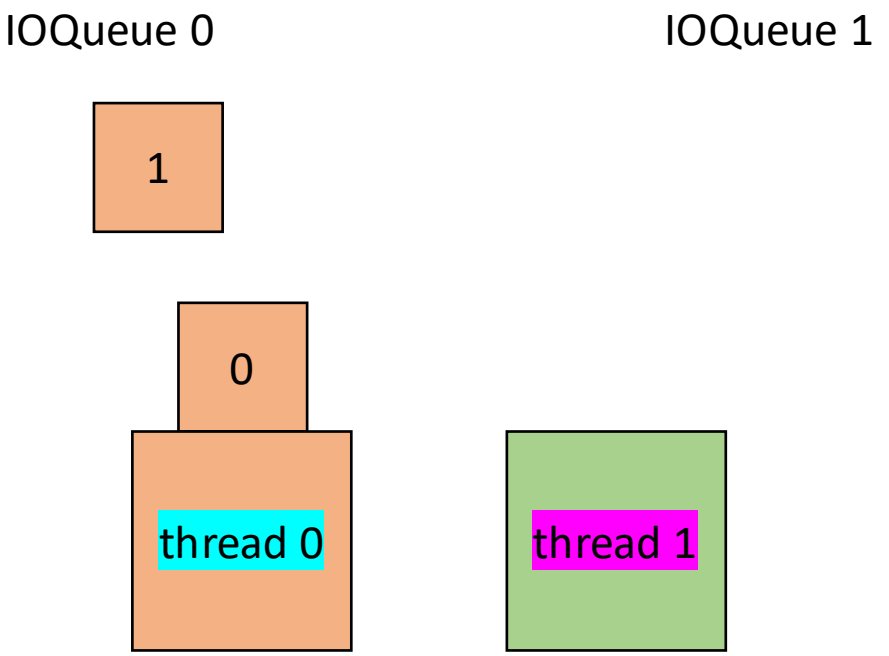



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

finished_threads: 1

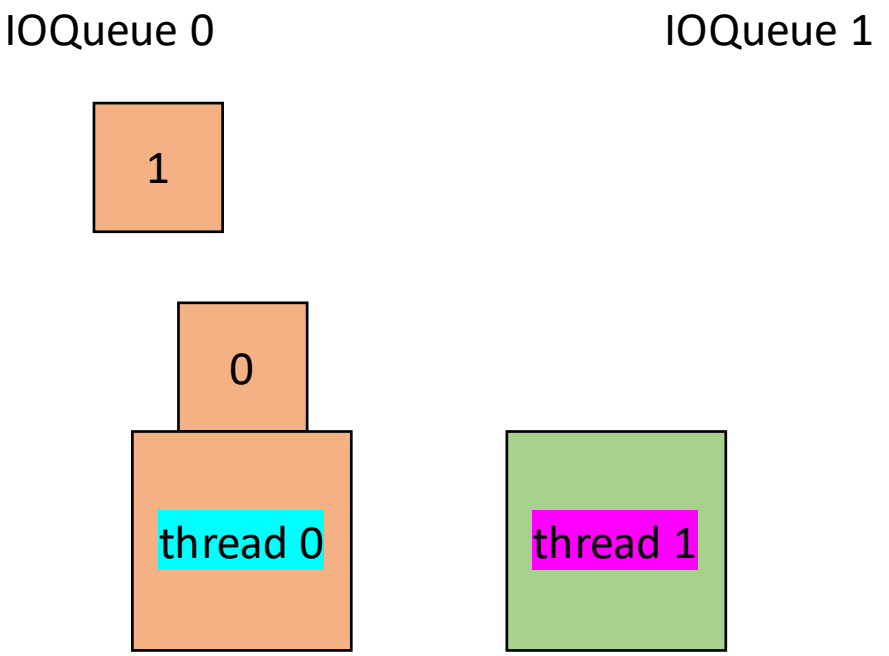



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

finished_threads: 1

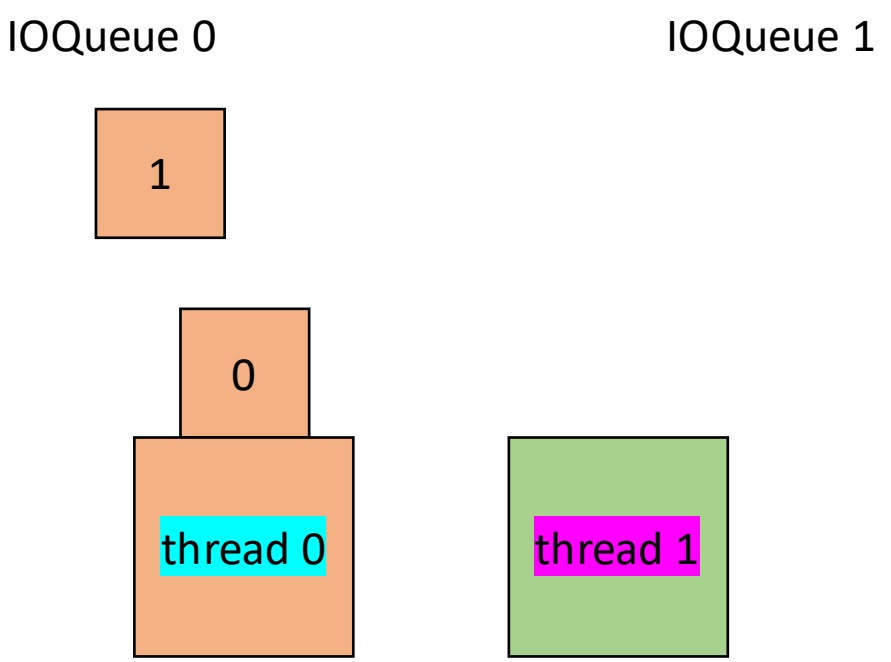



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

finished_threads: 1

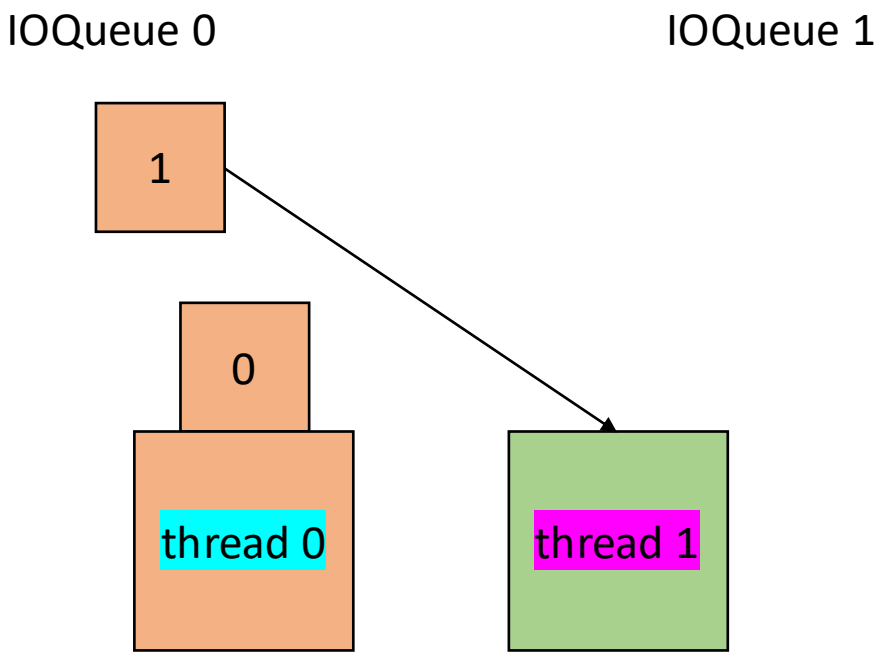



```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

finished_threads: 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

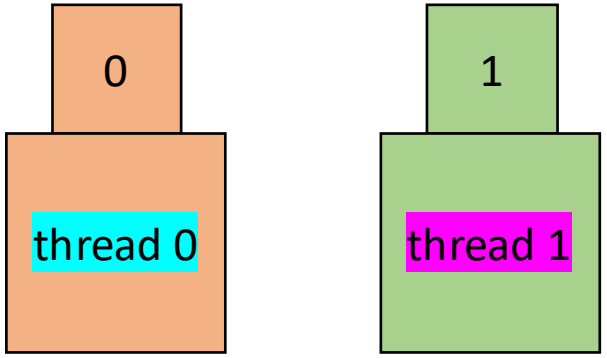
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

finished_threads: 1

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

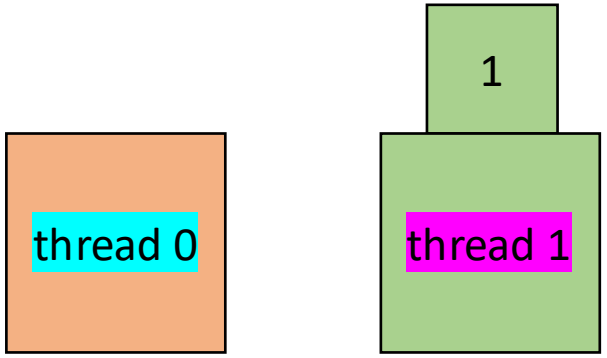
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

```
finished_threads: 1
```

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

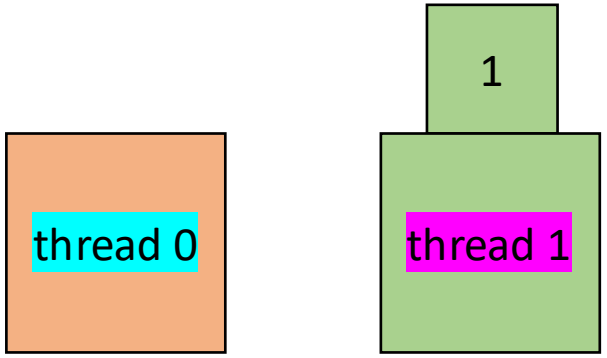
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

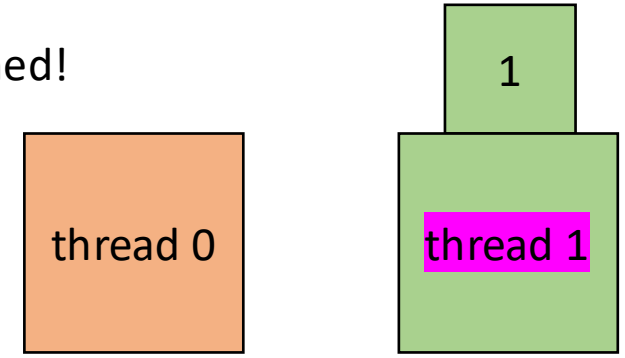
# Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1

finished!




```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

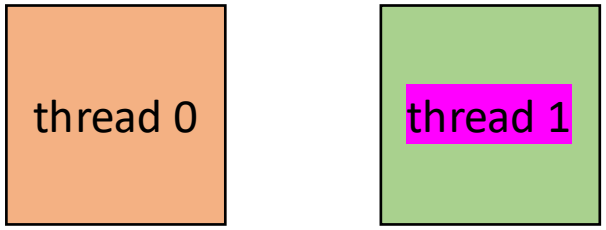
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1







```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

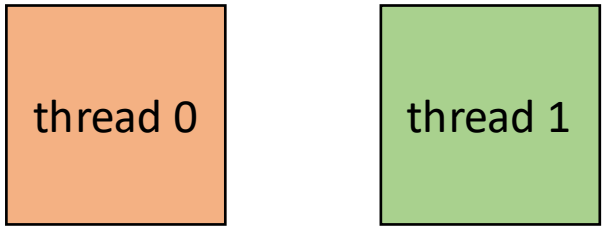
  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Work stealing - local worklists

finished_threads: 2

IOQueue 0

IOQueue 1







```
atomic_int finished_threads(0);
void parallel_loop(..., int tid, int num_threads) {

  int task = 0;
  for (x = cq[tid].deq(); x != -1; x = cq[tid].deq())
  {
    // dynamic work based on task
  }
  atomic_fetch_add(&finished_threads,1);
  while (finished_threads.load() != num_threads) {
    int target = // pick a thread to steal from
    int task = cq[target].deq();
    if (task != -1) {
       // perform task
    }
  }
}
```

# Quiz 10.

Which of the following is NOT an overhead of the local worklist workstealing parallel schedule.

*Select one*

A. Initialization of the queues

B. Checking a global variable to ensure all work is completed

C. Managing concurrent enqueues to the worklists

Scan to answer

**forms.gle/MmDQEFBLaqSTjkCm8**

chart: docs.google.com/forms/d/e/1FAIpQLSe6ULaZ2AbL0nqH2QodJl9cJWas3N6XlaAJ4JPpO7jzyawL5A/viewanalytics

# Quiz 10.

Which of the following is NOT an overhead of the local worklist workstealing parallel schedule.

*Select one*

A. Initialization of the queues

B. Checking a global variable to ensure all work is completed

C. Managing concurrent enqueues to the worklists

There were not concurrent enqueues.

Scan to answer

**forms.gle/MmDQEFBLaqSTjkCm8**

chart: docs.google.com/forms/d/e/1FAIpQLSe6ULaZ2AbL0nqH2QodJl9cJWas3N6XlaAJ4JPpO7jzyawL5A/viewanalytics

# Quiz 10.

Write a few sentences about the pros and cons of using local workstealing queues over the global worklist.

*Free response: a few sentences on the form*

Scan to answer

**forms.gle/VdDu2GrUrNz7noef9**

chart: docs.google.com/forms/d/e/1FAIpQLSf7H1ZVBLzzNO-8XiHyBSIcF56WEH8PW5BMuFJ-d8tDuC9fNw/viewanalytics

# Quiz 10.

Write a few sentences about the pros and cons of using local workstealing queues over the global worklist.

*Free response: a few sentences on the form*

Simpler vs. More efficient.

Scan to answer

**forms.gle/VdDu2GrUrNz7noef9**

chart: docs.google.com/forms/d/e/1FAIpQLSf7H1ZVBLzzNO-8XiHyBSIcF56WEH8PW5BMuFJ-d8tDuC9fNw/viewanalytics

# Open mp

- Code examples

# Next topic

- General concurrent set